

06 — สะพานเชื่อม: โหนด 05 ↔ โหนด 07 ↔ โค้ดจริง

โหนด 05 อธิบาย *ทฤษฎี* จากศูนย์ (สมการ (1)–(34)) และโหนด 07 ไล่ *ตัวเลข* ของตาราง 2×4 คุมหนึ่ง iteration · โหนดนี้เป็นสะพานระหว่างสองอย่างนั้นกับ**บรรทัดโค้ดจริง** ทำสื่อย่าง: (1) ตารางเชื่อม โหนด 05 → สมการ → โค้ด ทีละสิ่งที่เกิดขึ้น (§2) (2) สอนวิธีอ่าน notation ให้ได้ความหมายเชิงสัญชาตญาณ (§3) และกลไกระดับโค้ดของการปรับ weight — ℓ กับ $\cdot \text{grad}$, Adam, กฎ lr — พร้อมสมการ (35)–(37) ที่โหนด 05 ไม่ได้ derive (§3.4) (3) ไล่สายส่งค่าจาก observation ถึง gradient ทีละสมการ (E1–E15) พร้อมโค้ด 1:1 และ เวกเตอร์ 8 มิติจริง (§3.5) (4) errata ของเอกสารต้นทาง `sources/training_numerical_walkthrough.md` (§4) — **ทุกข้ออ้างในโหนดนี้ตรงกับโค้ดใน `tron1-r1-isaacrlab/` แล้ว**

ทุกคำ ตัวย่อ สัญลักษณ์ และเลขสมการ นิยามที่เดียวใน `00-glossary.md` — โหนดนี้ไม่มีนิยามซ้ำ (ตัวย่อ: หัวข้อ 1 · สัญลักษณ์: หัวข้อ 4–5 · คู่ที่สับสน: หัวข้อ 7 · คำทุกตัวของตาราง 2×4 : หัวข้อ 8) · ตัวเลข 8 มิติใน §3.5.5 คือ "การขยาย" ช่อง `e0 t0` ของตารางนั้น (อภิธานศัพท์หัวข้อ 8.6)

ไดอะแกรมประกอบหกชิ้น เปิดใน browser (สมการ render ด้วย MathJax): **A** `tron1-actor-critic.html` · **B** `tron1-ppo-iteration.html` · **C** `tron1-value-flow.html` · **D** `tron1-deploy.html` · **E** `tron1-policy-objects.html` · **F** `tron1-five-units.html`

ตัวย่อและสัญลักษณ์ — อยู่ในอภิธานศัพท์

โหนดนี้ใช้ตัวย่อตั้งแต่บรรทัดแรก ๆ (RL, PPO, GAE, TD, KL, MSE, lr, SGD, SNR, RNG, MLP, NN, ELU, VAE, ONNX, PD, IMU) — ทุกตัวย่อไว้ใน `00-glossary.md` หัวข้อ 1 พร้อมประโยคความันคืออะไรในบริบท Tron1 · Adam ไม่ใช่ตัวย่อเป็นชื่อเฉพาะของ optimizer (§3.4 อธิบายว่ามันทำอะไร) · ชนิดของวัตถุ 11 แบบ (ฟังก์ชัน · การแจกแจง · เวกเตอร์ · scalar · ...) ที่ทุกตารางในโหนดนี้ใช้ในคอลัมน์ "ชนิด" นิยามไว้ที่อภิธานศัพท์หัวข้อ 0

0. แผนที่ความสูงสามระดับ

การอ่านให้เข้าใจ "ทั้งหมด" ต้องสลับไปมาระหว่างสามระดับความสูง เอกสารแต่ละชิ้น อยู่คนละระดับ:

ระดับ	คำถามที่ตอบ	เอกสาร	ไดอะแกรม
ทฤษฎี	ทำไม ถึงต้องมีสิ่งนี้	โหนด 05	—
โครงสร้าง	อะไร คุยกับ อะไร	โหนด 05 ภาค 2, 4	A — Actor-Critic (architecture)
ขั้นตอน	เกิด ตามลำดับ ยังไงใน 1 iteration	โหนด 05 §4.7, โหนด 07 §0	B — PPO iteration (flowchart) · F — ห้าหน่วยนับ
ตัวเลข	shape เท่าไหร่ ค่าเท่าไหร่	โหนด 07 (ตาราง 2×4 ทุกระดับ) · §3.5.5 ของโหนดนี้ (เวกเตอร์ 8 มิติ)	—
โค้ด	บรรทัดไหน	ตาราง §2 ด้านล่าง	—

วิธีใช้: เจอสัญลักษณ์ที่ไม่เข้าใจ → เปิดอภิธานศัพท์ (ชนิดก่อน ค่าทีหลัง) → ตามลิงก์ไป โหนด 05 ที่ derive มัน → กลับมาดูตัวเลขในโหนด 07 หรือ §3.5.5 อีกครั้ง ความเข้าใจจะ "ล็อก" เมื่อสามระดับตรงกัน

1. แก่ก่อนอย่างอื่น: ปริศนา 42 vs 39 — ไหแล้ว

ถ้านับจากโน้ต 03 จะได้ $36 + 3 = 39$ ไม่ใช่ 42 — แต่ตัวเลข 42 ถูกต้อง เพราะมีก้อนที่สามที่โน้ต 03 ไม่ได้นับรวมใน "36":

```
# tron1-rl-isaacrl/rsl_rl/rsl_rl/algorithm/ppo.py:138
torch.cat((encoder_out, obs, commands), dim=-1)
#           —3—      -36-   —3—      = 42 ← actor input

# ppo.py:134
torch.cat((critic_obs, commands), dim=-1)
#           —227—      —3—      = 230 ← critic input
```

commands คือ velocity_commands = เป้าหมายความเร็ว (v_x, v_y, v_z) ที่ ผู้ใช้สั่ง — 3 ค่า มันเป็น **observation group แยก** ชื่อ "commands" ใน ObservationsCfg (limx_base_env_cfg.py:227) ไม่ได้อยู่ใน group "policy" ที่มี 36 ค่า โน้ต 03 นับแค่ group policy จึงได้ 36

ก้อน	มิติ	มาจาก	ใครเห็น
latent $\hat{\mu}$ (encoder output)	3	obsHistory h 360 $\rightarrow$ MLP $\rightarrow$ 3	actor
policy obs	36	IMU + joint encoder + gait clock	actor
commands	3	เป้าหมายความเร็วที่สั่ง	ทั้ง actor และ critic
critic obs (privileged)	227	ซิมรู้ทุกอย่าง	critic

$$\text{actor input} = 3 + 36 + 3 = 42 \qquad \text{critic input} = 227 + 3 = 230$$

โน้ต 05 หัวข้อ 2.2 และ 4.4 ใช้ตัวเลขนี้แล้ว · ตัวเลข 227 เป็นความกว้างของ group critic ตอนรับ (on_policy_runner.py:61) ยืนยันจาก checkpoint จริง (in_features=230 ของชั้นแรกของ critic)

2. ตารางเชื่อมสามชั้น: สิ่งที่เกิดขึ้น $\rightarrow$ โน้ต 05 $\rightarrow$ สมการ $\rightarrow$ โค้ด

อ่านแนวนอน: หนึ่งแถว = หนึ่งสิ่งที่เกิดขึ้น มองจากสี่มุม · คอลัมน์แรกคือหมายเลขชั้นใน sources/training_numerical_walkthrough.md (เอกสารต้นทาง — ใช้เป็นดัชนีเท่านั้น ตัวเลขในนั้นอย่าเชื่อ)

ช่วง A — เก็บข้อมูล (โน้ต 07 §2 · sources §1)

sources §	ทำอะไร	shape	โน้ต 05	สมการ	โค้ด
1.1	อ่านเซนเซอร์ 4 ก้อน	$(B, 36) \quad (B, 3)$ $(B, 360) \quad (B, 230)$	2.1 ใครเห็นอะไร · 1.3 POMDP	—	ObservationsCfg
1.2	encoder: ประวัติด h $\rightarrow$ latent $\hat{\mu}$	$(B, 360) \rightarrow (B, 3)$	4.4	—	ppo.py:136
1.3	concat เป็น actor input	$(B, 42)$	2.2 · §1 ข้างบน	—	ppo.py:138
1.4	actor MLP $\rightarrow \mu$	$(B, 42) \rightarrow (B, 8)$	2.2 node/layer · ทำไ้ ชั้นสุดท้ายไม่มี activation	(10) (11)	actor_critic.py:69-90

sources §	ทำอะไร	shape	ไม้ต 05	สมการ	โค้ด
1.5	สุ่ม $a = \mu + \sigma \epsilon$ · คำนวณ $\log \pi$	$(B, 8) \cdot (B, 1)$	2.1 (9) · 2.3 $\mu/a/\pi$ · 2.4 ทำไม้ตต้องสุ่ม	(13) (14) (15) (16)	actor_critic.py:155-164
1.6	critic $\rightarrow V(s)$	$(B, 230) \rightarrow (B, 1)$	2.1 · 1.6 · 4.2	(3)	actor_critic.py:170-172
1.7	$\times 0.25 \rightarrow PD \rightarrow$ physics $\rightarrow$ reward	$(B, 8) \rightarrow (B, 1)$	2.5 ท่อ 5 ชั้น	(17) (18)	joint_actions.py:134,160
1.8	เก็บลง buffer $\times 24$	$(24, B, \cdot)$	4.7 ช่อง A	—	rollout_storage.py:131 <code>add_transitions</code>

ช่วง B — advantage (ไม้ต 07 §4 · sources §2)

sources §	ทำอะไร	shape	ไม้ต 05	สมการ	โค้ด
2.1	bootstrap $V(s_{24})$	$(B, 1)$	3.2 bootstrapping · 4.5 · 4.7 ช่วง B	(4)	on_policy_runner.py:221-225 $\rightarrow$ ppo.py:176 $\rightarrow$ rollout_storage.py:187
2.2	ไล่ย้อน $t = 23 \rightarrow 0: \delta_t, A_t, R_t$	$(24, B, 1)$	3.2 (8) · 4.5	(19) (26) (27)	rollout_storage.py:187-201
2.3	normalize $\hat{A}_t = (A_t - \bar{A})/\text{std}(A)$	$(24, B, 1)$	4.5	(28)	rollout_storage.py:204-206

ช่วง C — อัปเดต (ไม้ต 07 §5 · sources §3)

sources §	ทำอะไร	shape	ไม้ต 05	สมการ	โค้ด
3.0	flatten $\rightarrow 4$ minibatch $\times 5$ epochs	$(49152, \cdot) \rightarrow (12288, \cdot)$	4.7 ทยวิภาค · ทำไม้ต 24 step	—	rollout_storage.py:222 <code>mini_batch_generator</code>
3.1	$\rho = \exp(\log \pi_{new} - \log \pi_{old})$	$(MB, \cdot)$	3.3 importance sampling · 4.6 (ก)	(20) (21)	ppo.py:252
3.2	clipped surrogate	scalar	4.1 (23) · 4.6 (ก)	(29)	ppo.py:256-260
3.3	value loss (clipped MSE)	scalar	4.3 critic เรียน จาก reward · 4.6 (ง)	(32)	ppo.py:263-269
3.4	entropy H	scalar	4.6 (ก) · 2.4 σ	(31)	actor_critic.py:151-153
3.5	รวม loss $\rightarrow$ backward $\rightarrow$ Adam	—	2.2 gradient descent · 4.6 (จ)	(33) (12) · (36) §3.4	ppo.py:287-290
3.6	KL-adaptive lr ($\div 1.5 / \times 1.5$)	—	4.6 (ข)	(30) · (37) §3.4	ppo.py:236-244

ตารางสัญลักษณ์ครบชุด (ชนิด · shape · โค้ด · ค่าใน toy · ที่นิยาม) อยู่ทีอภีธานศัพท์หัวข้อ 3–5 โ้บ้ตนี้่ไม่ค้ดล่อกมา · หกตัวที่ §3.4–3.5 ใช้ที่่ที่สุด:

สัญลักษณ์	ชนิด	e0 t0	อภิธานศัพท์	derive ที่โ้บ้ต 05
s	เวกเตอร์ (42,) = $[\hat{v}; o; c]$	—	หัวข้อ 4	2.2, 4.4
$\pi_\theta(a \mid s) \cdot \log \pi$	scalar ต่อ env	0.7365 · -0.3058	หัวข้อ 4	2.3 สมการ (13)–(15)
$\hat{A}_t$	scalar ต่อช่อง (normalize ทั้งตารางแล้ว)	+0.7975	หัวข้อ 5	4.5 สมการ (28)
ρ_t	scalar ต่อช่อง	1.0356 (หลัง μ +0.05)	หัวข้อ 5	3.3 สมการ (21)
$\ell \cdot \sigma = e^\ell$	เวกเตอร์ (8,) — ลูกบิ่ด · ความกว้าง	-0.6931 · 0.5	หัวข้อ 4	2.1 สมการ (9)
L	scalar ตัวเดียวทั้งระบบ — เข้ม	—	หัวข้อ 5	4.6 สมการ (33)

3.3 คู่ที่่สับสน — และวิธีอ่านสมการยาว ๆ

คู่ที่่หน้าตาคล้ายแต่คนละความหมาย ($\theta/\phi/\psi$, $\pi/\pi(a \mid s)$, V/V_ϕ , $A/A_t/\hat{A}_t$, $R(s, a)/R_t/G_t/r_t$, $\mathbb{E}/\text{mean}()$, $\sim/=/\approx$, $\epsilon/\epsilon/\text{eps}$, L/ℓ , $z_i/\hat{v}$, T) รวมไว้ที่่เดียวใน อภิธานศัพท์หัวข้อ 7 พร้อมกฎของโ้บ้ตชุดนี้่สำหรับแต่ละคู่ — เป้ดตารางนั้นทุกครั้งที่่สะด

วิธีอ่านสมการยาว ๆ — ทำสามรอบ · ใช้สมการ (29) ในรูปของเปเปอร์เป็นตัวอย่าง:

$$L^{CLIP} = \mathbb{E} \left[\min \left(\rho_t \hat{A}_t, \text{clip}(\rho_t, 1 - \epsilon, 1 + \epsilon) \hat{A}_t \right) \right]$$

รอบ	ทำอะไร	ผลที่่ได้
1. หาโครงสร้าง	มองข้ามตัวห้อย/ตัวยกทั้งหมด หา operator หลัก	$\mathbb{E} [\min (_, _)]$ — "เฉลี่ยของค่าที่่น้อยกว่าระหว่างสองอย่าง"
2. เติมข้อ	แทนสัญลักษณ์ด้วยค่าจากตาราง	"เฉลี่ยของ $\min(\text{ratio} \times \text{advantage}, \text{ratio-ที่่ถูกตัด} \times \text{advantage})$ "
3. ถามว่าทำไ้	แต่ละส่วนป้องกันอะไร	$\min$ = เลือกทางที่่ให้รางวัลน้อยกว่า = กันไม่ให้ actor โลกเกิน · clip = ขอบ $\pm 20\%$

หลังรอบ 3 สมการจะกลายเป็นประโยค: "ให้รางวัล action ที่่ดี แต่ถ้า policy เปลี่ยน เกิน 20% แล้ว หยุ่ดให้" — นี่คือความหมายเชิงสัญชาตญาณ และคือสิ่งที่่โค้ด [ppo.py:256-260](#) ทำจริง

3.4 ศัพท์ของการเรสู — L, ℓ , gradient, optimizer

โ้บ้ต 05 นิยาม loss (§2.2 และ (33)), gradient descent (12), และสามพจน์ของ PPO ((29)–(32)) ไว้แล้ว — หัวข้อนี้่ไม่นิยามซ้ำ แต่ลงไปที่่กลไกระดับโค้ดที่่โ้บ้ต 05 ไม่ได้ derive: ℓ กับ `.grad` เป็นคนละกล่อง, สมการ (35) $\partial L / \partial \ell$, (36) กำของ Adam, (37) กฎปรับ lr และพฤติกรรมของมินตลอดการเรสู · ถ้าอ่าน §3.5.4 แล้วสะดุดคำว่า "loss" "gradient" "Adam" ให้ย้อนกลับมาที่่นี้่

ภาพก่อนสัญลักษณ์: การเรสูคือการหมุนลูกบิ่ด

บิ่กถึงเครื่องเสียงที่มีลูกบิ่ด 469,905 ตัว และมีเข้มวัดหนึ่งอัน เข้มบอก "เสียงเพี้ยน แค้ไหน" · งานของเราคือหมุนลูกบิ่ดทุกตัวจนเข้มลงต่ำที่่สุด

- ลูกบิ่ด คือตัวเลขที่่ปรับได้ — weight ของ actor (187,272) + weight ของ critic (282,625)
- `logstd` (8) = 469,905

- **เข็ม** คือตัวเลขตัวเดียวที่บอกว่าตอนนี้แยแค่ไหน — ตัวนี้แหละที่เรียกว่า L
- **การเทรน** คือการหมุนลูกบิดทีละนิด ๆ หลายล้านครั้ง

ทำไมไม่ใช่ 595,604 — Tron1 มีพารามิเตอร์ที่เรียนได้ทั้งหมด 595,604 ตัวก็จริง แต่ **encoder (125,699 ตัว) มีเครื่องเสียงของตัวเองแยกอีกเครื่อง** — optimizer คนละตัว (`ppo.py:88-89`) เข็มคนละอัน (`extra_loss`) และ `.step()` คนละบรรทัด (`:321`) • ที่เด็ดขาดกว่านั้นคือ `encode()` คำนวณแบบ `.detach()` เมื่อ `output_detach = True` (`mlp_encoder.py:95-100`, ตั้งไว้ที่ `limx_rsl_rl_ppo_cfg.py:109`) — แปลว่า encoder **ไม่ได้อยู่ในกราฟของ loss เลยด้วยซ้ำ** • ตลอดหัวข้อนี้ "ลูกบิด" หมายถึง 469,905 ตัวของเครื่องแรกเท่านั้น

ทุกคำที่เหลือในหัวข้อนี้เป็นชื่อของชิ้นส่วนในภาพนี้

L — เข็มวัดความแย (loss)

L คือ **ตัวเลขตัวเดียว** (scalar) ที่บอกว่า "ตอนนี้ทำได้แยแค่ไหน" • ยิ่งน้อยยิ่งดี

ทำไมเรียก loss ไม่ใช่ score — วงการ optimization ตกลงกันว่า **ad** ค่าเสมอ ไม่ใช่เพิ่ม • อะไรที่อยากได้มาก ๆ ก็แค่ใส่เครื่องหมายลบข้างหน้าแล้วกลายเป็นของที่ต้องลด — จะเห็นลูกเล่นนี้ทั้งใน L^{CLIP} และในพจน์ entropy ของ PPO

L ไม่ได้ตกลงมาจากฟ้า — **เราออกแบบมันเอง** • ของ PPO มีสามพจน์บวกกัน (สมการ (33), derive ทีละพจน์ในโบด 05 §4.6) ซึ่งท้ายหัวข้อนี้จะทวนสั้น ๆ ในมุมมองโค้ด

ℓ กับ L — หน้าตาเหมือนกัน คนละเรื่องสิ้นเชิง

นี่คือจุดที่สับสนที่สุดในโบดนี้ และมันเป็นความผิดของตัวอักษรล้วน ๆ

เขียน	อ่านว่า	คืออะไร	ขนาด	เป็นของใคร
L	"แอล ใหญ่"	loss — เข็มวัดความแย	scalar ตัวเดียว	ทั้งระบบมีอันเดียว
ℓ	"เอล เล็ก"	logstd — ลูกบิดคุมความกว้างการสุ่ม หนึ่งตัวต่อหนึ่งข้อต่อรวม 8 ตัว	เวกเตอร์ (8,)	8 ตัวนี้เป็นส่วนหนึ่งของลูกบิด 469,905 ตัว

ℓ เป็น **ลูกบิด** • L เป็น **เข็ม** • ลูกบิดตัวหนึ่งไปขยับเข็ม ไม่ใช่ของชนิดเดียวกันเลย

ความสัมพันธ์กับ σ ที่เห็นในตารางสัญลักษณ์:

$$\sigma = \exp(\ell)$$

ℓ **คืออะไรแบบจับต้องได้** — ไม่ใช่ฟังก์ชัน ไม่ใช่เลเยอร์ ไม่ใช่ผลลัพธ์ของอะไรเลย มันคือ **รายการเลข 8 ตัว** • ตอนเทรนจริงมันอยู่บน GPU (`.to(device)`, `on_policy_runner.py:77`) พิมพ์ออกมาดูจะได้แบบนี้

```
>>> self.logstd # actor_critic.py:118
Parameter containing:
tensor([0., 0., 0., 0., 0., 0., 0., 0.], device='cuda:0', requires_grad=True)
```

อะไรทำให้มันเป็น ลูกบิด — ไม่ใช่ `requires_grad=True` ตัวเดียว • tensor ที่มี `requires_grad=True` แต่ไม่ได้ถูกส่งให้ optimizer จะมีแค่ `.grad` ถูกเติมแล้วนอนนิ่ง ไม่ขยับเลย • ที่ทำให้มันขยับคือมันเป็น `nn.Parameter` จึงติดไปกับ `self.actor_critic.parameters()` และไปโผล่ใน Adam ที่ `ppo.py:85`

แถวล่างสุดคือเหตุผลทั้งหมด — **ไม่มีอะไรห้าม** optimizer ลาก σ ที่เก็บตรง ๆ ลงไปถึง 0 หรือติดลบ · ถ้าเกิดขึ้นเมื่อไร $\log \sigma$ ใน `log_prob` และ `entropy` กลายเป็น NaN **เจียบ ๆ** เพราะ repo นี้ปิด validation ไว้ (`Normal.set_default_validate_args = False`, `actor_critic.py:121` พร้อมคอมเมนต์ "*disable args validation for speedup*") · **exp** ปิดประตูนั้นตั้งแต่แรก

ไม่ได้แปลว่าเก็บ σ ตรง ๆ แล้วเทรนไม่ได้ — บั๊กรัดที่ถูกคอมเมนต์ทิ้งไว้ที่ `actor_critic.py:117` คือ `self.std = nn.Parameter(init_noise_std * torch.ones(num_actions))` ซึ่งเป็นวิธีมาตรฐานของ `rs1_rl/legged_gym` ต้นทาง และมันเทรนได้จริง · การเก็บเป็น log แค่ว่า **ตัดความเสี่ยงนั้นทิ้งไปเลย** (เหตุผลเต็ม ๆ อยู่ที่ E4)

Remark. เวลาเจอ $\partial L / \partial \ell$ ให้อ่านว่า "*เขียนยับเท่าไร ถ้าหมุนลูกบิด `logstd` นิดนึง*" · ตัวบนเป็นเขียน ตัวล่างเป็นลูกบิดเสมอ

ประโยคเดียวที่ควรจำเรื่อง ℓ — เลข 8 ตัวที่นอนอยู่ ข้าง ๆ NN ไม่ใช่ข้างใน · หนึ่งตัวต่อหนึ่งข้อต่อ · หน้าทีเดียวคือบอกว่าข้อต่อนั้นจะสั้นแรงแค่ไหน · เก็บในรูป log เพื่อกันไม่ให้ σ ติดลบ

ℓ เป็น **ตัวแปร** ไม่ใช่ **นิพจน์** — เส้นแบ่งทั้งหมดอยู่ตรงนี้

```
mean = self.actor(observations) # :156 นิพจน์ — คำนวณใหม่ทุกครั้งที่เราเรียก แล้วทิ้ง
self.logstd # :118 ตัวแปร — ไม่มีการคำนวณ ค่าค้างอยู่เฉย ๆ
```

(μ คือชื่อทางคณิตศาสตร์ของ `mean` ในโค้ด) · μ ไม่มีค่าของตัวเอง มันเป็น *ผลลัพธ์* · ℓ มีค่าของตัวเอง ไม่ได้ถูกคำนวณมาจากอะไรเลย · **ไม่มีสูตรผลิตมัน และระหว่าง การเทรน ค่ามันเปลี่ยนได้ทางเดียวคือมีคนเขียนกับ** (อีกทางคือ `load_state_dict` ตอน โหลด checkpoint — ดู E4)

และคนที่เขียนกับคือ `optimizer.step()` ซึ่งย่อแล้วคือลูป assignment สรรดา:

```
for p in self.param_groups[0]['params']: # ppo.py:85 มีกลุ่มเดียว · วนจนมาถึง self.logstd
    g = p.grad # ค่าที่ .backward() เพิ่งเติมไว้ให้
    ... # Adam คิด m-hat กับ v-hat จาก g (ดูหัวข้อ Adam ข้างล่าง)
    p -= lr * m_hat / (v_hat**0.5 + eps) # <- บั๊กรัดนี้ · เขียนกับ จบ
```

ของจริงเขียนสั้นกว่านี้ — `torch.optim.Adam` ไม่ได้แตะ `.data` เลยสักครั้ง · บั๊กรัดที่เขียนกับจริงคือ `param.addcdiv_(exp_avg, denom, value=-step_size)` (`torch/optim/adam.py:432` ใน `torch 2.5.1` — เลขบั๊กรัดขึ้นกับเวอร์ชัน) ซึ่งเขียนกับ **in-place** · ส่วน bias correction ถูกพับเข้าไปใน `step_size` กับ `denom` แล้วพีชคณิตออกมาเท่ากับสูตรข้างบนเป๊ะ

แล้ว `g.grad` มาจากไหน ในเมื่อ ℓ ไม่มี input เข้ามาเลย

นี่คือคำถามที่ควรค้าง · weight ในเลเยอร์มีสิทธิ์มีเสียงเพราะมันคูณกับ input ที่ไหลเข้ามา แต่ ℓ ไม่มีใครป้อนอะไรให้เลย

คำตอบคือ ℓ ไม่ต้องมี input · autograd ไม่สนใจว่าตัวไหนเป็นเลเยอร์ มันสนใจข้อเดียว — **ตัวเลขนี้ไหลอยู่ในนิพจน์ที่นำไปสู่ L หรือเปล่า**

ลูกบิด	ไหลในนิพจน์ไหน
weight w ในเลเยอร์	$y = wx + b$

สังเกตสองอย่าง: **entropy** แยกออกจาก **sigma** ไม่ใช่จาก 1 โดยตรง (ในโค้ดคือ `self.distribution.entropy()` ซึ่งอ่าน `scale` — `actor_critic.py:152-153`) และ **มีการยุบมิติสองครั้ง** คือ `.sum(dim=-1)` รวม 8 มิติ แล้ว `.mean()` เฉลี่ยทั้ง minibatch (`ppo.py:260` และ `:273`) กว่าจะเหลือ scalar ตัวเดียวคือ L

α เป็น **ค่าคงที่** ที่หยิบมาจาก buffer — มันเป็นแค่ *ตัวเลขที่ถูกป้อนเข้าสู่สูตร* ไม่ใช่ทางที่ gradient ไหลผ่าน (มันถูก `.detach()` ไปแล้วที่ `ppo.py:139`)

ประโยคที่ควรจำ — ℓ ไม่ต้อง "รู้" อะไรเลย และไม่ต้องมีใครป้อน input ให้ · มันแค่ **บังเอิญอยู่ในสูตร** ที่ใช้คำนวณ L · autograd จึงคิดได้ว่า "ถ้า ℓ มากขึ้น นิดนึ่ง L จะเปลี่ยนเท่าไร" แล้ว `.step()` ก็เขียนตามตามนั้น

ℓ กับ `1.grad` เป็นสองกล่องที่วางอยู่ข้างกัน

นี่คือจุดที่ทำให้ประโยค " ℓ คือตัวเลขที่คำนวณมาจากสายนั้น" ผิด · สิ่งที่สายนั้น คำนวณคือ `1.grad` ไม่ใช่ 1

```
self.logstd      -> [ 0.000000 , ... ]  <- ค่า 8 ตัว · มีตั้งแต่สร้างโมเดล
self.logstd.grad -> None           <- ยังไม่มี · จะกลายเป็น 8 ตัวหลัง backward() ครั้งแรก
```

	1	1.grad
คืออะไร	ค่าตั้ง ของโมเดล	ความชัน ที่เพิ่งคำนวณได้
มาจากไหน	ถูกตั้งตอนสร้างโมเดล แล้วถูกเขียนทับ	ถูกคำนวณใหม่ทุกรอบ
ใครเขียน	<code>optimizer.step()</code> (<code>ppo.py:290</code>)	<code>loss.backward()</code> (<code>:288</code>) แล้ว <code>clip_grad_norm_</code> (<code>:289</code>) ย่อทับอีกที
อยู่นานแค่ไหน	ค้างตลอดการเทรน	ถูกล้างทุกรอบด้วย <code>zero_grad()</code>

`backward()` ไม่เคยแตะ 1 มันเขียนลง `1.grad` อย่างเดียว · `step()` ไม่ได้คำนวณ *gradient* — มันอ่าน `1.grad` แล้วเขียนทับ 1 (ตัว Adam เองยังคิด $\hat{\mu}, \hat{v}$ อยู่ แต่ คิดจาก `1.grad` ที่มีอยู่แล้ว ไม่ได้ไปแตะกราฟ) · ลำดับเต็มคือ `zero_grad` → `backward` → `clip_grad_norm_` → `step` ที่ `ppo.py:287-290`

แล้วค่าแรกของ ℓ โผล่มาจากไหน — มันไม่ได้โผล่ มันถูกตั้ง

```
self.logstd = nn.Parameter(torch.zeros(num_actions)) # actor_critic.py:118
```

ไม่มีการคำนวณ ไม่มีสูตร ไม่มีที่มา — เหมือนค่าเริ่มต้นของตัวแปรในโปรแกรมทั่วไป จากนั้นมันก็ถูกเขียนทับไปเรื่อย ๆ สเติ้ลปะนิต

`backward()` คือ **backpropagation** หรือเปล่า — ใช่ ตรงตัว

มันคือการ **เดิน chain rule ย้อนกลับ** ตามกราฟที่ forward pass บันทึกไว้ · ℓ มี **สองสาย** ไม่ใช่สายเดียว (ตรงกับรูปข้างบน)

สายหลัก — ผ่าน $\log \pi$ มีสองข้อต่อ · ให้ n เป็นเลขตัวอย่างใน minibatch

ข้อต่อ	อนุพันธ์	ค่า (e0 t0, p = 1)
L ไรต่อ $\log \pi_n$ แคไหน	$\frac{\partial L}{\partial \log \pi_n} = -\frac{1}{\text{MB}} \hat{A}_n \rho_n$	-0.7975/MB

ข้อต่อ	อนุพันธ์	ค่า (e0 t0, p = 1)
$\log \pi_n$ ไรต์ ℓ_i แคโคโน	$\frac{\partial \log \pi_n}{\partial \ell_i} = z_{n,i}^2 - 1$	-0.8400
คูณกันแล้วรวมทุกตัวอย่าง	$-\frac{1}{\text{MB}} \sum_n \hat{A}_n \rho_n (z_{n,i}^2 - 1)$	+0.6699

สายที่สอง — แยกออกจาก σ ไปทาง entropy ให้ -0.01 คงที่ · ไม่มี 1/MB เพราะ H ขึ้นกับ ℓ_i อย่างเดียว ทุกตัวอย่างได้ค่าเท่ากัน เจลี่ยแล้วได้ตัวเดิม (เหตุผลเต็มอยู่ที่ E4) · เขียนสองสายเป็นสมการ (ต่อหนึ่งตัวอย่าง ก่อนหาร MB):

$$\boxed{\frac{\partial L_n^{\text{CLIP}}}{\partial \ell_i} = -\hat{A}_n \rho_n (z_{n,i}^2 - 1), \quad \frac{\partial H}{\partial \ell_i} = 1} \quad (35)$$

ความหมาย: ทำดี ($\hat{A} > 0$) ที่อยู่ไกล ($|z| > 1$) → ค่าลบ → เพิ่ม ℓ → กว้างขึ้น · ทำดีที่อยู่ใกล้ → ค่าบวก → แคบลง · ทำแยกลับเครื่องหมาย · ส่วน entropy ดันกว้างด้วยแรง คงที่ 1 (คูณ $-c_H$ ในสมการ (33) ได้ -0.01 ใน .grad) — มาจาก (31) ที่ $\partial H / \partial \ell_i = 1$ เพราะ H มี ℓ_i อยู่ตรง ๆ หนึ่งพจน์

$\frac{1}{\text{MB}}$ ตัวนี้สำคัญ อย่างองข้าม — surrogate_loss จบด้วย .mean() (ppo.py:260) · ตัวอย่าง ข้างล่างสมมติว่าทั้ง minibatch 12,288 ตัวอย่างหน้าตาเหมือนกันหมด ผสมรวมหารด้วย MB จึงได้ +0.6699 พอดี · ถ้าในความจริงมีตัวอย่างแบบนี้อยู่ตัวเดียว ที่เหลือเจลี่ยเป็น ศูนย์ พจน์นี้จะเหลือ 0.6699/12,288 $\approx$ 0.0000545 ซึ่ง แพ้ entropy -0.01 แล้ว σ จะ ทางออก แทนที่จะหุบ · ทิศสุดท้ายตัดสินด้วยค่าเจลี่ยทั้ง minibatch เสมอ ไม่ใช่ด้วยตัวอย่างเดียว

ไล่หนึ่งสเต็ปเต็ม ๆ ด้วยตัวเลขจริง

ก่อนเริ่ม — toy ตรึง $\sigma = 0.5$: $1 = -0.693147 \cdot 1.\text{grad} = \text{None}$ (Tron1 จริ่งเริ่มที่ $1 = 0$, $\sigma = 1.0$ — กลไกเหมือนกันทุกประการ)

ขั้น 1 · forward — $a_i = 0.40$ ไม่ได้สุ่มในรอบนี้ มันถูกสุ่มไว้ตั้งแต่ตอนเก็บ rollout แล้วอ่านกลับมาจาก buffer (ppo.py:212-214) · forward รอบนี้คำนวณใหม่แค่ $\mu_i = 0.20$ กับ σ จาก ℓ เท่านั้น (act() ที่ :205 ก็สุ่มนะ แต่ค่าที่มันคืนถูกโยนทิ้ง) → $z_i = 0.40$, $z_i^2 = 0.16$ · critic บอกว่าทำนี้ดี $\hat{A} = +0.7975$ (อภิธานศัพท์ 8.4) · เป็น gradient step แรกจาก 4 (Tron1: จาก 20) จึงยังไม่ยับ θ ทำให้ $\rho = 1$

ขั้น 2 · backward() — ตรงนี้คือ backprop

ที่มา	ค่าที่ได้
จาก surrogate: $-\hat{A}\rho(z_i^2 - 1) = -0.7975 \times (0.16 - 1)$ — สมการ (35)	+0.669900
จาก entropy: $-\text{entropy_coef} \times 1 = -0.01$ (cfg:97)	-0.010000
รวม เขียนลง 1.grad	+0.659900

ขั้น 3 · step() — ตรงนี้คือการเขียนกับ

$$\ell_{\text{ใหม่}} = \underbrace{-0.693147}_{\ell \text{ เดิม}} - \underbrace{10^{-3}}_{lr} \times \underbrace{0.659900}_{1.\text{grad}} = -0.693807$$

$$\sigma_{\text{ใหม่}} = e^{-0.693807} = 0.499670 \quad (\text{เดิม } 0.500000)$$

ตีความ — ถ้าดี ($\hat{A} > 0$) และอยู่ ใกล้ จุดกลาง ($|z_i| = 0.40 < 1$) → **หุบแคบลง** ตรงกับตารางสีกรีนในหัวข้อ "กลไกที่ σ เปลี่ยนค่า" ที่ E4 · ใช้ lr จริงของ Tron1 (10^{-3}) เพื่อให้เห็นว่าก้าวจริงเล็กแค่ไหน — ส่วนอื่นของ toy ตรง σ ไว้ที่ 0.5 ตลอด

```
บรรทัดสุดท้ายเขียนแบบ SGD (สมการ (12)) เพื่อให้เห็นเลขชัด ๆ · ของจริงเป็น Adam ซึ่งก้าวแรก จะออกมาราว ๆ lr
พอดี ไม่ได้เป็น lr × 0.659900 (ดูหัวข้อ Adam สมการ (36))

บรรทัด step() ข้างบนยังสมมติอีกข้อว่า norm รวมของ gradient ไม่เกิน max_grad_norm = 1.0 (cfg:105) ถ้าเกิน
clip_grad_norm_ จะย่อ l.grad ลงตามสัดส่วนก่อน step() จะได้เห็น

สามบรรทัดที่ควรจำ backward() คำนวณ l.grad — ไม่แตะ 1 step() เขียนกับ 1 — ไม่ได้คำนวณ gradient 1
เองไม่เคยถูกคำนวณจากอะไรเลย มันถูกตั้งค่าครั้งแรก แล้วถูกเขียนทับซ้ำ ๆ
```

Gradient — ความชัน

คำถามที่ต้องตอบก่อนหุมนลูกบิดคือ "หมุนทางไหน" · gradient คือคำตอบ

$$\frac{\partial L}{\partial l_i} = \text{"ถ้าเราเพิ่ม } l_i \text{ ขึ้น นิดเดียว } L \text{ จะเปลี่ยนไปเท่าไร"}$$

อ่านเครื่องหมายอย่างเดียวก็พอ:

ถ้า $\partial L / \partial l_i$	แปลว่า	ดังนั้นควร
เป็น บวก	เพิ่ม l_i แล้ว แ่ลง	ลด l_i
เป็น ลบ	เพิ่ม l_i แล้ว ตีขึ้น	เพิ่ม l_i
เป็น ศูนย์	ลูกบิดนี้ไม่มีผลกับเขมเลย	ไม่ต้องแตะ

เราไม่เคยคำนวณมันด้วยมือ — `loss.backward()` (ppo.py:288) ไล่ chain rule ย้อนกลับให้ทั้ง 469,905 ตัวในคำสั่งเดียว แล้วเอาผลไปแปะไว้ที่ `.grad` ของแต่ละลูกบิด · ที่โน้นนี่เขียนสูตร $\partial L / \partial l$ ออกมาก็เพื่อให้ เห็นว่า มันคิดอะไรอยู่ ไม่ใช่เพราะต้องเขียนโค้ดเอง

SGD — กฎการหุมนที่ง่ายที่สุด

รู้ทิศแล้วก็หุมนสวนทางกับความชัน:

$$l_i \leftarrow l_i - \alpha \frac{\partial L}{\partial l_i}$$

(สมการ (12) ของโน้ต 05 เขียนเฉพาะลูกบิด l_i)

เรียกว่า **gradient descent** — "ไถลงเนิน" · α คือ **learning rate** = ก้าวยาวแค่ไหน

ทำไมเป็นเครื่องหมาย **ลบ**:

ชัน	เหตุผล
สมมติ $\partial L / \partial l_i = +3$	เพิ่ม l_i แล้ว L แ่ลง เราจึงอยาก ลด l_i
$l_i \leftarrow l_i - \alpha(+3)$	ลดจริง ✓
สมมติ $\partial L / \partial l_i = -3$	เพิ่ม l_i แล้ว L ตีขึ้น เราจึงอยาก เพิ่ม l_i

ขั้น	เหตุผล
$l_i \leftarrow l_i - \alpha(-3) = l_i + 3\alpha$	เพิ่มจริง ✓

เครื่องหมายลบตัวเดียวจึงถูกทั้งสองกรณี ไม่ต้องมี if

"S" ใน SGD มาจากไหน — *stochastic* แปลว่า "สุ่ม" · โดยทั่วไปการคิด L ให้ถูกเป๊ะ ต้องใช้ข้อมูล**ทั้งหมด** ซึ่งมักแพงเกินไป จึงสุ่มหยิบมาก้อนเดียวแล้วคิดจากก้อนนั้น ได้ gradient ที่ **ประมาณ** ของจริง

แต่ของ Tron1 ไม่ได้หั่นเพราะแพง — 49,152 ตัวอย่างใส่ GPU ได้สบาย · ที่หั่นเป็น 4 ก้อน (MB = 12,288) แล้ววน 5 epoch ก็เพื่อให้ได้ **20 gradient step ต่อหนึ่งรอบ การเก็บข้อมูล** ต่างหาก (โน้ต 05 §4.7) · การสุ่มยังเป็นการสุ่มจริง — torch.randperm ที่ rollout_storage.py:230 (สับครั้งเดียวต่อ iteration นอก loop epoch :251)

Adam — SGD ที่ปรับความยาวก้าวเอง

ปัญหาของ SGD คือ α ตัวเดียวใช้กับลูกบิตทุกตัว · ลูกบิตที่ความชันใหญ่จะกระโดดแรง เกินจนเลย ส่วนลูกบิตที่ความชันจืดจะแทบไม่ขยับเลย

Adam แก้ด้วยการจำลองอย่างต่อลูกบิตหนึ่งตัว แล้วหารความชันทิ้ง:

$$l_i \leftarrow l_i - \alpha \frac{\hat{m}_i}{\sqrt{\hat{v}_i} + \text{eps}} \quad (36)$$

ไม่ใช่ ϵ นะ — โบนัสของ ϵ ไว้ให้ noise มาตรฐาน และ ϵ ให้ขอบ clip ไปแล้ว (ตาราง "สัญลักษณ์หน้าตาคล้าย" ใน §3) · ตัวกันหารศูนย์ของ Adam จึงเขียนว่า eps ตรง ๆ ไม่ใช่ ϵ กัน

สัญลักษณ์	คือ	ทำหน้าที่
$\hat{m}_i$	ค่าเฉลี่ยเคลื่อนที่ของ ความชัน (แก้ bias แล้ว — นั่นคือความหมายของหมวก)	โมเมนตัม — ความชันที่ชี้ทางเดิมซ้ำ ๆ จะสะสมแรง
$\hat{v}_i$	ค่าเฉลี่ยเคลื่อนที่ของ ความชันยกกำลังสอง (แก้ bias แล้ว)	วัดว่าลูกบิตนี้ปักติช้นแค่ไหน
eps	เลขจืด (10^{-8} — ค่าปริยายของ PyTorch ไม่ใช่ค่าที่ repo นี้ตั้ง)	กันหารศูนย์

สิ่งที่ต้องจำจาก Adam มีข้อเดียว — เปรียบส่วนมีหน่วยเดียวกัน ตัวหารจึง **ลบขนาด ของความชันทิ้ง** · ถ้าความชัน ชี้ทางเดิมสม่ำเสมอ ขนาดก้าวจะอยู่ที่ราว ๆ α ไม่ว่าความชันจะใหญ่หรือเล็ก

แต่ไม่ใช่ α เสมอไป — ตาราง SNR ของ Adam · $\hat{m}/\sqrt{\hat{v}}$ เป็น อัตราส่วนสัญญาณต่อสัญญาณรบกวน (signal-to-noise ratio, SNR) ไม่ใช่ทศล้น ๆ จำลองด้วยค่าปริยาย PyTorch ($\beta_1 = 0.9$, $\beta_2 = 0.999$):

ความชันที่ป้อนเข้า ($g = \mu_g + \text{noise}$)	ก้าวสุทธิเฉลี่ยในทิศของ μ_g
ชี้ทางเดิมสม่ำเสมอ (noise 0)	1.00α
สลับทิศไปมา สัญญาณเท่าสัญญาณรบกวน ($\mu_g = \sigma_g$)	$\approx 0.7 \alpha$
สลับทิศไปมา สัญญาณน้อยกว่ามาก ($\mu_g = 0.03 \sigma_g$)	$\approx 0.03 \alpha$ (แต่ละก้าวยาว $\sim 0.2\alpha$ แต่แหว่งไปมาจนหักล้างกัน)
เป็นศูนย์	0
สม่ำเสมอแต่เล็กมาก (10^{-9})	0.09α — พ้น eps เริ่มมีผล

(จำลองด้วย Adam ของ PyTorch เอง 6,000 ก้าว เฉลี่ยครั้งหลัง — ตัวเลขเปลี่ยนตาม seed เล็กน้อย)

เวลาอ่านสูตร **gradient** ในโน้ตนี้ ให้อ่านเอาทศและอัตราส่วนของแรง อย่าอ่านเอา ขนาดก้าวจริง

Tron1 ใช้ Adam (ppo.py:85) โดย α เริ่มที่ $1e-3$ (cfg:100) แล้วถูก KL ปรับขึ้นลงระหว่างทาง

KL คืออะไร

`kl` ในหัวข้อถัดไปถูกควบคุมด้วยค่านี้ จึงต้องรู้จักมันก่อน

ชื่อเต็มคือ **Kullback–Leibler divergence** — เป็นชื่อคนสองคน (Solomon Kullback กับ Richard Leibler, 1951) ไม่ใช่ตัวย่อของศัพท์เทคนิค

ชนิดก่อนอย่างอื่น

$$D_{KL} : \Delta(\mathcal{A}) \times \Delta(\mathcal{A}) \rightarrow [0, \infty)$$

รับการแจกแจงสองอัน คำนวณตัวเลขที่**ไม่ติดลบ** · ไม่ได้รับตัวเลข — รับ *ระขังทั้งใบ* สองใบ (สัญลักษณ์ $\Delta(\mathcal{A})$ คือเซตของการแจกแจงบน $\mathcal{A}$ — §3.5.0)

เขียนว่า	ชนิด	หมายถึง	ในโค้ด
D_{KL}	ฟังก์ชัน	ตัวเทียบ ยังไม่ได้ป้อนอะไรเข้าไป	—
$D_{KL}(p \parallel q)$	<code>scalar</code> ≥ 0	ความต่างของระขังสองใบ	<code>kl (MB,)</code> · ppo.py:223-231
$\overline{kl}$	<code>scalar</code>	เฉลี่ยทั้ง minibatch	<code>kl_mean</code> · :233

หมายถึงอะไร

$$D_{KL}(p \parallel q) = \mathbb{E}_{a \sim p} [\log p(a) - \log q(a)]$$

อ่านว่า "ถ้าคุณเชื่อว่าโลกเป็น q แต่ความจริงมันเป็น p — คุณจะต้องโศกเศร้าแค่ไหน เฉลี่ยทีหน่วย" · หน่วยคือ **nat** เพราะใช้ลอการิทึมฐาน e · เท่ากับ 0 พอดีเมื่อ p กับ q เหมือนกันทุกจุด · ระบุปิดสำหรับ Gaussian สองใบคือสมการ (30) ของโน้ต 05 §4.6 (ข) ซึ่งตรงกับ ppo.py:223-232 ทุกพจน์ · ความหมายของขีดตั้งสองอัน $\parallel$ และเหตุผลที่สลับที่ไม่ได้ อยู่ที่ §3.5.2

ใน Tron1 มันตอบคำถามเดียว: "ตอนนี้ **policy** ห่างจากตอนเก็บข้อมูลไปแค่ไหนแล้ว"

คำว่า "KL" ในวงการมีสามความหมาย

ใครพูด	หมายถึง	Tron1 ใช้ไหม
ตำราคณิตศาสตร์	ปริมาณ $D_{KL}(p \parallel q)$	✓
คนดู tensorboard	ตัวเลข <code>Policy/mean_kl</code>	✓
คนพูดถึง "KL penalty"	พจน์หนึ่งใน loss	✗ ไม่มี

ตรงข้ามกับ **entropy** — สองตัวนี้มาคู่กันเสมอ แต่คนละเรื่อง

	วัดอะไร	ป้อนเข้าไปที่อื่น	อยู่ใน loss ไหม
entropy H	ความไม่แน่นอนของ หนึ่ง ระขัง	1	✓ อยู่ สับประสิทธิ์ −0.01
KL	ความต่างระหว่าง สอง ระขัง	2	✗ ไม่อยู่เลย

ประโยชน์ที่ควรจำ — KL ไม่ได้อยู่ในสูตร loss สักตัว (ppo.py:274-278 มีแค่สามพจน์ ไม่มี KL) · มันเป็นแค่ **มาตรวัด** ที่อ่านออกมาแล้วเอาไปหมุนวาลว `lr` เท่านั้น · ถ้าลบบล็อก KL ทั้งก้อน gradient จะยังคำนวณได้เหมือนเดิมทุกประการ แค่ `lr` จะไม่ปรับตัวอีก

(ถึง `early_stop` ที่ ppo.py:246-249 ก็ใช้ KL เหมือนกัน แต่ `early_stop` ใช้ค่า default คือปิด — กิ่งนั้นไม่เคยทำงาน)

ตัวอย่างจาก §3.5.2 (ขยับ μ 0.20 $\rightarrow$ 0.25 และ σ 0.5 $\rightarrow$ 0.45): KL รายมิติ **0.018096** คูณ 8 มิติได้ $\approx$ **0.145** เทียบกับ `desired_kl = 0.01` $\rightarrow$ เกิน **0.02** ไปมาก $\rightarrow$ `lr` จะถูกลดด้วย 1.5 · ส่วนใน toy ที่ σ คงที่และ μ ขยับ 0.05: $0.05^2 / (2 \cdot 0.25) = 0.0050$ ต่อมิติ (นับ 07 §5.7)

`lr` มาจากไหน — และทำไมมันไม่คงที่

`lr` ย่อมาจาก **learning rate** — ตัวเดียวกับ α ในหัวข้อ SGD ข้างบน เขียนสองแบบ เพราะได้ใช้ชื่อ `lr` ส่วนตำราใช้ α · หน้าที่มันคือ **บอกว่าการยาวแค่ไหน**

$$\ell \leftarrow \ell - \underbrace{lr}_{\text{การยาวแค่ไหน}} \times \underbrace{\ell.\text{grad}}_{\text{ทิศทางของทิศ}} \quad (\text{รูป SGD})$$

นี่คือรูป SGD — Adam แทน $\ell.\text{grad}$ ด้วย $\hat{m}/(\sqrt{\hat{v}} + \text{eps})$ ซึ่งปรับสเกลรายลูกบิด ทำให้ ทิศ ที่ก้าวจริงไม่เท่ากับทิศของ gradient เป๊ะ ๆ และ `lr` ไม่ได้เป็นตัวกำหนดระยะเพียงตัวเดียว · เขียนรูป SGD ไว้เพราะมันอ่านง่ายกว่าและ **บทบาทของ `lr` เหมือนกันทั้งสองแบบ** คือเป็นตัวคูณความยาวก้าว

ถ้า $lr = 0$ ลูกบิดไม่ขยับเลย ต่อให้ gradient ใหญ่แค่ไหนก็ตาม

ช่วงที่ 1 · ค่าเริ่มต้นมาจากมนุษย์พิมพ์ลงไฟล์

```
learning_rate=1.0e-3,      # limx_rsl_rl_ppo_cfg.py:100
```

ไม่มีใครคำนวณมันมา · มันคือ **hyperparameter** — ตัวเลขที่คนตั้งเอง จูนเอง ลองผิดลองถูกเอา

	ลูกบิดทั้งหมด θ, ϕ, ℓ (parameter)	<code>lr</code> (hyperparameter)
ค่าแรกมาจาก	โค้ด init (สุ่ม / ศูนย์)	มนุษย์พิมพ์ลง config
เปลี่ยนยังโงะหว่างเรน	<code>optimizer.step()</code> จาก gradient	กฎ KL — ไม่มี gradient เกี่ยวข้องเลย
มีกี่ตัว	469,905	1 ต่อ optimizer — การรันนี้มีสองตัว ดูเกร็ดท้ายหัวข้อ

ช่วงที่ 2 · ระหว่างเรน KL ปรับให้เอง เหมือน thermostat

`lr` ของ repo นี้ **ไม่คงที่** เพราะ `schedule = "adaptive"` (cfg:101)

```
if kl_mean > self.desired_kl * 2.0:                # > 0.02
    self.learning_rate = max(1e-5, self.learning_rate / 1.5)
elif kl_mean < self.desired_kl / 2.0 and kl_mean > 0.0:  # < 0.005
    self.learning_rate = min(1e-2, self.learning_rate * 1.5)
for param_group in self.optimizer.param_groups:
    param_group["lr"] = self.learning_rate           # ppo.py:236-244
```

$$lr \leftarrow \begin{cases} \max(10^{-5}, lr/1.5) & \bar{kl} > 2 d_{KL} = 0.02 \\ \min(10^{-2}, lr \times 1.5) & 0 < \bar{kl} < d_{KL}/2 = 0.005 \\ lr & \text{อื่น ๆ} \end{cases} \quad (37)$$

อ่าน k1_mean ได้	แปลว่า	ทำอะไร
> 0.02	policy ขยับ เยอะเกิน	หรือ $lr \div 1.5$ (ไม่ต่ำกว่า $1e-5$)
< 0.005	ขยับ น้อยเกิน เสียเวลา	เร่ง $lr \times 1.5$ (ไม่เกิน $1e-2$)
อยู่ระหว่างนั้น	กำลังดี	ไม่แตะ

ใน toy: ขยับ μ 0.05 ทุกช่อง $\rightarrow \bar{kl} = 0.0050 \rightarrow$ คงเดิม $\cdot$ ขยับ ± 0.15 ตามกฎคู่แฝดของ โน้ต 07 §5.3 $\rightarrow 0.045 \rightarrow \div 1.5$

desired_kl = 0.01 (cfg:104) คือ **อุณหภูมิเป้าหมาย** $\cdot$ k1_mean คือ **อุณหภูมิที่วัดได้** $\cdot$ lr คือ **วาล์ว**

สองข้อที่น่าแปลกใจ — บล็อกนี้อยู่ **ในรูป minibatch** (ดูเริ่มที่ ppo.py:190 บล็อก KL อยู่ที่ :236 ย่อหน้าลึกกว่าหนึ่งชั้น) แปลว่า lr ปรับได้ถึง **20 ครั้งต่อหนึ่ง iteration** ไม่ใช่ครั้งเดียว $\cdot$ และจาก $1e-3$ ไปชนเพดาน $1e-2$ ใช้แค่ **6 ครั้ง** ส่วนไปชนพื้น $1e-5$ ใช้ **12 ครั้ง** — ขยับเร็วกว่าที่คิดมาก

ทำไมไม่ตั้งค่าคงที่ไปเลย

lr	เกิดอะไรขึ้น
ใหญ่เกิน	ก้าวเลยจุดดี $\cdot$ policy เปลี่ยนฮวบ $\cdot$ ρ หลุดกรอบ clip เป็นวงกว้าง $\cdot$ ตัวอย่างที่ขยับถูกทางไปไกลเกินจะให้ gradient เป็น 0 (ตัวที่ขยับผิดทางยังแก้ได้ — โน้ต 07 §5.3) $\rightarrow$ สัญญาณเรียนรู้เหลือน้อยลงมาก และ surrogate เล็กประมาณเป้าหมายจริง $\rightarrow$ เทรนพัง
เล็กเกิน	ไม่พัง แต่ช้า เพา GPU ฟรี

จุดที่พอดี **เปลี่ยนไปตลอดการเรน** (ตอนต้นภูมิประเทศชันมาก ตอนปลายชันน้อย) — ค่าคงที่ตัวเดียวจึงผิดเสมอ อย่างน้อยก็ช่วงใดช่วงหนึ่ง $\cdot$ KL เหมาะเป็นตัววัดเพราะมันบอก ว่า **policy ตอนนีห่างจาก policy ที่ใช้เก็บข้อมูลรอบนี้ไปแค่ไหน** (เฉลี่ยบน state ที่เจอ ใน minibatch)

ระวัง: ไม่ใช่ "ก้าวที่แล้วขยับไปเท่าไร" — old_mu_batch/old_sigma_batch มาจาก buffer (rollout_storage.py:145-146) ซึ่งบันทึกไว้ตอนเก็บ rollout $\cdot$ k1_mean จึงเป็น **ระยะทางสะสม**ตั้งแต่ต้น update() ไม่ใช่ระยะของ gradient step ล่าสุด $\cdot$ ผลที่ตามมา ที่สังเกตได้: ที่ minibatch แรกของทุก iteration ยังไม่มี .step() เกิดขึ้นเลย μ, σ จึงเท่าของเก่าเป๊ะ และ k1 เหลือแค่ $8 \log(1 + 10^{-5}) \approx 8 \times 10^{-5}$ จาก eps 10^{-5} กันหาร ศูนย์ที่ ppo.py:224 — ซึ่ง **น้อยกว่า 0.005** ทำให้ถึง "เร่ง" ทำงาน **ทุก iteration** ที่ minibatch แรก

เกร็ด — anneal_lr = False (ppo.py:64) จึงมีแต่กฎ KL อย่างเดียว $\cdot$ ถ้าเปิด anneal_lr มันจะ**ไม่ใช้**การลด lr ตามเวลาการเรน แต่เป็นฟันเลื่อยภายในหนึ่ง iteration (1.0 ลงไป 1/20 ข้าม 20 minibatch แล้วรีเซ็ต เพราะ num_updates ถูกล้างทุกครั้งที่เข้า update() — ppo.py:280-284)

encoder มี lr ของตัวเองแยก — est_learning_rate = $1.0e-3$ ซึ่งเป็นค่า default ของ PPO.__init__ (ppo.py:61) ไม่ได้ตั้งใน cfg $\cdot$ ใช้สร้าง extra_optimizer ที่ ppo.py:88-89 $\cdot$ KL ไม่แตะมันเลย — ยืนยันได้ว่า extra_optimizer.param_groups ไม่เคยถูกเขียนที่ไหน

Optimizer และ .step()

optimizer คือวัตถุที่ถือกฎการหมุน (SGD หรือ Adam) และถือรายชื่อลูกบิตที่มันดูแล · หนึ่งรอบการเรียนรู้มีคำสั่งเรียงกันเสมอ:

คำสั่ง	ทำอะไร	ที่ไหน
<code>optimizer.zero_grad()</code>	ล้าง <code>.grad</code> เอาทิ้ง	ppo.py:287
<code>loss.backward()</code>	คิด $\partial L / \partial (\text{ลู่ กบิ ฤ กติ ๖})$ ไปใส่ <code>.grad</code>	:288
<code>clip_grad_norm(...)</code>	ถ้า norm ของ gradient ทั้งก้อนเกิน <code>max_grad_norm</code> ให้ย่อลงตามสัดส่วน — เขียนกับ <code>.grad</code>	:289
<code>optimizer.step()</code>	เขียนกับค่าลูกบิต ตามกฎข้างบน	:290

บรรทัดที่สี่ (`optimizer.step()`, [ppo.py:290](#)) คือที่เดียวที่ตัวเลขของ actor, critic และ `logstd` เปลี่ยนค่า (encoder มี `.step()` ของตัวเองที่ [:321](#))

L ของ PPO ประกอบด้วยสามพจน์ — ทวนในมุมมองของโค้ด

โน้ต 05 §4.6 derive ทั้งสามพจน์แล้ว ((29) surrogate, (32) value loss, (31) entropy → (33) รวม) ตรงนี้ทวนเฉพาะสิ่งที่เห็นจากโค้ด:

$$L = \underbrace{L^{CLIP}}_{\text{actor เร็ย ๓}} + \underbrace{1.0 \cdot L_V}_{\text{critic เร็ย ๓}} - \underbrace{0.01 \cdot H}_{\text{โน้ต ๓ ความน่า'ว}} \quad (\text{สมการ (33)})$$

([ppo.py:274-278](#) · สัมประสิทธิ์ 1.0 และ 0.01 มาจาก [cfg:94](#) และ [cfg:97](#) ไม่ใช่ค่าปริยายของ `ppo.py`)

อ่านเป็นภาษาคน — "ทำให้ actor เลือกค่าที่ advantage สูงขึ้น และ ทำให้ critic ทำนายแม่นยำขึ้น และ อย่าเพิ่งรีบมันใจจนเล็กลงของใหม่"

ระวังเครื่องหมายของ L^{CLIP} — บรรทัดข้างบนใช้ L^{CLIP} แบบที่ **โค้ดเติมลบ ไว้แล้ว** (`surrogate = -advantages * ratio`, [ppo.py:256](#)) จึงบวกเข้ามาตรง ๆ ได้ · ส่วนสูตร $L^{CLIP} = \mathbb{E}[\min(\rho A, \dots)]$ ที่ §3 เขียนไว้ เป็นแบบของเปเปอร์ ซึ่ง **ไม่มีลบ** และเป็นของที่ต้องทำให้ **ใหญ่** · สองอันนี้คือตัวเดียวกัน ต่างกันแค่เครื่องหมาย

พจน์ที่ 1 · L^{CLIP} — surrogate loss · *surrogate* แปลว่า "ตัวแทน" และมันเป็น ตัวแทนของอะไร คือเรื่องที่ควรเข้าใจ

สิ่งที่เราอยากได้จริง ๆ คือ "ถ้าใช้ policy ใหม่ จะเก็บแต้มได้เท่าไร" — แต่คำนวณไม่ได้ เพราะต้องเอา policy ใหม่ไปวิ่งจริงถึงจะรู้ และถ้าต้องวิ่งใหม่ทุกครั้งที่ยับ weight นิดเดียว ข้อมูลที่เก็บมาทั้ง 49,152 ตัวอย่างก็ใช้ได้ครั้งเดียวแล้วทิ้ง

L^{CLIP} จึงเป็น **ฟังก์ชันตัวแทน**: คำนวณได้จากข้อมูลเก่า แต่มีความชัน ณ จุดปัจจุบัน ตรงกับของจริง · ตรงจุดที่ยังไม่ยับ ($\theta = \theta_{old}$) จะได้ $\rho = 1$ พอดี และ ความชันของ $\rho \hat{A}$ กลายเป็น $\hat{A} \nabla \log \pi$ ซึ่งคือ **policy gradient** (สมการ (23) ของโน้ต 05 §4.1) — ต่างกันแค่เครื่องหมายลบที่โค้ดเติมไว้ ตามกรอบข้างบน

แต่ตัวแทนจะเหมือนของจริงเฉพาะแถว ๆ จุดเดิมเท่านั้น ยับไกลเมื่อไรมันเริ่มโกหก — และนี่คือเหตุผลหลักที่ต้องมี `clip` · ระวังคำ: `clip` ไม่ได้ *บังคับ* ไม่ให้เดินไกล มันแค่ **ตัดแรงจูงใจ** — พอ ρ หลุดกรอบในทศที่ดีขึ้นแล้ว พจน์นั้นกลายเป็นค่าคงที่ gradient เป็นศูนย์ ตัวอย่างนั้นก็เล็กลงแรงดัน · ρ ทะลุ 1.2 ได้จริงและเกิดขึ้นจริง

พจน์ที่ 2 · L_V — value loss · critic ทำนาย $V(s)$ แล้วเทียบกับเป้า R_t ด้วยกำลังสองของส่วนต่าง · **แต่มีสองจุดที่ไม่ใช่ supervised ssmta**

1. repo นี้เปิด `use_clipped_value_loss = True` (cfg:95) สิ่งที่เราจริงคือ **max** ของ MSE สองแบบ (ppo.py:263-269) — MSE เหล่า ๆ อยู่ใน `else` ที่ไม่ถูกใช้
2. R_t ไม่ใช่ "ผลจริง" — rollout_storage.py:201 สร้างมันจาก `advantage + self.values[step]` โดย `advantage` มาจากสูตร GAE ซึ่งมีค่าทำนายของ critic เองปนอยู่ · เป้าที่ critic ไล่ตามจึงมีตัวมันเองอยู่ข้างใน

พจน์ที่ 3 · H — entropy · entropy วัด "ความไม่แน่นอน" ของการแจกแจงหนึ่งอัน

ระบัพ	entropy	แปลว่า
กว้าง (σ ใหญ่)	สูง	สุ่มออกมาแล้วเดาไม่ถูกว่าจะได้อะไร
แคบ (σ เล็ก)	ต่ำ	สุ่มก็ครั้งก็ได้ค่าใกล้เคียง ๆ เดิม

สำหรับเกาส์เซียน **หนึ่งมิติ** คือ $H_i = \frac{1}{2} \log(2\pi e) + \ell_i$ — ขึ้นกับ σ อย่างเดียว ไม่เกี่ยวกับ μ เลย (ระบัพใบเดิมเลื่อนไปไหนก็ไม่น่าสนใจเท่าเดิม)

ส่วน H ที่เข้าไปอยู่ใน L จริง ๆ รวม 8 มิติก่อน (`.sum(dim=-1)` , actor_critic.py:153) แล้วค่อยเฉลี่ยทั้ง minibatch (ppo.py:273):

$$H = \sum_{i=1}^8 \left[\frac{1}{2} \log(2\pi e) + \ell_i \right] = 8(1.4189) + \sum_i \ell_i$$

ตรวจกับ toy: $\sigma = 0.5$ ทั้ง 8 มิติ $\rightarrow 8 \times 0.7258 = 5.8063$ ✓ (สมการ (31) ต่อมิติ ให้ **0.7258** — ตัวที่เข้า loss คือผลรวม 8 มิติ)

เครื่องหมาย **ab** หน้า **0.01H** คือลูกเล่น "อยากได้มากก็ใส่ลบ" จากต้นหัวข้อ · ทำให้ optimizer ซึ่งพยายามลด L กลายเป็นพยายาม **เพิ่ม** $H \rightarrow$ **เพิ่ม** $\sigma \rightarrow$ บังคับให้ รุ่นยังลองของใหม่ ไม่รีบหุบเข้าหากำแพงตั้งแต่ยังไม่เก่ง

สรุปหัวข้อนี้

คำ	คืออะไร	ประโยคเดียว
L	scalar	เข้มงวดความแท้ · ยิ่งน้อยยิ่งดี
ℓ	เวกเตอร์ (8,)	ลูกบิตที่คุม σ ผ่าน $\sigma = e^\ell$ · ไม่ใช่ L
gradient	ตัวเลขต่อลูกบิต	"หมุนขึ้นแล้วเข้มไปทางไหน" · ได้จาก <code>.backward()</code> · ของ ℓ คือสมการ (35)
learning rate α	scalar	ก้าวยาวแค่ไหน · ปรับด้วยกฎ (37)
SGD	กฎ (12)	ไล่สวนความชัน — ก้าวแปรตามขนาดความชัน
Adam	กฎ (36)	ไล่สวนความชัน แต่ หารขนาดทั้ง — ถ้าความชันชี้ทางเดิมสม่ำเสมอ ก้าวจะราว ๆ α · ถ้ามันจะน้อยกว่านั้นมาก
optimizer	วัตถุ	ผู้ถือกฎและรายชื่อลูกบิต · <code>.step()</code> คือที่เดียวที่ค่าเปลี่ยน
surrogate	ฟังก์ชัน	ตัวแทนของเป้าหมายจริงที่คำนวณไม่ได้ · เชื้อได้เฉพาะใกล้จุดเดิม $\rightarrow$ จึงต้อง clip
entropy H	scalar	ความไม่แน่นอนของระบัพ · ขึ้นกับ σ อย่างเดียว · ในโค้ดรวมครบ 8 มิติแล้ว

ถึงตรงนี้คุณมีศัพท์ครบพอจะอ่าน §3.5.4 ได้ทุกบรรทัด รวมถึงบล็อก "*กลไกที่ σ เปลี่ยนค่า*" ใน E4 ซึ่งเราทุกคำในตารางนี้มาใช้พร้อมกัน

3.5 สายส่งค่าทั้งสาย — E1 ถึง E15 พร้อมโค้ด 1:1

§3 ข้างบนสอนอ่านสัญลักษณ์ทีละตัว หัวข้อนี้ต่อให้เห็น **การส่งค่าต่อกัน**: ค่าตัวไหน ออกจากบรรทัดไหน ไปเข้าเป็น input ของ บรรทัดไหน shape เปลี่ยนตรงจุดใด ตั้งแต่ observation ถึงจนถึง gradient ที่ขยับ θ

เริ่มที่ §3.5.0 ถ้ายังไม่แน่ใจว่า "policy" คืออะไรกันแน่ — หัวข้อย่อยที่เหลือทั้งหมด แยกออกมาจากนิยามบรรทัดเดียวใน นั้น

หมายเหตุการนับเลข — ใช้ป้าย **E1-E15** (E = สายส่งค่า) เพื่อไม่ให้ชนกับสมการ **(1)-(12)** ของโน้ต 05 ซึ่งเป็นคนละชุด · ตัวเลขทุกตัวใน §3.5.5 คำนวณด้วย Python แล้ว สคริปต์อยู่ที่ §3.5.7

3.5.0 policy คืออะไรกันแน่ — นิยามหนึ่งบรรทัด

$$\pi_\theta : \mathcal{S} \longrightarrow \Delta(\mathcal{A})$$

อ่านว่า "policy คือฟังก์ชันที่รับ state แล้วคืน การแจกแจง บน action"

สัญลักษณ์	อ่านว่า
$\mathcal{S}$	เซตของ state ทั้งหมดที่เป็นไปได้
$\mathcal{A}$	เซตของ action ทั้งหมดที่เป็นไปได้ (ของ Tron1 คือ $\mathbb{R}^8$)
$\mathbb{R}^8$	เซตของเวกเตอร์ 8 ช่องทั้งหมด — ไม่ใช่เวกเตอร์ตัวเดียว
$\Delta(\mathcal{A})$	เซตของการแจกแจงบน $\mathcal{A}$ — สมาชิกแต่ละตัวคือระฆังหนึ่งใบ

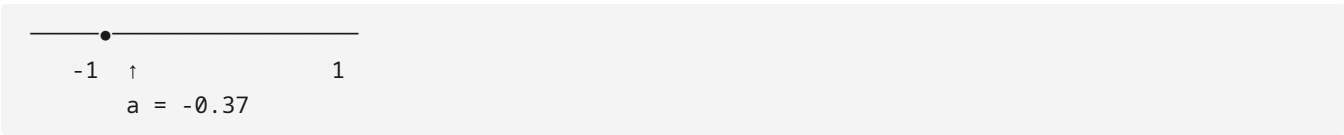
$\mathbb{R}^8$ มีสมาชิกกี่ตัว — อนันต์ ไม่ใช่ 8 · เลข 8 บอกจำนวนช่องของสมาชิกแต่ละตัว ไม่ได้บอกจำนวนสมาชิก · เวกเตอร์หนึ่งตัวเขียนว่า $\mathbf{a} \in \mathbb{R}^8$ ($\in$ = "เป็นสมาชิกของ") — เรื่องเดียวกับ $\mathcal{A}$ กับ $\mathbf{a}$ ด้านล่าง

แถวหรือหลัก — ในคณิตศาสตร์ถือเป็นหลักตั้ง 8×1 เพราะเขียน $\mu = Ws$ โดย W อยู่ซ้าย ($8 \times 42 \cdot 42 \times 1 = 8 \times 1$) แต่ใน PyTorch tensor 1 มิติ (8,) ไม่มีแนวคิดแถว/หลัก และพอ batch เข้ามา ตัวอย่างหนึ่งตัวกลายเป็นหนึ่งแถว (B, 8) nn.Linear จึงคำนวณ $\mathbf{y} = \mathbf{x}W^T + \mathbf{b}$ ไม่ใช่ $Ws + \mathbf{b}$ — ตัวเลขเท่ากันเป๊ะ แค่ว่า กลับด้าน (weight ยังเก็บทรง (out, in) = (8, 42) เหมือนสูตร) · เวลาไล่ shape ในโน้ตนี้ **มิติแรกคือจำนวนตัวอย่างเสมอ มิติหลังคือจำนวนช่อง** ไม่ต้องแปลงเป็นแถว/หลักในหัว

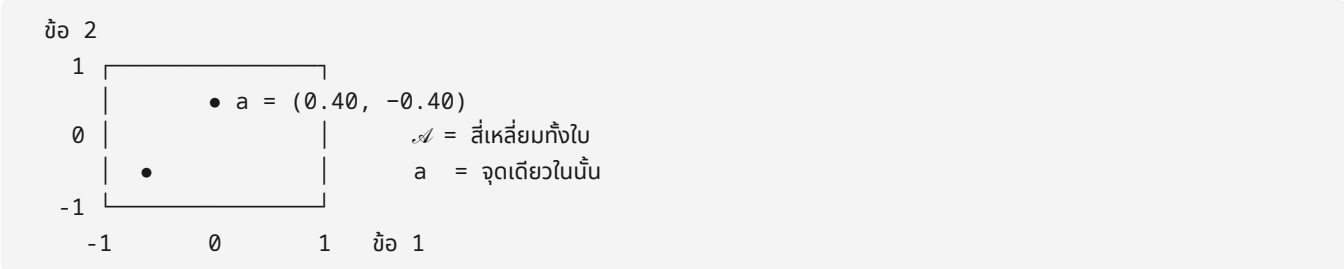
$\mathcal{A}$ หน้าตาเป็นยังไง — ไล่จากง่ายไปยาก

ถ้าหุ่นมีแค่ 3 ปุ่ม — $\mathcal{A} = \{ \text{เลิ นหนั ำ, ถอยหลั ง, หยุ ด} \}$ เขียนออกมา ได้ครบ มี 3 สมาชิก · action หนึ่งอันคือหยิบมาหนึ่งตัว เช่น $\mathbf{a} = \text{เลิ นหนั ำ}$

ถ้าหุ่นมีข้อต่อเดียว หุ่นได้ต่อเนื่อง — $\mathcal{A} = [-1, 1]$ · คราวนี้เขียนออกมาให้ครบ ไม่ได้ เพราะระหว่าง -1 กับ 1 มีจำนวนจริงอนันต์ตัว (0.5, 0.51, 0.5137, ...) แต่ยังไม่ภาพได้ — มันคือเส้นตรงหนึ่งเส้น และ $\mathbf{a}$ คือจุดหนึ่งจุดบนเส้นนั้น



ถ้ามีสองข้อต่อ — $\mathcal{A} = [-1, 1] \times [-1, 1]$ กลายเป็นสี่เหลี่ยมจัตุรัส และ ทุกจุดในสี่เหลี่ยมคือ action หนึ่งอัน



ภาพนี้คือกระดานปาลูกดอกอันเดียวกับตัวอย่างเรื่อง "บน/ของ" ข้างล่าง — กระดานทั้งใบ คือ $\mathcal{A}$ · รูที่ลูกดอกเกาะคือ a

Tron1 มี 8 ข้อ — $\mathcal{A} = \mathbb{R}^8$ วาดไม่ได้แล้ว แต่แนวคิดเหมือนเดิมเป๊ะ

จำนวนข้อ	$\mathcal{A}$ หน้าตาเป็น
1	เส้นตรง
2	สี่เหลี่ยม
3	ก้อนลูกบาศก์
8 (Tron1)	กล่อง 8 มิติ — วาดไม่ได้ แต่คิดแบบเดียวกัน

สมาชิกจริง ๆ ของ $\mathcal{A}$ ใน Tron1 หน้าตาแบบนี้

สมาชิกหนึ่งตัว (= action หนึ่งอัน)	แปลว่าหุ่นทำอะไร
[0, 0, 0, 0, 0, 0, 0, 0]	ยืนตามท่าเริ่มต้น ไม่ขยับข้อไหนเลย
[0.40, -0.40, 0.40, 0.40, -0.40, 1.20, -0.90, 0.40]	ท่าที่สุ่มได้ตอน step นี้ (ตัวเลขจาก §3.5.5)
[-2.0, -2.0, -2.0, -2.0, -2.0, -2.0, -2.0, -2.0]	ย่อตัวสุดแรง น่าจะล้ม แต่ยังเป็นสมาชิกที่ถูกต้อง
... อีกอนันต์ท่า	

$\mathcal{A}$ คือทุกระบรทัดในตารางนี้รวมกัน (ซึ่งเขียนไม่มีวันจบ)

จุดที่มักเข้าใจผิด — $\mathcal{A}$ ไม่ได้แปลว่า "มี 8 actions" · ที่ถูกคือ action หนึ่งอันมี 8 ตัวเลข และ action มีให้เลือกอนันต์อัน · เลข 8 คือจำนวนข้อต่อ ไม่ใช่จำนวนทางเลือก

และ $\mathbb{R}^8$ ไม่มีขอบทางคณิตศาสตร์ — แม้แต่ [1000, 0, ...] ก็เป็นสมาชิก · พอส่งไปจริงมันถูกคูณ 0.25 แล้วเข้า PD ซึ่งมี effort_limit 80 N·m คอยตัด ขอบเขตอยู่ที่ฟิสิกส์ ไม่ได้อยู่ที่นิยามของ $\mathcal{A}$

อ่านบรรทัดประกาศชนิดนี้ยังไ

$\underbrace{\pi_\theta}_{\text{ชื่อ "มี ชนิด เป็น" }} : \underbrace{\mathcal{S}}_{\text{รับ "ฟังก์ชันไปยัง" }} \rightarrow \underbrace{\Delta(\mathcal{A})}_{\text{คือ " "}}$

บรรทัดนี้ไม่ได้คำนวณอะไรเลย — มันแค่ประกาศว่าของชิ้นนี้เป็นอะไร เหมือน type annotation ในโปรแกรม

เครื่องหมาย $\rightarrow$ แปลว่า "เป็นฟังก์ชันจาก...ไปยัง..." ซึ่งเป็นการใช้งานคนละแบบ กับลูกศรอื่นในโน้ตนี้ · ตัวชี้ขาดคือ มี

เครื่องหมาย : นำหน้าหรือเปล่า — ถ้ามี แปลว่า กำลังประกาศชนิด ลูกศรตัวนั้นคือลูกศรฟังก์ชัน (เทียบลูกศรทุกแบบได้ที่ E6 §3.5.4)

เครื่องหมาย Δ ไม่ใช่ผลต่าง และไม่ใช่เซตว่าง · $\Delta(\mathcal{A})$ อ่านว่า "เซตของการ แจกแจงความน่าจะเป็นทั้งหมดที่เป็นไปได้บน $\mathcal{A}$ " — เป็นเซตจริง แต่สมาชิกของมัน ไม่ใช่ตัวเลข สมาชิกแต่ละตัวคือระฆังหนึ่งใบ

ทำไมใช้รูปสามเหลี่ยม — เพราะวาดออกมาแล้วเป็นสามเหลี่ยมจริง ๆ · ลูกเต๋า 3 หน้า การแจกแจงหนึ่งอันคือ (p_1, p_2, p_3) ที่ $p_i \geq 0$ และ $p_1 + p_2 + p_3 = 1$ · วาดจุดพวกนี้ ในปริภูมิ 3 มิติจะได้สามเหลี่ยมที่มีมุมสามมุมคือ $(1, 0, 0)$, $(0, 1, 0)$, $(0, 0, 1)$

$\mathcal{A}$ มีกี่ทางเลือก	$\Delta(\mathcal{A})$ หน้าตาเป็น
2	เส้นตรง
3	สามเหลี่ยม ← ที่มาของสัญลักษณ์
4	ทรงสี่หน้า
ต่อเนื่อง (Tron1)	วาดไม่ได้ มีติดันต์ — แต่ความหมายเดิม

ศัพท์เทคนิคของรูปนี้คือ **simplex** · Δ จึงเป็นสัญลักษณ์มาตรฐานของ "เซตของการแจกแจง"

ระวังสับสน — Δ ตัวใหญ่ = เซตของการแจกแจง · δ_t ตัวเล็ก = TD error ในสมการ GAE · ตัวอักษรกรีกตัวเดียวกัน คนละขนาด คนละความหมายสิ้นเชิง

เอาสัญลักษณ์นี้ไปอ่านของที่รู้จักอยู่แล้ว

ของ	ประกาศชนิด	อ่านว่า
ยกกำลังสอง	$f : \mathbb{R} \rightarrow \mathbb{R}$	รับจำนวนจริง คืนจำนวนจริง
Actor NN	$\mu_\theta : \mathbb{R}^{42} \rightarrow \mathbb{R}^8$	รับเวกเตอร์ 42 คืนเวกเตอร์ 8
Critic	$V_\phi : \mathbb{R}^{230} \rightarrow \mathbb{R}$	รับเวกเตอร์ 230 คืนเลขตัวเดียว
Encoder	$f_\psi : \mathbb{R}^{360} \rightarrow \mathbb{R}^3$	รับ 360 คืน 3
Policy	$\pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$	รับ state คืน ะยังหนึ่งใบ

สามแถวกลางคืนตัวเลข · แถวล่างคืนของที่ไม่ใช่ตัวเลข — นั่นคือทั้งหมดที่ Δ ทำหน้าที่บอก

Δ หายไปตัวเดียว ความหมายเปลี่ยนทั้งบรรทัด

$$\mu_\theta : \mathcal{S} \rightarrow \mathcal{A} \quad \text{เที ยบกับ} \quad \pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$$

ไม่มี $\Delta \rightarrow$ คืน action ตรง ๆ = deterministic policy = ตอน deploy · มี $\Delta \rightarrow$ คืน การแจกแจง ต้องสุ่มอีกชั้น = stochastic policy = ตอนเรनु

เกร็ด — $\Delta(\mathcal{A})$ บรรจุการแจกแจง**ทุกแบบ**ที่คิดออก (สองยอด เบ้ซ้าย ประหลาด ยิ่งไงก็ได้) · การเลือกใช้ Gaussian คือการบอกว่า "จะหยิบจากมุมเล็ก ๆ มุมเดียวของเซตนี้" — ตรงกับที่ §3.5.2 เรียกว่า interface กับ implementation พอดี

"การแจกแจง **บน** action" กับ "การแจกแจง **ของ** action" ต่างกัน

ต้นตอของความสับสนคือ คำว่า "action" ในภาษาไทยกำกวม — มันเป็นได้ทั้งสองอย่าง:

"action" หมายถึง	สัญลักษณ์	คืออะไร
ท่าที่เป็นไปได้ทั้งหมด	$\mathcal{A}$	เซต — รายการทางเลือก นอนนิ่งอยู่เฉย ๆ
ท่าที่สุ่มได้จริงในสแต็ปนี้	a	ตัวแปรสุ่ม — เปลี่ยนทุกครั้งที่ทอย

พอพูดว่า "action" เลย ๆ จึงเลือกคำเชื่อมไม่ถูก · พอเปลี่ยนเป็นสัญลักษณ์ ความกำกวม หายทันที

ตัวอย่างที่ 1 — ปาลูกดอก · กระดาน = ทุกตำแหน่งที่ลูกดอกไปลงได้ ($\mathcal{A}$) · ลูกดอกที่ปาไปเมื่อไหร่ = ลงตรงไหนก็ไม่รู้ ปาใหม่กี่คนละที่ (a)

ประโยค	ถูกไหม	เพราะ
"ความน่าจะเป็นกระจายอยู่ บนกระดาน"	✓	กระดานเป็นพื้นที่ ให้โปรยได้
"การแจกแจง ของลูกดอกที่ปา"	✓	ลูกดอกลูกนั้นเป็นของที่สุ่ม
"การแจกแจง ของกระดาน"	✗	กระดานไม่ได้สุ่มอะไร มันเขวอนอยู่เฉย ๆ
"การแจกแจง บนลูกดอก"	✗	ลูกดอกลูกเดียว ไม่ใช่พื้นที่ให้กระจาย

ตัวอย่างที่ 2 — ลูกเต๋า · $\mathcal{A} = \{1, \dots, 6\}$ (เซต) · $a = 4$ ผลการทอยครั้งนี้

ประโยค	ถูกไหม
"การแจกแจงบน $\{1, \dots, 6\}$ "	✓
"การแจกแจงของผลการทอย"	✓
"การแจกแจงของ $\{1, \dots, 6\}$ "	✗ เซตไม่ทอยตัวเอง
"การแจกแจงบนผลการทอย"	✗ ผลการทอยเป็นเลขตัวเดียว

ตัวอย่างที่ 3 — $Tron1 \cdot \mathcal{A} = \mathbb{R}^8$ ท่าที่เป็นไปได้ทั้งหมด · $a = [0.40, -0.40, 0.40, \dots]$ ท่าที่สุ่มได้จริงในสแต็ปนี้

ประโยค	ถูกไหม
" $\Delta(\mathcal{A})$ = เซตของการแจกแจงบน $\mathbb{R}^8$ "	✓
" $\pi_\theta(\cdot s)$ คือการแจกแจงของ a "	✓
"การแจกแจงของ $\mathbb{R}^8$ "	✗

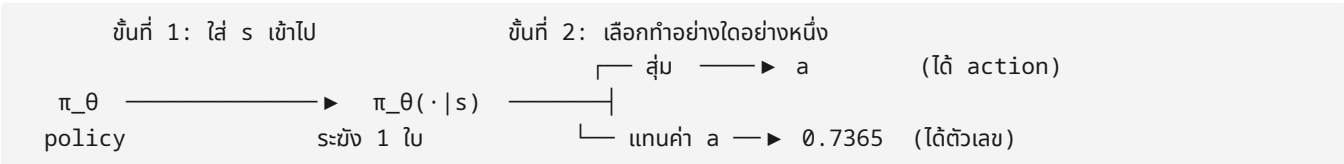
การทดสอบข้อเดียวที่ใช้ได้ทุกครั้ง — ถามว่า "สิ่งนี้สุ่มได้ไหม" · สุ่มไม่ได้ (เป็นพื้นที่ / รายการทางเลือก) → ใช้ "บน" · สุ่มได้ (ทำใหม่ได้ค่าใหม่) → ใช้ "ของ" · จำสั้นกว่านั้น: "บน" ตอบว่า *ที่ไหน* · "ของ" ตอบว่า *ใคร*

ดังนั้นสองประโยคนี้นี้ไม่ได้ขัดกัน แค่มองคนละมุมของเรื่องเดียวกัน — และในบรรทัด $\pi_\theta : \mathcal{S} \rightarrow \Delta(\mathcal{A})$ ตัวอักษรเขียนไว้ชัดแล้วว่า เป็น $\mathcal{A}$ ตัวใหญ่ จึงอ่านว่า "บน" อย่างเดียว ไม่มีทางเลือกอื่น



ภาษาอังกฤษแยกด้วยคำบุพบทเหมือนกัน — ตำรา RL เขียน "*a distribution over actions*" ในนิยาม policy เสมอ ส่วน "*the distribution of X* " ใช้เมื่อ X เป็นตัวแปรสุ่ม

จุดสำคัญที่สุด: มันคือ *การแจกแจง ไม่ได้คือ action* — ความสับสนแทบทั้งหมดเรื่อง สัญลักษณ์ของ policy เกิดจากตรงนี้จุดเดียว เพราะมันแปลว่ากว่าจะได้ action ต้องผ่าน *สองขั้น* ไม่ใช่ขั้นเดียว:



ขั้นที่ 1 คือการเรียกฟังก์ชัน · ขั้นที่ 2 มีสองอย่างให้ทำกับระดัังใบนั้ **ซึ่งเป็นคนละ ปฏิบัติการกัน** (รายละเอียดว่าสองอันนี้เกี่ยวกันตรงไหน อยู่ท้ายหัวข้อนี) — และนี่คือ เหตุผลที่สัญลักษณ์ต้องมีหลายแบบ

เขียนว่า	ชนิด	หมายถึง	ในโค้ด
π	ฟังก์ชัน	policy ทั่วไป ไม่ระบุว่าของใคร	—
π_θ	ฟังก์ชัน	policy ตัวนี้ ที่มีพารามิเตอร์ θ	①+②+③ ที่อยู่ใน <code>self.actor_critic</code> — ไม่ใช่ <code>self.actor_critic</code> เอง เพราะตัวนี้มี critic ปนอยู่ (:111)
$\pi_\theta(\cdot \mid s)$	การแจกแจง	ระดััง 1 ใ้ ได้จากการใส่ s	<code>self.distribution</code> (อ็อบเจกต์) — สร้างโดย <code>Normal(...)</code>
$a \sim \pi_\theta(\cdot \mid s)$	การกระทำ	สุ่มจากระดัังใบนั้	<code>.sample()</code>
$\pi_\theta(a \mid s)$	scalar	ความหนาแน่นที่จุด a	<code>.log_prob(a).sum(-1)</code>
$\mu_\theta(s)$	เวกเตอร์ (8,)	ตำแหน่งกลางของระดััง	<code>self.actor(obs)</code>

ชนิดเปลี่ยนทุกแถว — ฟังก์ชัน → การแจกแจง → เวกเตอร์ → เลขตัวเดียว

policy ของ Tron1 ประกอบด้วยสามชิ้น

$$\pi_\theta(\cdot \mid s) = \mathcal{N}\left(\underbrace{\mu_\theta(s)}_{\text{① NN } 42 \rightarrow 8}, \underbrace{\sigma_\theta}_{\text{② logstd}}\right) \quad \text{③ ฐั ประดััง ง (คนเลื อก)}$$

ชิ้น	คืออะไร	ขนาด	เรียนไหม
① NN	ให้ตำแหน่งระดััง	187,272 weight	✓
② logstd	ให้ความกว้าง	8 ตัว	✓
③ สูตร $\mathcal{N}$	ให้รูปทรง	0 (เป็นบรรทัดโค้ด)	✗ ตายตัว

ถ้าจำได้สมการเดียว ให้จำสมการนี้ — ทุกหัวข้อย่อยที่เหลือใน §3.5 แรกออกมาจากมัน

สายนี้เดินผ่านวัตถุสี่ชนิด ไม่ใช่ชนิดเดียว — ประโยค " π_θ ส่งค่าให้ $\pi_\theta(\cdot \mid s)$ ส่งค่าให้ a " ทำให้เข้าใจผิดว่าทั้งสามเป็นของประเภทเดียวกันที่ไหลต่อกันเป็นสายน้ำ · ความจริงคือมันเปลี่ยนชนิดของวัตถุทุกครั้งทีข้ามลูกศร (ต่อไปนี้เรียกว่า ช่วงที่ 1-4 คนละเรื่องกับ ①②③ ข้างบนที่เป็นชิ้นส่วนของ policy):

ช่วงที่	สิ่งที่อยู่ตรงนั้น	เป็นวัตถุนิดอะไร	มี weight ไหม	อยู่นานแค่ไหน
1	s	tensor (42,) — ตัวเลขล้วน ๆ	✗	สแต็ปเดียวแล้วทิ้ง
2	① actor + ② logstd	NN + พารามิเตอร์ — ให้ $\mu(8,)$ กับ $\ell = \log \sigma(8,)$ (ยังไม่ใช่ σ)	✓ 187,280 ตัว	ค้างในแรมตลอดการเทรน
3	$\pi_\theta(\cdot \mid s)$	อ็อบเจกต์ Python <code>Normal</code> ที่เก็บแค่ $\mu(8,)$ กับ $\sigma(8,)$	✗	สร้างใหม่และทิ้งทุกสแต็ป
4	$a / \pi_\theta(a \mid s)$	tensor (8,) / scalar	✗	สแต็ปเดียวแล้วทิ้ง

shape ในตารางนี้เป็น "ต่อ 1 env" — ในโค้ดจริงทุกก้อนมี B (จำนวน env) นำหน้า: s เป็น (B,42) · μ, σ เป็น (B,8) · และ `self.distribution` เป็น `Normal` ก้อนเดียว ที่ถือระดััง $B \times 8$ ใ้พร้อมกัน ไม่ใช่ใ้เดียว ·

`.log_prob(a).sum(-1)` จึงคืน $(B,)$ ไม่ใช่ scalar เดี่ยว · ที่เขียนแบบ 1 env เพราะกำลังอธิบาย *ชนิดของวัตถุ* ไม่ใช่ขนาด

และลูกศรทุกเส้นไม่ใช่วัตถุ — มันคือบรรทัดโค้ดที่รันจริง

ลูกศร	บรรทัดที่ทำงาน	ไฟล์
1 → 2	<code>mean = self.actor(observations)</code>	actor_critic.py:156
2 → 3	③ สูตร $\mathcal{N} = \text{Normal}(\text{mean}, \exp(\text{logstd}))$	actor_critic.py:157
3 → 4 (สุ่ม)	<code>self.distribution.sample()</code>	actor_critic.py:161
3 → 4 (แทนค่า)	<code>self.distribution.log_prob(actions).sum(dim=-1)</code>	actor_critic.py:164

จุดที่คนพลาดบ่อยที่สุดคือ **ช่วงที่ 3** เพราะมองไม่เห็นว่ามันมีตัวตน — แต่ในโค้ดมันคือ `self.distribution` ที่จับต้องได้จริง มี attribute ให้อ่าน (`.mean`, `.stddev`) และ มี method ให้เรียก (`.sample()`, `.log_prob()`, `.entropy()`) · ถ้าข้ามช่วงนี้ไป **สมการ E4-E9** ทั้งหมดจะไม่มีที่ยืน

แล้ว a ที่เอาไปแทนค่าใน `.log_prob(a)` มาจากไหน — คำถามนี้สำคัญมาก เพราะ `.log_prob()` ไม่ได้ผลิต a เอง มันเป็นฟังก์ชันที่ต้องรับ a เข้าไป · ในโค้ดจริง `get_actions_log_prob` ถูกเรียก **สองที่** และ a มาคนละทาง:

ตอนไหน	a มาจากไหน	ระงับที่ถาม	ได้อะไร
เก็บข้อมูล (ทุก env step)	<code>.sample()</code> ที่เพิ่งเรียกบรรทัดก่อนหน้านี้ — ต้องสุ่มก่อนจริง ๆ	θ ปัจจุบัน	$\log \pi_{\text{old}}(a \mid s)$
อัปเดต (ทุก gradient step)	<code>actions_batch</code> จาก buffer — สุ่มไว้นานแล้วด้วย θ_{old}	θ ที่เพิ่งขยับ	$\log \pi_{\text{new}}(a \mid s)$

ตอนเก็บข้อมูล — [ppo.py:137](#) สุ่มก่อน แล้ว [ppo.py:147-149](#) ค่อยเอา action ตัวนั้นไปถามความหนาแน่น · ตรงนี้คุณคิดถูก **ลำดับบังคับ: สุ่มก่อน แทนค่าทีหลัง**

```
self.transition.actions = self.actor_critic.act(...).detach() # สุ่ม → a
...
self.transition.actions_log_prob = self.actor_critic.get_actions_log_prob(
    self.transition.actions).detach() # แทนค่า a ตัวเดิม
```

ตอนอัปเดต — [ppo.py:205-213](#) กลับกันสิ้นเชิง a ไม่ได้มาจาก `.sample()` ของรอบนี้เลย:

```
self.actor_critic.act(torch.cat((encoder_out_batch, obs_batch, commands_batch), dim=-1))
actions_log_prob_batch = self.actor_critic.get_actions_log_prob(actions_batch)
```

สังเกตบรรทัดแรก — **ไม่มีตัวแปรรับค่า** · เรียก `act()` เพื่อ *side effect* อย่างเดียว คือให้ `update_distribution` สร้างระงับใบใหม่ด้วย θ ปัจจุบัน ส่วน action ที่มันสุ่มออกมา **ถูกโยนทิ้งทันที** · แล้ว `actions_batch` ที่เอาไปแทนค่าคือ action เก่าจาก buffer

และนี่คือเหตุผลที่ ρ มีอยู่ได้เลย

$$\rho = \frac{\pi_{\text{new}}(a \mid s)}{\pi_{\text{old}}(a \mid s)} = \exp \left(\log \pi_{\text{new}}(a \mid s) - \log \pi_{\text{old}}(a \mid s) \right)$$

a ตัวเดียวกัน ถูกเอาไปถามระงับ **สองใบ** (θ_{old} กับ θ ปัจจุบัน) · ถ้าจำเป็นต้องสุ่มใหม่ทุกครั้งก่อนถาม จะได้ a คนละตัว แล้วเศษกับส่วนจะไม่ใช่เรื่องเดียวกัน — ρ จะไม่มีความหมายและ PPO ทั้งอัลกอริทึมจะพัง ([ppo.py:252-254](#))

สรุปคำตอบ — `.sample()` กับ `.log_prob()` เป็น *ปฏิบัติการ* คนละตัว (ตัวหนึ่งสุ่ม ตัวหนึ่งอ่านค่า) แต่ **ไม่ได้แปลว่าใช้แยกกันได้ตามใจ** · `.log_prob()` ต้องมี a จาก ที่ไหนสักแห่งเสมอ — ตอนเก็บข้อมูลได้มาจาก `.sample()` บรรทัดบน · ตอนอัปเดตได้มาจาก `buffer` · **จุดสำคัญคือ `.log_prob()` ไม่สนใจว่า a มาจากอะไรข้างไหน** มันแค่ อ่านค่าความหนาแน่นที่ตำแหน่ง a บนระนาบที่มีอยู่ตอนนั้น และความ "ไม่สนใจ" นี้ คือสิ่งที่ทำให้การนำข้อมูลเก่ามาใช้ซ้ำด้วย ρ เป็นไปได้

สังเกตว่าไม่มีช่วงไหนเลยที่เท่ากับ π_θ — ช่วงที่ 2 เป็นแค่ ①+② ส่วน ③ เป็น *ลูกศร* ไม่ใช่กล่อง เพราะมันคือบรรทัดโค้ดที่ไม่มี `weight ·  $\pi_\theta$` ตัวเต็มจึงกิน ตั้งแต่ช่วงที่ 2 ไปจนถึงลูกศร ③ — ส่วนช่วงที่ 3 ไม่ได้เป็นส่วนหนึ่งของฟังก์ชัน มันคือ **ค่าที่ฟังก์ชันคืนออกมา** · (ฟังก์ชันไม่ได้บรรจุผลลัพธ์ของตัวเองไว้ข้างใน — นี่คือการแบ่ง f กับ $f(x)$ เดิมที §3.5.2 จะพูดถึง)

และนี่คือคำตอบว่าทำไม "เรียก NN ว่า policy" ถึงขัดกันเอง — มันขัดจริง แต่ขัดเฉพาะ ตอนเทรน · ดูที่ *ชนิดของสิ่งที่คืนออกมา* แล้วจะเห็นทันที

ตอนไหน	เรียกอะไรจริง ๆ	คืนอะไร	ชนิด	NN เดี่ยว ๆ เป็น policy ไหม
เทรน	①+②+③	การแจกแจง	$\mathcal{S} \rightarrow \Delta(\mathcal{A})$	✗ ไม่ใช่ — <code>self.actor(obs)</code> คืนเวกเตอร์ ไม่ใช่ระนาบ
deploy	① อย่างเดียว	action ตรง ๆ	$\mathcal{S} \rightarrow \mathcal{A}$	✓ ใช่ — <code>act_inference</code> คืน μ แล้วจบ

ตอน deploy ② ถูกทิ้งและ ③ ไม่ถูกเรียก policy จึงกลายเป็น **deterministic ($a = \mu_\theta(s)$)** และ ณ จุดนั้น NN กับ policy คือของชิ้นเดียวกันเป๊ะ — `policy.onnx` จึงชื่อถูกต้อง 100%

กฎที่ใช้ตัดสินได้ทุกครั้ง — ถามว่า "สิ่งนี้คืนการแจกแจง หรือคืนเวกเตอร์" · คืนการแจกแจง → เป็น policy · คืนเวกเตอร์ → เป็นแค่ μ_θ ซึ่งเป็น **ชิ้นส่วน** ของ policy ยังไม่ใช่ policy · ระวางอย่างเดียวนั่นคือ **ตอน deploy นิยามของ policy เปลี่ยนไปเป็นแบบ deterministic** เกณฑ์นี้จึงใช้กับตอนเทรนเท่านั้น

เปิดรูป [E diagrams/tron1-policy-objects.html](#) ดูคู่กับตารางข้างบน — ในรูปนั้น **กล่อง = วัตถุ · ลูกศร = บรรทัดโค้ด** · กรอบเส้นประคือ "ช่วง" ทั้งสี่ · และวงเล็บสองอันล่างสุดคือขอบเขตของคำว่า **policy** ตอนเทรนกับตอน deploy

คำว่า "policy" ในชีวิตจริงมีสามความหมาย

ใครพูด	หมายถึง	ตรงกับ
ตำรา RL	ฟังก์ชัน $\mathcal{S} \rightarrow \Delta(\mathcal{A})$	① + ② + ③
โปรแกรมเมอร์	NN ที่ deploy ลงหุ่น	① อย่างเดียว
ไฟล์ <code>policy.onnx</code>	NN ที่ export แล้ว	① อย่างเดียว

ทั้งสามความหมาย**ตรงกันพอดีตอน deploy** เพราะ ② ถูกทิ้งและ ③ ไม่ถูกใช้ เหลือแค่ ① · จึงไม่มีใครพูดผิดเวลาเรียก `policy.onnx` ว่า policy · แต่**ตอนเทรนทั้งสามไม่ตรงกัน** — แถวที่ 2 กับ 3 กลายเป็นการเรียกชื่อแบบหลวม ๆ ที่ผิดตามนิยามของแถวแรก · กล้วย ๆ คือ ถ้าประโยคนั้นพูดถึงความน่าจะเป็นหรือการสุ่ม ต้องใช้นิยามแถวแรกเสมอ ถ้าไม่พูดถึงจะเรียก NN ว่า policy แบบล้าลองก็ได้ — แต่ต้องรู้ตัวว่ากำลังพูดหลวม ๆ อยู่

Deterministic กับ Stochastic ใช้คนละสัญกรณ์

แบบ	สัญกรณ์	คืนอะไร	ใครใช้
Stochastic	$a \sim \pi_\theta(\cdot s)$	การแจกแจง แล้วสุ่ม	Tron1 ตอนเทรน · PPO, SAC
Deterministic	$a = \mu_\theta(s)$	action ตรง ๆ	Tron1 ตอน deploy · DDPG, TD3

ต่างกันที่เครื่องหมาย $\sim$ กับ $=$ จุดเดียว และมันบอกทุกอย่าง

ประโยคปิดท้ายที่ควรจำ — policy ไม่ใช่ "ตัวเลือก action" แต่เป็น "ตัวออกความเห็นว่าจะ action น่าเล่นแค่ไหน" · ส่วนการเลือกจริงเป็นอีกขั้นที่ต่อจากนั้น — ตอน train ขั้นที่สองคือ **สุ่ม** ตอน deploy ขั้นที่สองคือ **หยิบจุดกลาง** · policy ตัวเดียวกัน แต่ขั้นที่สองคนละแบบ

3.5.1 ซิดตั้ง | ต่างจากคอมมา อย่างไร

$f(a, s)$ กับ $f(a \mid s)$ รับ input เท่ากันทั้งคู่ (สองตัว) และคืน scalar เหมือนกัน arity ไม่ต่างกันเลย สิ่งที่แตกต่างกันคือซิดตั้งประกาศเงื่อนไขเพิ่มหนึ่งข้อที่คอมมาไม่ประกาศ:

$$\int \pi_\theta(a \mid s) da = 1 \quad \text{สำหรับ } s \text{ ทุกค่า}$$

รวมทางฝั่งซ้ายของซิดได้ 1 เสมอ · รวมทางฝั่งขวาไม่มีความหมาย ($\sum_s \pi(a \mid s) \neq 1$) ซิดตั้งจึงเป็นตัวบอกว่า **ตัวไหนคือตัวแปรสุ่ม ตัวไหนคือสิ่งที่ตรงไว้** ถ้าเขียน $\pi(a, s)$ จะแปลว่าเป็น joint distribution — กระจายบนทั้งคู่ ซึ่งผิด เพราะ policy ไม่ได้บอกว่า state ไหนน่าจะเกิด (นั่นเป็นหน้าที่ของ P ใน MDP — ไม้ต 05 หัวข้อ 1.2)

ในวงเล็บนี้มีถึง สามช่อง คนละชนิด:

π
parameter

θ
ตัวแปรสุ่ม

$($

a
ตัวแปรสุ่ม

$|$

s
เงื่อนไข

$)$

ในทางโปรแกรมไม่มี operator สำหรับซิดตั้ง แต่มันตรงกับโครงสร้างโค้ดพอดี:

ในสูตร	ในโค้ด	ทำไม
ขวาสิด (s)	argument ของ constructor <code>Normal(...)</code>	ใช้กำหนดรูปร่างของ distribution
ซ้ายซิด (a)	argument ของ method <code>.log_prob()</code>	ค่าที่เอาไปถามว่าหนาแน่นเท่าไร
ห้อยล่าง (θ)	self ของ <code>nn.Module</code>	ไม่ได้ส่งเข้าไป มันอยู่ใน object

นี่คือเหตุผลที่ `update_distribution(obs)` กับ `get_actions_log_prob(actions)` เป็น **name method** — ถ้าซิดตั้งไม่มีความหมาย มันคงเขียนเป็น `log_prob(a, s)` ฟังก์ชันเดียวจบ

3.5.2 สามสัญลักษณ์ที่สับสนบ่อยที่สุด · และเครื่องหมาย = ระหว่างการแจกแจง

ก่อนอื่น — f กับ $f(x)$ ต่างกันยังไง

ทุกอย่างในหัวข้อนี้ตั้งอยู่บนความต่างข้อเดียว: **ตัวฟังก์ชัน** กับ **ค่าที่ฟังก์ชันคายออกมา** เป็นคนละสิ่งกัน

	คืออะไร	ทำอะไรกับมันได้
เครื่องซิงกาแพ	ตัวเครื่อง 1 เครื่อง	ยกไปวางที่อื่น · ถามว่ากินไฟกี่วัตต์
กาแพ 1 แก้วที่มันซง	ของเหลว 1 แก้ว	ดื่ม · วัตถุนหภูมิ

ดื่มเครื่องซิงไม่ได้ และเสียบปลั๊กแก้วกาแพไม่ได้ — คณิตศาสตร์เหมือนกันเป๊ะ:

	เขียนว่า	คืออะไร
ตัวเครื่อง	f	กฎการแปลง — ใส่อะไรเข้าไปได้อะไรออกมา

	เขียนว่า	คืออะไร
ของที่ออกมา	$f(x)$	ตัวเลขหนึ่งตัว

ให้ f = "ยกกำลังสอง" · ประโยคที่พูดถึง**ตัวเครื่อง** คือ "กราฟของ f เป็นพาราโบลา" หรือ " f มีค่าต่ำสุดที่ $x = 0$ " — พูดถึงทั้งเส้นกราฟ · ส่วน $f(3) = 9$ คือ**เลขตัวเดียว** จุดเดียวบนกราฟ

วิธีแยกที่ใช้ได้ทุกครั้ง — ถามว่า "ตอบเป็นตัวเลขได้ไหม" · f ตอบไม่ได้ เพราะมันคือ กฎ/กราฟทั้งเส้น · $f(3)$ ตอบได้ คือ 9

เอามาใช้กับ policy — ช่วง e0 t0: $\mu = 0.20$ และ $\sigma = 0.5$ ระบั้งอันนี้กำหนดความสูงไว้ให้ **ทุกค่าของ a ที่เป็นไปได้** ไม่ใช่แค่ค่าเดียว:

a	-1.0	0.0	0.20	0.40	1.5	... ทุกค่าบนเส้นจำนวนจริง
ความสูง	0.0448	0.7365	0.7979	0.7365	0.0272	...

- ทั้งตารางรวมกัน (ที่จริงยาวไม่รู้จบ) = $\pi_\theta(\cdot \mid s)$ — **ตัวเครื่อง**
- หนึ่งช่องในตาราง = $\pi_\theta(a \mid s)$ — **ของที่ออกมา**

ในโค้ดก็แยกกันชัดเจนเหมือนกัน · `print(dist)` จะได้ `Normal(loc: 0.2, scale: 0.5)` ไม่ใช่ตัวเลข เพราะมันคือเครื่อง ไม่ใช่กาแฟ:

```
dist = Normal(0.20, 0.5)      # ← ตัวเครื่อง : ตัวแปล dist เก็บระบั้งทั้งอัน
dist.log_prob(0.40)          # ← หนึ่งช่อง : เลขตัวเดียว (-0.3058)
```

ศัพท์ที่ใ้ที่นี่ใช้ — เรียก $\pi_\theta(\cdot \mid s)$ ว่า "**ตัวการแจกแจง**" และบางทีเรียก ให้เห็นภาพว่า "**ก้อนหมอกความน่าจะเป็น**" (ระบั้งทั้งอันที่คลุมทุกค่าของ a) · สองคำนี้หมายถึงสิ่งเดียวกัน คือ object ในตารางข้างบน

จุดกลมคือช่องว่างที่ยังไม่ได้เติม — เป็นสัญลักษณ์ที่บอกว่า "**ยังไม่ได้กดปุ่ม ยังไม่มีกาแฟ ออกมา นี่คือตัวเครื่องนะ**"

π_θ ไม่ใช่ NN — NN คายแค่ $\mu \cdot \pi_\theta = \text{NN} + \sigma$ + ฐประบั้ง `logstd` เป็น `nn.Parameter` แยกต่างหาก ไม่ได้ออกมาจาก NN เลย ([actor_critic.py:118](#))

เขียนว่า	a อยู่ตรงไหน · แปลว่าอะไร	คืออะไร	shape	โค้ด
$\mu_\theta(s)$	ไม่มี a เลย — สูตรนี้ไม่เกี่ยวกับ a	เวกเตอร์ค่ากลาง	(B, 8)	<code>self.actor(obs)</code>
$\pi_\theta(\cdot \mid s)$	ไม่มี a — วงเล็บว่าง (จุดกลม)	object ไม่ใช่ตัวเลข	—	<code>Normal(mean, std)</code>
$a \sim \pi_\theta(\cdot \mid s)$	a อยู่ หน้าเครื่องหมาย $\sim \rightarrow a$ คือผลลัพธ์ที่โผล่ออกมา	action จริง	(B, 8)	<code>.sample()</code>
$\pi_\theta(a \mid s)$	a อยู่ ในวงเล็บ $\rightarrow a$ คือสิ่งที่เราป้อนเข้าไปถาม	scalar	(B,)	<code>.log_prob(a).sum(-1)</code>

กฎจากตำแหน่งของ a — อยู่หน้า $\sim$ แปลว่า **ยังไม่มี a กำลังจะสร้างมันขึ้นมา** · อยู่ในวงเล็บแปลว่า **มี a อยู่แล้ว กำลังเอาไปถามคำถามกับมัน** · ลำดับเวลาจึงเป็น `.sample()` ก่อนเสมอ แล้วค่อย `.log_prob(a)`

$\sim$ ไม่ใช่การเรียกฟังก์ชัน — และนี่ไม่ใช่ "ฟังก์ชันเดียวที่คืนค่าได้สองแบบ"

สองบรรทัดล่างของตารางหน้าตาใกล้เคียงกันมากจนดูเหมือนเป็นฟังก์ชันเดียวกันที่คืนค่าได้สองแบบ (แบบหนึ่งสุ่มให้ a อีกแบบรับ a แล้วคืนความหนาแน่น) — **ไม่ใช่** ใช้นิยามของคำว่า ฟังก์ชันตัดสินได้เลย:

ฟังก์ชัน = input เดิม ต้องได้ output เดิมเสมอ

`.sample()` ละเมิดข้อนี้เต็ม ๆ — เรียกสองครั้งด้วย s เดิมเป๊ะ ได้คนละคำตอบ มันจึง**ไม่ใช่** การเรียกฟังก์ชันตั้งแต่แรก และนั่นคือเหตุผลที่สัญกรณ์ใช้ $\sim$ ไม่ใช่ $=$

สิ่งที่เกิดขึ้นจริงคือ `dist` เป็นวัตถุที่ถือตัวเลขสองตัว (μ กับ σ) แล้วมี **ปฏิบัติการหลายอย่าง** ที่หยิบตัวเลขคู่้นั้นไปใช้คนละแบบ:

ปฏิบัติการ	สัญกรณ์	ต้องมีวัตถุดิบเพิ่ม	คืนอะไร	สุ่มไหม
ประเมินความหนาแน่น	$\pi_\theta(a \mid s)$	ค่า a	scalar	ไม่ — a เดิมได้เลขเดิมเสมอ
สุ่ม	$a \sim \pi_\theta(\cdot \mid s)$	เลขสุ่ม ϵ	action (8,)	ใช่
ความกว้าง	$H[\pi_\theta(\cdot \mid s)]$	—	scalar	ไม่
ค่ากลาง	<code>.mean</code>	—	(8,)	ไม่

ดูคอลัมน์ **"ต้องมีวัตถุดิบเพิ่ม"** — การสุ่มต้องการของที่**ไม่ได้**อยู่ใน μ, σ เลย คือ เลขสุ่ม ϵ · ในโค้ด `sample()` ไปหยิบมันจากเครื่องสุ่มของ PyTorch เอง (`torch.normal`) — ถ้าล็อก seed ไว้ ผลจะซ้ำเดิมทันที ซึ่งพิสูจน์ว่ามี input ที่มองไม่เห็นซ่อนอยู่จริง พอตรึง input ตัวนั้น มันก็กลายเป็นฟังก์ชันปกติ

อุปมาที่ไม่พัง — ลูกเต๋าส่องน้ำหนัก

ในอุปมา	สัญกรณ์	ใน Tron1 จริง
ตารางความน่าจะเป็นของลูกเต๋า	$\pi_\theta(\cdot \mid s)$	ระบั้ง $\mathcal{N}(0.20, 0.5)$
ทอย แล้วออกหน้า 6 · ทอยใหม่ได้คนละหน้า	$a \sim \pi_\theta(\cdot \mid s)$	สุ่มได้ $a = 0.40$
เปิดตารางดู ว่าหน้า 6 มีโอกาสเท่าไร · เปิดกี่ครั้งก็ได้เลขเดิม	$\pi_\theta(a=6 \mid s)$	$\pi_\theta(0.40 \mid s) = 0.7365$

ทั้งสองอย่างใช้ตารางใบเดียวกันแต่เป็นคนละกิจกรรม และ**ไม่มีการ "ยัดหน้า 6 กลับเข้าไป ในลูกเต๋า"** เพราะการเปิดตารางดูไม่ได้ทำอะไรกับลูกเต๋aley แค่อ่านค่าจากกระดาน

เลข 6 ในแถวล่างคือค่า a นั่นเอง — จะถามหน้าไหนก็ได้ ตารางมีค่าให้ทุกหน้า และไม่ต้อง ทอยก่อนด้วยซ้ำ · แต่ **PPO เลือกถามหน้าที่ทอยได้จริงเสมอ** เพราะคำถามที่ต้องการคำตอบคือ *"policy ชอบ action ที่มันเล่นไปจริง ๆ แค่ไหน"* — ค่า a ที่ป้อนเข้าไปจึงมาจาก buffer ตลอด (ดู E12)

ข้อจำกัดของอุปมานี้ — ลูกเต๋ามี 6 หน้านับได้ ส่วน action ของ Tron1 เป็นจำนวนจริง ต่อเนื่อง · ค่าที่เปิดตารางลูกเต๋าคือ **ความน่าจะเป็น** (รวมทุกช่อง = 1 เกิน 1 ไม่ได้) ส่วนของ Gaussian คือ **ความหนาแน่น** (พื้นที่ใต้กราฟ = 1 แต่ตัวค่า เกิน 1 ได้ เช่น $\sigma = 0.25$ ยอดระบั้งสูง 1.596 — โน้ต 05 §2.3) · ใช้ลูกเต๋าทแทน "ทอย vs เปิดตารางดู" ได้ แต่ห้ามลากไปสรุปเรื่องขนาดของค่า

และอุปมาเครื่องชั่งกาแฟข้างบนใช้ได้เฉพาะความต่างระหว่าง *ตัวเครื่อง* กับ *ของที่ออกมา* (f กับ $f(x)$) เท่านั้น · ใช้กับเรื่องสุ่ม-vs-ประเมินไม่ได้ เพราะไม่มีใครเอากาแฟยัด กลับเข้าเครื่อง

เครื่องหมาย = ในบรรทัด $\pi_\theta(\cdot \mid s) = \mathcal{N}(\mu, \sigma)$ แปลว่าอะไร

จุดที่สะดุดกันมากที่สุดคือ "สองข้างเป็นวัตถุคนละอย่างกัน แล้วเท่ากันได้ยังไง" — คำตอบคือ มันเป็นวัตถุชนิดเดียวกัน ทั้งคู่:

	ข้างซ้าย $\pi_\theta(\cdot \mid s)$	ข้างขวา $\mathcal{N}(\mu, \sigma)$
เป็นวัตถุชนิดอะไร	ฟังก์ชัน $\mathbb{R}^8 \rightarrow \mathbb{R}_{\geq 0}$	ฟังก์ชัน $\mathbb{R}^8 \rightarrow \mathbb{R}_{\geq 0}$
รับอะไร	action หนึ่งชุด	action หนึ่งชุด
คืนอะไร	ความหนาแน่น (เลขตัวเดียว)	ความหนาแน่น (เลขตัวเดียว)
มาจากไหน	"สิ่งที่ policy คายออกมาตอนอยู่ที่ s "	"สูตระวังที่มีพารามิเตอร์ μ, σ "

ทั้งคู่เป็นสมาชิกของเซตเดียวกัน คือเซตของการแจกแจงบน $\mathbb{R}^8$ — พอเป็นสมาชิก เซตเดียวกัน ก็ใส่เครื่องหมายเท่ากับได้ตามปกติ เหมือนเขียน $f = g$ สำหรับฟังก์ชันสองตัว

= ระหว่างฟังก์ชัน แปลว่า "เท่ากันที่ทุกจุด input" ถ้าเขียนไม่ย่อคือ

$$\pi_\theta(a \mid s) = \mathcal{N}(a; \mu, \sigma) \quad \text{สำหรับ } a \text{ ทุกค่าที่ 'เป็นไปได้'}$$

ตรวจด้วยตัวเลขได้ ($\mu = 0.20, \sigma = 0.5$, ดูมิติเดียว):

ป้อน a	ข้างซ้ายให้	ข้างขวาให้
0.40	0.7365	0.7365
0.20	0.7979	0.7979
1.50	0.0272	0.0272

ซ้ายคือ "ตำแหน่ง" ขวาคือ "คนที่มานั่งตำแหน่งนั้น" — เหมือนประโยค "ผู้จัดการฝ่ายขาย = สมชาย" ซึ่งเท่ากันได้เพราะทั้งสองข้างชี้ไปที่คนคนเดียวกัน · ข้างซ้ายบอก บทบาท ("ตัวแจกแจงที่ policy ใช้เลือก action ตอนอยู่ที่ s " — ยังไม่บอกว่าหน้าตาอย่างไร) ข้างขวา บอก ตัวจริงที่มารับบทนั้น (ระวังที่มีพารามิเตอร์สองตัว)

และนี่คือประเด็นสำคัญที่สุด: สมการนี้ไม่ได้พิสูจน์มา แต่เป็นการ **เลือก** ของคนออกแบบ ไม่มีกฎธรรมชาติข้อไหนบังคับว่า policy ต้องเป็นเกาส์เซียน — ทีม rsl_rl ตัดสินใจให้มันเป็น แล้วเขียนลงโค้ด ถ้าเลือกอย่างอื่นก็ได้:

ถ้าเลือก...	สมการจะเป็น	ใช้เมื่อไหร่
เกาส์เซียน	$\pi_\theta(\cdot \mid s) = \mathcal{N}(\mu, \sigma)$	action ต่อเนื่อง ไม่มีขอบ ← Tron1 ใช้อันนี้
Beta	$\pi_\theta(\cdot \mid s) = \text{Beta}(\alpha, \beta)$	action มีขอบชัด เช่น $[-1, 1]$
Categorical	$\pi_\theta(\cdot \mid s) = \text{Cat}(p_1, \dots, p_k)$	action เป็นตัวเลือก เช่นเกม Atari
Tanh-Normal	$\pi_\theta(\cdot \mid s) = \tanh(\mathcal{N}(\mu, \sigma))$	SAC ใช้

ข้างซ้ายเหมือนกันหมด ข้างขวาต่างกัน — ข้างซ้ายคือ **interface** ข้างขวาคือ **implementation**

แล้วทำไมต้องมีระวัง — ทำไมไม่เทรนให้ NN คาย action ออกมาตรง ๆ

คำถามนี้มีสองชั้น และชั้นที่สองคือเหตุผลจริง

ชั้นที่ 1 · NN สุ่มไม่ได้ — nonlinear ไม่ใช่ random

ความไม่เชิงเส้นกับความสุ่มเป็นคนละเรื่องกันสิ้นเชิง:

ความสามารถ	NN มีไหม	ได้มาจากไหน
แปลง input เป็น output ที่ซับซ้อนแค่ไหนก็ได้	มี (universal approximation)	ELU + หลายชั้น
ให้คำตอบต่างกันเมื่อ input เหมือนเดิม	ไม่มี	ต้องมี RNG จากข้างนอกเท่านั้น

ELU ไม่ได้สร้างความสับสน มันแค่ทำให้เส้นโค้งงอได้ · ป้อน s เดิมเข้า NN พ้นครั้ง ได้เลขเดิม เป๊ะพ้นครั้ง — นี่คือนิยามของ deterministic function · ดังนั้น "ให้ NN คาย action ตรง ๆ" = deterministic policy = ทำ action เดียวต่อ state = ไม่มีการสำรวจ

ขั้นที่ 2 · PPO ต้องการ ตัวเลข $\pi(a \mid s)$ ไม่ใช่แค่ action

สมมติมีกล่องดำที่คาย action สุ่มได้เองโดยไม่ต้องมีระบบ — PPO ก็ยังใช้ไม่ได้อยู่ดี:

สมการ	ถ้าไม่มีระบบ
E6 สุ่ม action	กล่องดำทำได้ ✓
E7-E8 ความหนาแน่น	คำนวณไม่ได้ ไม่รู้สูตร ✗
E9 $\log \pi(a \mid s)$	ไม่มี ✗
E13 $\rho = \pi_{\text{new}} / \pi_{\text{old}}$	เขียนไม่ได้ ✗
E14 L^{CLIP}	ไม่มี loss ให้ backward ✗

เหตุผลจริง — เราไม่ได้เลือก Gaussian เพราะอยากได้ความสับสน (ความสับสนหาที่ไหนก็ได้ ถูกมาก) แต่เลือกเพราะมันเป็นตัวสุ่มที่ ย้อนกลับไปคำนวณได้ว่าโอกาสที่จะสุ่มได้แบบนี้ มีเท่าไร ในสูตรปิดสามพจน์

Gaussian ให้สองอย่างพร้อมกัน ซึ่งหายาก — และทั้งคู่เป็นแค่บวกลบคูณหาร จึงทำ $12,288 \times 20 = 245,760$ ครั้งต่อ iteration (= ทุกช่อง 49,152 ช่อง $\times$ 5 epoch) ไหว:

$$a = \underbrace{\mu + \sigma \epsilon}_{\text{สุ่มง่ายมาก}}$$

$$\log \pi = \underbrace{-\frac{(a-\mu)^2}{2\sigma^2} - \log \sigma - 0.919}_{\text{ประเมิน นัย ่อนหลัง ง่ายมาก}}$$

แล้วความสามารถ nonlinear ของ NN ถูกใช้เต็มที่ไหม — เต็มที่อยู่แล้ว

$$\pi_\theta(\cdot \mid s) = \mathcal{N}\left(\underbrace{\mu_\theta(s)}_{\text{ตรงนี้ \textbf{nonlinear} เต็มรูปแบบ}}, \sigma\right)$$

ระบบไม่ได้จำกัดความซับซ้อนของการตัดสินใจเลย มันจำกัดแค่รูปทรงของการกระจายรอบ ๆ คำตอบ · เทียบเป็นภาพ: มือป้อนที่เล็งเก่งมาก (NN คิดซับซ้อน) แต่กระสุนกระจายรอบเป้า เป็นวงกลม (Gaussian) — ความเก่งอยู่กับการเล็ง ไม่ได้อยู่ที่รูปทรงการกระจาย

ทางเลือกอื่นที่มีจริงในวงการ

แนวทาง	ทำยังไง	ทำไม Tron1 ไม่ใช้
NN คาย σ ด้วย	output 16 ตัว ($8 \mu + 8 \log \sigma$)	ทำได้จริง หลายงานทำ · rsl_rl เลือกไม่ทำเพราะเทรนนิ่งกว่า
Deterministic + noise ข้างนอก	DDPG / TD3	คำนวณ ρ ไม่ได้ → ต้องเปลี่ยนไปใช้ off-policy + Q-function ทั้งระบบ
หั่น action เป็นช่วง ๆ	Categorical — NN คาย logits แล้วสุ่ม	8 บิต $\times$ k ระดับ = ทางเลือกระเบิด และเสียความต่อเนื่องของมุม บิต
Normalizing flow	คาย action ตรง ๆ และคำนวณ density ได้ด้วย	แพงกว่ามาก เกินจำเป็นสำหรับ locomotion

แถวที่สองคือ "ให้ NN คาย action ตรง ๆ" ของจริง — มีคนทำ แต่ต้องทิ้ง PPO ทั้งตัวแล้ว เปลี่ยนไปใช้อัลกอริทึมคนละตระกูล

สรุป — อะไรตายตัว อะไรถูกเรียน

จุดที่คนสรุปผิดพลาดที่สุดคือคิดว่า "แทนไปเรื่อย ๆ จนมันกลายเป็น normal distribution" — ไม่ใช่ · มันเป็น normal ตั้งแต่วินาทีแรกที่รับโค้ด และเป็น normal ตลอดไปจนวินาทีสุดท้าย เพราะรูปทรงระฆังไม่ได้มาจากการเรียน แต่มาจากบรรทัด `Normal(...)` ที่คนเขียนไว้ตายตัว สิ่งที่ยืนยันคือ **ระฆังใบนั้นควรวางตรงไหน และควรแคบแค่ไหน**

	ตายตัวตั้งแต่แรก (คนออกแบบเลือก)	ถูกเรียน (Adam ปรับ)
รูปทรง = ระฆัง Gaussian	✓ ไม่มีวันเปลี่ยน	—
จำนวนมิติ = 8	✓	—
σ ไม่ขึ้นกับ s	✓ (เป็นการออกแบบ)	—
ตำแหน่งระฆัง $\mu_\theta(s)$	—	✓ ผ่าน weight 187,272 ตัว
ความกว้าง σ	—	✓ ผ่าน logstd 8 ตัว

สองแถวล่างถูกฝึกด้วย Adam ตัวเดียวกัน — `optim.Adam(self.actor_critic.parameters())` ที่ [ppo.py:85](#) ครอบคลุมทั้ง NN และ logstd (encoder มี optimizer แยกที่บรรทัด 88)

	ระฆังหน้าตาอย่างไร
iteration 0	เป็นระฆังเรียบร้อยแล้ว แต่วางผิดที่ (weight มั่ว) และกว้างสุด ($\sigma = 1.0$ เพราะ <code>logstd = 0</code>)
ระหว่างเทรน	ค่อย ๆ เลื่อนไปวางถูกที่มากขึ้นสำหรับแต่ละ s และค่อย ๆ แคบลง (ดู <code>mean_noise_std</code> ใน tensorboard)
ตอน deploy	ไม่สร้างระฆังแล้ว ใช้แต่จุดกลาง μ

และอย่าลืมว่า π_θ กับ $\pi_\theta(\cdot | s)$ คนละอย่างกัน — π_θ คือ *policy ทั้งหมด* (เครื่องที่รับ state ใดก็ได้แล้วคายระฆังออกมา) ส่วน $\pi_\theta(\cdot | s)$ คือ *ระฆังหนึ่งใบ* ที่ได้จากการป้อน s เข้าไป · หุ่นยนต์หนึ่งได้ระฆังใบหนึ่ง ขยับไปอีกท่า ได้อีกใบ — policy เดียวกัน แต่คนละระฆัง

แล้ว s หายไปไหนในข้างขวา — ซ่อนอยู่ใน μ เขียนเต็มไม่ย่อคือ

$$\pi_\theta(\cdot | s) = \mathcal{N}(\underbrace{\mu_\theta(s)}_{s \text{ อยู่ตรงนี้}}, \underbrace{\sigma_\theta}_{\text{ไม่มี } s})$$

s เปลี่ยน $\rightarrow \mu$ เปลี่ยน $\rightarrow$ ระฆังย้ายตำแหน่ง $\rightarrow$ ได้การแจกแจงคนละตัว · ส่วน σ ไม่มี s อยู่在那นั่น ระฆังจึงกว้างเท่าเดิมเสมอไม่ว่าจะอยู่ state ไหน

= **ตัวนี้คือ = ตัวเดียวกับในโค้ดเป๊ะ ๆ** — `self.distribution = Normal(mean, std)` ทั้งสองที่หมายถึง "ฝั่งขวาสร้างวัตถุขึ้นมา แล้วให้ชื่อฝั่งซ้ายชี้ไปที่มัน" เรียกว่า *definitional equality* (นิยามให้เท่า) ไม่ใช่ *derived equality* (พิสูจน์ได้ว่าเท่า) · บางตำราจึงเขียน $:=$ หรือ $\triangleq$ เพื่อย้ำจุดนี้

ทำไมตำรานิยมเขียนรูปย่อ — มันไม่ได้ย่อเพื่อประหยัดที่ แต่พูดคนละเรื่อง

	รูปเต็ม $\pi_\theta(a s) = \mathcal{N}(a; \mu, \sigma)$	รูปย่อ $\pi_\theta(\cdot s) = \mathcal{N}(\mu, \sigma)$
พูดถึง	ค่า ณ จุด a จุดหนึ่ง	ตัวการแจกแจง ทั้งหมด
เป็นประโยคว่า	"ที่จุดนี้ ความสูงเท่ากัน"	"policy คือ ระฆังอันนี้"
ทำอะไรต่อได้	เอาเลขไปคำนวณ	สุ่มจากมัน · หา entropy · หา KL

เหตุผลจริงคือ มีคำถามหลายอย่างที่ถามกับค่าจุดเดียวไม่ได้ — สามสมการนี้ใช้จริงในโค้ด Tron1 และทุกอันต้องการตัวการแจกแจงทั้งอัน:

$$a \sim \pi_\theta(\cdot \mid s) \quad H[\pi_\theta(\cdot \mid s)] \quad D_{KL}(\pi_{\theta_{old}}(\cdot \mid s) \parallel \pi_{\theta_{new}}(\cdot \mid s))$$

- **สุ่ม** — ต้องมีระยะบังทั้งอันก่อน ถ้ามีแค่ "ความสูงที่ $a = 0.40$ " สุ่มไม่ได้
- **entropy** — "ระยะบังกว้างแค่ไหน" เป็นสมบัติของระยะบังทั้งอัน จุดเดียวตอบไม่ได้
- **KL** — "สองระยะบังนี้ต่างกันแค่ไหน" ต้องมีสองตัวมาเทียบ

ชิดตั้งสองอัน || ในสมการ KL คืออะไร

มันคือ **ตัวค้น** ไม่ใช่ตัวดำเนินการ · อ่านว่า "เทียบกับ" — $D_{KL}(p \parallel q)$ แปลว่า " **p ต่างจาก q แค่ไหน**" · ไม่ใช่หารด้วย ไม่ใช่ "หรือ" ไม่ใช่ norm · ตัวมันเองไม่ได้คำนวณอะไรเลย แค่แยกของสองชิ้นที่ป้อนเข้า D_{KL}

ถ้าเขียนแบบอื่น	ถ้าไม่ใช้
$D_{KL}(p, q)$	คอมมาชวนให้คิดว่าสลับที่ได้ เหมือน $\max(a, b)$ — แต่สลับไม่ได้
$D_{KL}(p \mid q)$	ชิดเดียวถูกจองไว้แปลว่า "โดยมีเงื่อนไขว่า" แล้ว (§3.5.1) · เกมในวงเล็บมีชิดเดียวอยู่แล้วใน $\pi(\cdot \mid s)$ จะชนกันเอง
$D_{KL}(p \parallel q)$	✓ ชิดสองอันเป็น ป้ายเตือนว่าลำดับสำคัญ

ทำไมลำดับถึงสำคัญ — เพราะนิยามของมันเลยบนตัวซ้ายเท่านั้น

$$D_{KL}(p \parallel q) = \mathbb{E}_{a \sim p}[\log p(a) - \log q(a)]$$

ตัวซ้าย = ตัวที่เราสุ่มมาเฉลี่ย · **ตัวขวา** = ตัวที่ถูกเอามาเทียบ หน้าทีคนละอย่าง สลับแล้วได้คนละเลข · ลองด้วยสูตรที่อยู่ในโค้ดนี้เอง (สมการ (30)) ใส่ระยะบังเก่าของ $e0$ to $\mu_{old} = 0.20, \sigma_{old} = 0.5$ กับระยะบังใหม่สมมติ $\mu_{new} = 0.25, \sigma_{new} = 0.45$ (ต้องให้ σ ต่างกันด้วย — ถ้า σ เท่ากัน KL สองทิศจะบังเอิญเท่ากัน)

รายมิติ (ช่อง ใจ ช่องเดียว)	ค่า
$D_{KL}(\pi_{old} \parallel \pi_{new})$	0.018096
$D_{KL}(\pi_{new} \parallel \pi_{old})$	0.015361 ← ต่างกัน 1.18 เท่า

(ของจริง axis=-1 บวกครบ 8 มิติก่อน — ถ้าทุกมิติหน้าตาแบบนี้จะได้ ≈ 0.145 ซึ่งสูงกว่า `desired_kl = 0.01` มาก แล้ว learning rate จะถูกหารด้วย 1.5 — สมการ (37))

ถ้าสลับที่ได้ สองบรทัดนี้ต้องเท่ากัน · มันไม่เท่า — **นี่คือสิ่งที่ || เตือน** และเป็น เหตุผลที่ KL ไม่ใช่ "ระยะทาง" (ระยะทางต้องสมมาตร) ตำราจึงเรียกว่า *divergence*

โค้ดใช้ลำดับไหน — `ppo.py:223-231` คำนวณ

$$\log \frac{\sigma_{new}}{\sigma_{old}} + \frac{\sigma_{old}^2 + (\mu_{old} - \mu_{new})^2}{2\sigma_{new}^2} - \frac{1}{2}$$

โค้ดไม่ตรงสูตรเป๊ะ ๆ — บรทัดจริงเขียน `torch.log(sigma_batch / old_sigma_batch + 1.0e-5)` คือมี 10^{-5} บวกอยู่ข้างในลอการิทึม กับ `log 0` · มันไม่เปลี่ยนว่าเป็น KL ทิศไหน แต่ทำให้ค่าสูงขึ้นนิดหน่อย (ตัวอย่างข้างบนจะได้ 0.018107 แทน 0.018096)

เทียบกับสูตรปิดของเกาส์เซียนแล้วคือ $D_{KL}(\pi_{old} \parallel \pi_{new})$ — ตรงกับลำดับที่ เขียนไว้ข้างบนพอดี

ทำไมตัวเก่าอยู่ซ้าย — และเหตุผลที่ผิด · บรรทัดนี้เป็นสูตรปิด อินพุตมีแค่ `mu_batch, sigma_batch, old_mu_batch, old_sigma_batch` · มันไม่แตะ `actions_batch` เลย ไม่มี a ที่เข้ามาเข้าไปในความคาดหมายสักตัว · ดังนั้นจะบอกว่า "ตัวเก่าอยู่ซ้ายเพราะ ข้อมูลถูกสุ่มด้วย policy เก่า" ไม่ถูก — สลับสี่อาร์กิวเมนต์ก็ได้อีกทีจาก tensor ชุดเดิม เหตุผลจริงคือ **ธรรมชาติของ TRPO/PPO**: วัดว่า policy ใหม่ขยับหนีของเก่าไปไกลแค่ไหน · ส่วนข้อมูลที่เก็บมาไปบังคับค่าเฉลี่ยชั้นนอก ต่างหาก คือ `torch.mean(kl)` ที่เฉลี่ยบน state ที่ policy เก่าเดินผ่าน

ค่านี้ไปไหนต่อ — `axis=-1` บวกครบ 8 มิติได้ KL ต่อ env shape (MB,) · แล้ว `kl_mean = torch.mean(kl)` ที่ :233 ยู่เป็นเลขตัวเดียว · `kl_mean` นี่แหละคือตัวที่เทียบกับ `desired_kl = 0.01` เพื่อปรับ learning rate (:236-244) · ส่วน `mean_kl` เป็นคนละตัว — เป็นค่าเฉลี่ยของ `kl_mean` ทั้ง 20 รอบ ที่ส่งขึ้น tensorboard อย่างเดียว (:295, `on_policy_runner.py:285`) **ไม่ได้ใช้ปรับ lr**

ระวังสับสนกับ norm — เครื่องหมายเดียวกันนี้ใช้เขียนความยาวเวกเตอร์ด้วย แยกด้วยตำแหน่ง: $\|x\|$ ครอบของชั้นเดียว = ความยาว · $D_{KL}(p \parallel q)$ คั่นกลางของสองชั้น = ตัวคั่น KL

ประโยคที่ควรจำ — ซิดเดียว = "โดยมีเงื่อนไข" · ซิดสองอัน = "เทียบกับ (ห้ามสลับ)"

เป็นเหตุผลเดียวกับที่เราพูดว่า "กราฟของ f เป็นพาราโบลา" แทนที่จะไล่บอกความสูงทีละจุด — สิ่งที่ต้องการพูดเป็นประโยคระดับตัวเครื่อง ไม่ใช่ระดับจุด · และมันตรงกับโค้ดพอดี: สร้างตัวการแจกแจงครั้งเดียว แล้วถามหลายคำถามกับตัวเดิม

```
dist = Normal(mean, std)      # ← รุปย่อ : สร้างตัวการแจกแจงครั้งเดียว
dist.sample()                # ↘
dist.log_prob(a).sum(-1)     # ↘ เติมจุดกลมด้วยคำถามต่าง ๆ กัน
dist.entropy().sum(-1)       # ↗
```

กฎอ่านที่ใช้ได้ทันที — เจอ · ในวงเล็บ ให้แปลว่า "ตัวการแจกแจงทั้งอัน" แล้วมองหา ว่าประโยคนั้นจะเอามันไปทำอะไร (สุ่ม? วัดความกว้าง? เทียบกับตัวอื่น?) · เจอตัวแปรจริงในวงเล็บ ให้แปลว่า "เลขตัวเดียว"

ตั้งแต่นี้เป็นต้นไป โน้ตนี้ใช้รูปย่อทั้งหมด — ถ้าสะกดตรงไหน ให้ทางเป็นรูปเต็ม ในหัวแล้วเติมคำว่า "สำหรับ a ทุกค่า" ต่อท้าย

3.5.3 สามทิศทางที่ต้องแยกให้ออก

ทิศ ก $\mu_\theta(s) \Rightarrow \pi_\theta$ — ประกอบร่าง (มีการคำนวณ) ต้องมี σ มาสมทบ จึงจะได้ π · เกิดที่ `actor_critic.py:155-157`

ทิศ ข $\pi_\theta \Rightarrow \mu_\theta(s)$ — อ่านฉลาก (ไม่มีการคำนวณ) เพราะ μ คือ พารามิเตอร์ตัวที่หนึ่งของ π อยู่แล้ว `action_mean` คือ `return self.distribution.mean` ซึ่งใน torch คือ `return self.loc` — tensor ตัวเดิมเป๊ะ

ทิศ ค $\pi_\theta \Rightarrow \pi_\theta(a | s)$ — ประเมินค่า (ต้องมี a ก่อน) เอา a หย่อนกลับ เข้าสู่ตรรกะข้าง ได้ scalar · เกิดที่ `actor_critic.py:163-164`

เปรียบเทียบ: μ, σ คือส่วนผสม · π คือขนมที่อบแล้ว · ทิศ ก คืออบ · ทิศ ข คือแกะฉลากดูว่าใส่อะไรไป — ไม่ใช่การแยกขนมกลับเป็นแป้งกับไข่ · ทิศ ค คือชิมแล้วให้คะแนนขึ้นที่หยิบมา · (ใช้ตัวอักษรไทย ก/ข/ค เพื่อไม่ชนกับ ①②③ ที่ §3.5.0 ใช้เรียกสามชั้นของ policy)

3.5.4 สมการทั้ง 15 ข้อ เรียงตามลำดับการส่งค่า

$$B = 2048 \text{ (ตอนเก็บข้อมูล)} \cdot MB = 12,288 \text{ (ตอนอัปเดต)}$$

ช่วง A — จาก state ไปถึง π_θ

ประโยคเดียวของช่วงนี้: จากสิ่งที่หุ่นรู้ $\rightarrow$ กลายเป็น "ความเห็นพร้อมความไม่มั่นใจ"

E1 · encoder เดาสิ่งที่วัดไม่ได้

$$\hat{v} = f_\psi(h)$$

รับ h (B, 360) $\rightarrow$ คำนวณ $\hat{v}$ (B, 3) · [ppo.py:136](#)

หมายถึงอะไร — "ดูประวัติ 10 step ล่าสุด (36 ค่า $\times$ 10) แล้วสรุปเป็นตัวเลข 3 ตัว ที่บอกสิ่งที่เซนเซอร์วัดตรง ๆ ไม่ได้"

ทำไม — obs 36 ค่า ณ วินาทีนั้นไม่พอจะรู้ว่าพื้นลื่นไหม ตัวเองกำลังไถลไหม แต่ *ลำดับ* ของ 10 step มีร่องรอยนั้นอยู่ (สั่งเดินหน้าเท่าเดิมแต่ท่าเปลี่ยน = พื้นเปลี่ยน) encoder คือเครื่องอ่านร่องรอย

ถ้าไม่มี — state ไม่เป็น Markov (โน้ต 05 §1.3) นโยบายต้องเดาแบบไม่มีข้อมูล

E2 · ต่อ input ของ actor

$$s = [\hat{v}; o; c]$$

รับ (B, 3), (B, 36), (B, 3) $\rightarrow$ คำนวณ s (B, 42) · [ppo.py:138](#)

หมายถึงอะไร — ต่อสามอย่างเป็นแถวเดียว: สิ่งที่เดาได้ ($\hat{v}$ — ค่าประมาณของ `base_lin_vel`, โน้ต 05 §4.4) + สิ่งที่วัดได้ (o) + สิ่งที่คนสั่ง (c)

ทำไม — NN รับได้แค่เวกเตอร์ก้อนเดียว · และ c ต้องอยู่ในนั้นด้วย ไม่จูน policy ไม่มีทางรู้ว่าจะให้เดินไปทางไหน จะได้หุ่นที่เดินท่าเดียวตลอด ไม่ฟังคำสั่ง — นี่คือที่มา ของเลข 42 ใน §1

E3 · NN คายค่ากลาง — นี่คือ $\mu_\theta(s)$

$$\mu = \mu_\theta(s) = W_4 \text{ELU}(W_3 \text{ELU}(W_2 \text{ELU}(W_1 s + b_1) + b_2) + b_3) + b_4$$

รับ s (B, 42) $\rightarrow$ คำนวณ μ (B, 8) · [actor_critic.py:156](#)

หมายถึงอะไร — "สถานการณ์แบบนี้ ท่าที่ฉันคิดว่าดีที่สุดคือ 8 คำนี" · ความรู้ทั้งหมด ที่เรียนมา 15,000 iteration ถูกบีบอยู่ในน้ำหนัก W, b ของสมการนี้

ทำไม — นี่คือ "การตัดสินใจ" ตัวจริง ทุกสมการที่เหลือนี้ไว้เพื่อให้สมการนี้ดีขึ้น

ทำไมเรียกค่ากลาง ไม่ใช่คำตอบ — ตอนเทรนเรายังไม่เชื่อมั่นเต็มร้อย ต้องเผื่อที่ให้อลอง ของใหม่ (E6) · ตอน deploy ถึงจะเชื่อเต็มร้อยแล้วใช้ μ เป็นคำตอบตรง ๆ

E4 · ความกว้างการสุ่ม (ไม่ผ่าน NN)

$$\sigma = \exp(\ell), \quad \ell \in \mathbb{R}^8 \text{ เป็น } \texttt{nn.Parameter}$$

รับ $\ell(8,) \rightarrow$ คืน $\sigma(8,)$ · [actor_critic.py:157](#) · ประกาศที่ :118

หมายถึงอะไร — "ตอนนี้กำลังลองนอกกลุ่มออกทางแค่นั้น" หนึ่งค่าต่อข้อ ทุก env ใช้ร่วมกัน

ทำไมต้อง exp — optimizer ขยับ ℓ ได้อิสระทั้งบวกและลบ แต่ $\exp$ บังคับให้ $\sigma > 0$ เสมอ · ถ้าเก็บ σ ตรง ๆ ไม่มีอะไรห้าม Adam ลากมันไปถึง 0 หรือติดลบ แล้ว $\log \sigma$ จะกลายเป็น NaN เจ็บ ๆ (validation ปิดไว้ที่ :121) — ดู §3.4 ประกอบ

ทำไมเริ่มที่ $\ell = 0$ — ได้ $\sigma = 1.0$ คือ "มั่วเต็มที่" ตอนเริ่ม แล้ว gradient จะ ค่อย ๆ ลดมันเองเมื่อหุ่นเริ่มมั่นใจ — ดูได้จาก mean_noise_std ใน tensorboard

σ ไม่ได้มาจาก critic — และไม่ได้มาจาก NN ตัวไหนเลย

เดาผิดที่พบบ่อยคือ "actor ให้ μ แล้ว critic ให้ σ " · ไม่ใช่ · critic ไม่ได้ ผลิต σ และไม่มีเส้น gradient ตรงเข้า logstd (ข้อควรระวังอยู่ในกรอบท้ายหัวข้อนี้)

	ได้มาจาก	เป็นอะไร	ขึ้นกับ s ไหม
μ	self.actor(observations) โดย observations = [v;o;c] (B,42) — NN 187,272 weight (:156)	เวกเตอร์ (B,8)	ขึ้น — เปลี่ยนทุก state
σ	torch.exp(self.logstd) (:157) — nn.Parameter ลอย ๆ 8 ตัว ประกาศที่ :118	เวกเตอร์ (8,) broadcast	ไม่ขึ้น — ทุก state ทุก env ใช้ชุดเดียวกัน
$V(s)$	self.critic(critic_obs) — NN 282,625 weight (:170-172)	หนึ่งค่าต่อ env — tensor (B,1) ไม่ใช่ scalar เดี่ยว	ขึ้น — แต่ไม่เกี่ยวกับ σ

สิ่งเดียวที่ critic ผลิต คือ $V(s)$ · มันถูกแปลงเป็น $\hat{A}$ ที่ช่วง B (rollout_storage.py:187-207 — GAE อยู่ที่ :187-201 ส่วน $\hat{A}$ ตัวที่ loss ใช้จริงถูกสร้างและ normalize ที่ :204-207) แล้วถูกใช้ที่ E14 · ไม่มี gradient จาก value loss วิ่งเข้า logstd เลย ($\partial L_{\text{value}} / \partial \ell = 0$)

แต่พูดว่า "critic ไม่เกี่ยวกับ σ เลย" ก็แรงเกินไป — $\hat{A}$ ที่สร้างจาก V เป็น ตัวคูณ ของ gradient ที่ขยับ ℓ อยู่ดี · critic ที่เพี้ยนจึงทำให้ σ ขยับ เร็ว/ช้าผิดปกติ · ที่ถูกต้องคือ critic ไม่ได้ผลิต σ และไม่มีเส้น gradient ตรง เข้า logstd — ไม่ใช่ "ไม่เกี่ยวกันเลย"

แล้ว σ เรียนรู้ได้ยังไง ถ้าไม่มี NN — nn.Parameter คือ tensor ที่ optimizer มองเห็นและขยับได้ เหมือน weight ทุกประการ แค่มันไม่มีใครป้อน input ให้มัน · Adam ตัวเดียวกับ actor ขยับมัน เพราะ ppo.py:85 optimize self.actor_critic.parameters() ซึ่งมัน logstd เข้าไปด้วย · สิ่งแปลก ๆ คือ entropy bonus (ดันให้กว้าง — entropy_coef = 0.01, ppo.py:272-277) สู้กับ surrogate loss ที่ โดยรวม บังคับให้แคบ · ระวัง: surrogate ไม่ได้บีบทุกตัวอย่าง — อนุพันธ์ของมันคือ $\partial L / \partial \ell_i = -\hat{A} \rho [((a_i - \mu_i) / \sigma_i)^2 - 1]$ ซึ่งถ้า $\hat{A} > 0$ และ $|a_i - \mu_i| > \sigma_i$ จะดัน σ กว้างขึ้น · การแคบลง เป็นผลรวมเชิงสถิติ (policy ไปกองที่ action ที่ $\hat{A}$ สูง) ไม่ใช่แรงเชิงกลรายตัวอย่าง

คัพ — σ คือ ส่วนเบี่ยงเบนมาตรฐาน ไม่ใช่ ความแปรปรวน · ความแปรปรวน คือ σ^2 · ใน `torch.distributions.Normal` ชื่อนี้ชื่อ `scale` และเก็บ σ

กลไกที่ σ เปลี่ยนค่า — ไล่ให้เห็นทั้งเส้น

บล็อกนี้ใช้คัพ L , gradient, optimizer, SGD, Adam, surrogate, entropy ทั้งหมด · ถ้าคำไหนยังไม่ชัด **S3.4**

นิยามไว้ครบแล้ว — โดยเฉพาะเรื่อง L (แอลใหญ่ = loss) กับ ℓ (แอลเล็ก = logstd) ที่หน้าตาเหมือนกันแต่คนละเรื่องสิ้นเชิง

จุดที่กำหังงคือ μ เป็น ผลลัพธ์ของการคำนวณ (ยัด s เข้า NN ออกมาเป็น μ) แต่ σ ไม่มีใครผลิต · มันคือ `exp` ของตัวเลข 8 ตัวที่นอนนิ่งอยู่ในแรม

```
self.logstd = nn.Parameter(torch.zeros(8)) # :118 ตัวเลข 8 ตัว เริ่มที่ 0
sigma = torch.exp(self.logstd) # ย่อจาก :157 อ่านออกมาใช้ ไม่ได้คำนวณจาก s
```

ถามว่า " σ มาจากไหน" → มาจากช่องเก็บของมันเอง · ถามว่า "อะไรเปลี่ยนค่ามันได้" → ระหว่างการเทรน มีตัวเดียวคือ `optimizer.step()` (`ppo.py:290`) · ไล่ `grep logstd` ทั้ง repo เจอแค่สามที่: ประกาศ :118 · อ่านใช้ :157 · อ่านไป `log on_policy_runner.py:269` — ไม่มี scheduler ไม่มี anneal ไม่มี `clamp_` (ที่เขียนกับได้อีกทางเดียวคือ `load_state_dict` ตอนโหลด checkpoint — `on_policy_runner.py:369`)

$$\ell_i \leftarrow \ell_i - \text{lr} \cdot \frac{\partial L}{\partial \ell_i} \quad (\text{รูป SGD — ดู } \texttt{\textbf{\textit{ท}} \textit{ค}} \text{ ก่อน ค่อยคุยเรื่องขนาด})$$

คำถามจึงยุบเหลือข้อเดียว: $\partial L / \partial \ell$ คืออะไร · loss มีสามพจน์ (`ppo.py:274-278`) แต่มีแค่สองพจน์ที่มี ℓ อยู่

พจน์ใน loss	$\partial / \partial \ell_i$	เพราะอะไร
<code>value_loss</code>	0	critic เป็น NN คนละก้อน (:111) ไม่มีเส้นถึง <code>logstd</code>
<code>entropy × entropy_coef = 0.01</code> (<code>cfg:97</code>)	$-0.01 \cdot 1 = -0.01$ เ๊ะ	torch เขียน $H = \frac{1}{2} + \frac{1}{2} \log 2\pi + \log \sigma = \frac{1}{2} \log 2\pi e + \ell$ (<code>torch/.../normal.py:105</code> — ไม่ได้อยู่ใน repo นี้) → $\partial H / \partial \ell_i = 1$
<code>surrogate_loss</code>	$-\hat{A}\rho(z_i^2 - 1)$	โดย $z_i = (a_i - \mu_i) / \sigma_i$

รวมเป็นสูตรเดียวที่ต้องจำ — ทิศที่ ℓ ขยับใน 1 gradient step

$$\ell_i \leftarrow \ell_i + \text{lr} \cdot \left[\underbrace{\frac{1}{\text{MB}} \sum_n \hat{A}_n \rho_n (z_{n,i}^2 - 1)}_{\text{surrogate — เฉลี่ยกัน 12,288 ตัวอย่าง}} + \underbrace{0.01}_{\text{entropy — ไม่ถูกเฉลี่ย}} \right]$$

ทำไมพจน์หนึ่งมี $\frac{1}{\text{MB}}$ อีกพจน์ไม่มี — `logstd` เป็นตัวเดียวกันทั้ง minibatch · พจน์ surrogate ต่างกันทุกตัวอย่าง `.mean()` จึงเฉลี่ยจริง · แต่ H ขึ้นกับ ℓ_i อย่างเดียว ทุกตัวอย่างได้ค่าเท่ากันหมด ผลรวม N ตัวหารด้วย N กลับมาเป็นตัวเดิม — สัมประสิทธิ์ที่ไปถึง ℓ_i จึงเป็น 0.01 เ๊ะ ไม่ใช่ $0.01/8$ หรือ $0.01/\text{MB}$

และ optimizer คือ Adam ไม่ใช่ SGD (`ppo.py:85`) · Adam กำด้วย $\text{lr} \cdot \hat{m} / (\sqrt{\hat{v}} + \epsilon)$ ซึ่ง normalize ขนาด gradient ทั้ง · สูตรข้างบนจึงใช้อ่าน ทิศ และ อัตราส่วนของสองแรง เท่านั้น — ขนาดกำจริงคุมด้วย `lr` ไม่ใช่ด้วย

ค่าตัวเลขในวงเล็บ · ถ้า gradient จาก surrogate เล็ก และพจน์ entropy ขีทางเดิมสม่ำเสมอ Adam จะขยาย 0.01 ขึ้นมาจนก้าวเกือบเต็ม 1x ได้ (เพื่อไข "สม่ำเสมอ" สำคัญ — ดู §3.4 ตาราง SNR ของ Adam)

แรงสองแรงสู้กัน

แรงที่ 1 — entropy bonus: ดันขึ้น ด้วยค่าคงที่ +0.01 ทุกสแต็ป ทุกมิติ ไม่สนว่า เกิดอะไรขึ้น

แรงที่ 2 — surrogate: ขึ้นกับว่าทำที่ได้อยู่ใกล้หรือไกลจุดกลาง · ใส่ค่าของ e0 t0: $\sigma_i = 0.5, \mu_i = 0.20$, $\hat{A} = \pm 0.7975, \rho = 1$ (สมการ (35) กลับเครื่องหมาย = แรงผลัก ℓ)

กรณี	a_i	$z_i^2 - 1$	พจน์ surrogate	ผล
ทำดี อยู่ไกล (เกิน 1σ)	1.00	+1.560	+1.244	σ กว้างขึ้น
ทำดี อยู่ใกล้	0.35	-0.910	-0.726	σ แคบลง
ทำแย่ อยู่ไกล	1.00	+1.560	-1.244	σ แคบลง
ทำแย่ อยู่ใกล้	0.35	-0.910	+0.726	σ กว้างขึ้น

(ใส่ค่านี้เป็นพจน์ surrogate ล้วน ยังไม่รวม entropy · บวก +0.01 เข้าไปได้
+1.254/ - 0.716/ - 1.234/ + 0.736 — ทิศไม่เปลี่ยนสักแถว · ช่อง e0 t0 จริง ($a = 0.40$, $z^2 - 1 = -0.84$) อยู่แถวที่สอง: -0.670 → แคบลง ตรงกับ §3.4)

อ่านเป็นประโยคเดียว: σ ขยับเพื่อให้ระว่างไปคลุมตรงที่ทำดี ๆ อยู่ — เจอของดี ใกล้จุดกลางก็ทางออกไปหา · ของดีอยู่ใกล้อยู่
แล้วก็หุบเข้าเล็กลง · ของแย่อยู่ไกลก็หุบเข้า เล็กเดินไปทางนั้น · ของแย่อยู่ใกล้ก็ทางออกไปหาที่อื่น

ทำไม mean_noise_std ถึงค่อย ๆ ลดจริง

กฎเจคือ $\mathbb{E}[\rho(z_i^2 - 1)] = 0$ เนะ · เหตุผล: ρ แปลงค่าเฉลี่ยจาก policy เก่ากลับไปอยู่ใต้ policy ปัจจุบัน ซึ่ง $z_i \sim \mathcal{N}(0, 1)$ พอดี จึง $\mathbb{E}[z_i^2] = 1$

ระวังอย่าตัด ρ ทิ้ง — a ถูกสุมด้วย μ_{old}, σ_{old} แต่ z_i ในสูตร gradient ใช้ μ, σ ปัจจุบัน · สองอันนี้ตรงกันเฉพาะ gradient step แรก จาก 20 step เท่านั้น · หลังจากนั้น $z \sim \mathcal{N}(0, 1)$ แล้ว และ ρ คือตัวที่ซ่อมให้ (การ clip ทำให้เพี้ยนไปเล็กน้อย)

แปลว่า ถ้า $\hat{A}$ ไม่สัมพันธ์กับ z^2 เลย แรงจาก surrogate เฉลี่ยแล้วเป็นศูนย์ เหลือแต่ entropy ที่ดันขึ้น → σ จะโตขึ้นเรื่อย ๆ

σ จะแคบลงได้ ก็ต่อเมื่อ $\hat{A}$ สูงไปเกาะกับ z^2 ตัวอย่างเป็นระบบ คือ "ทำดีมักอยู่ใกล้จุดกลาง" ซึ่งแปลว่า μ เดากแล้ว · σ ที่ลดลงคือ หลักฐาน ว่า actor เริ่มมั่นใจ ไม่ใช่ สาเหตุ

ตัวคูณอีกสองชั้นที่คุมขนาดก้าว — clip_grad_norm_ (:289) ตัดยอด norm ของ gradient ก่อน actor+critic+logstd (encoder มี optimizer แยกและ .step() ของตัวเอง ที่ :321) · และ 1x ที่ถูก KL ปรับขึ้นลงทีละ 1.5 เท่า โดยถูกหนีบไว้ในช่วง $[10^{-5}, 10^{-2}]$

ตัวอย่างที่ ρ หลุดกรอบ clip มี gradient เป็นศูนย์ไหม — แค่ครั้งเดียว

loss ใช้ `torch.max(surrogate, surrogate_clipped)` (:260) ดังนั้น `max` เป็นคนเลือกว่าจะใช้พจน์ไหน · อนุพันธ์ 0 ของ `clamp` จะมีผลก็ต่อเมื่อ พจน์ `clipped` ชนะเท่านั้น

$\hat{A}$	ρ	max เลือก	gradient
+	0.5 (ต่ำกว่ากรอบ)	พจน์ดิบ	ไหลเต็ม
+	1.5 (สูงกว่ากรอบ)	พจน์ clipped	0
-	0.5	พจน์ clipped	0
-	1.5	พจน์ดิบ	ไหลเต็ม

นี่คือความไม่สมมาตรที่ทำให้ PPO ไม่เคยขวางการแก้ผิด — รายละเอียดอยู่ในโน้ต 07 §5.3 · และแม้ในกรณีที่ gradient เป็น 0 ตัวอย่างนั้นก็ยังนับอยู่ในตัวหารของ `.mean()` และยังออกเสียงใน พจน์ entropy ตามส่วนของมันอยู่ดี

E5 · ประกอบร่างเป็น π — ทศ ก · สมการ (9)

$$\pi_{\theta}(\cdot | s) = \mathcal{N}(\mu, \sigma)$$

รับ μ (B,8) และ σ broadcast เป็น (B,8) → คำน `object` · `actor_critic.py:157`

หมายถึงอะไร — เปลี่ยนจาก "คำตอบเดียว" เป็น "ความเห็น + ความไม่มั่นใจ" · μ คือจุด · π คือก้อนหมอกความน่าจะเป็นรอบจุดนั้น

ทำไมทำโม — RL ต้องการสองอย่างที่ μ เดี่ยว ๆ ให้ไม่ได้: (1) ที่ทางให้ลองของใหม่ (2) มาตรฐานว่า "ชอบ action นี้แค่ไหน" เป็นตัวเลข — ทั้งคู่ต้องมีการแจกแจงก่อน

ทำไมต้องมี σ ด้วย — ถ้าไม่ลุ่ม gradient เป็นศูนย์เป๊ะ

ข้อนี้พิสูจน์ได้ตรง ๆ ไม่ต้องเชื่อคำอธิบาย · สมการ E15 (ท้ายหัวข้อนี้ ตอนนี้ยังไม่ต้องรู้ ที่มา ดูแค่วงเล็บตัวหนา) บอกว่า

$$\frac{\partial L}{\partial \mu_i} = -\hat{A} \rho \frac{(a_i - \mu_i)}{\sigma_i^2}$$

ถ้าไม่ลุ่ม แปลว่า $a_i = \mu_i$ เสมอ → วงเล็บเป็นศูนย์ → gradient เป็นศูนย์เป๊ะ ทุกมิติ ทุก step ตลอด 15,000 iteration หุ่นจะไม่ขยับเลยแม้แต่นิดเดียว นี่ไม่ใช่การเปรียบเทียบ แต่เป็นเลข 0 จริง ๆ ในสมการ

เห็นภาพด้วยตัวเลข — 3 env อยู่ในสถานการณ์เดียวกัน actor บอก $\mu = 0.20$ (env 1 คือ e0 t0 จริง อีกสองตัวสมมติ)

กรณีลุ่ม ($\sigma = 0.5$)

env	a ที่ลุ่มได้	reward	$\hat{A}$	$(a - \mu)$	ผลต่อ μ
1	0.40	1.0	+0.80	+0.20	ดึงขึ้น
2	-0.10	0.5	-0.80	-0.30	ดึงขึ้น (หนีจากที่แย่)
3	0.25	0.7	+0.10	+0.05	ดึงขึ้นนิดเดียว

สามตัวนี้ต่างกัน → เปรียบเทียบได้ → รู้ว่า 0.20 ควรขยับไปทาง 0.4

กรณีไม่ลุ่ม ($\sigma = 0$)

env	a	reward	$\hat{A}$	$(a - \mu)$	ผลต่อ μ
1	0.20	0.7	0.0	0	ไม่ยับ
2	0.20	0.7	0.0	0	ไม่ยับ
3	0.20	0.7	0.0	0	ไม่ยับ

ทุกตัวได้เหมือนกันหมด → ไม่มีอะไรให้เทียบ → critic ทยอยถูกทุกครั้ง advantage เป็น 0 หมด

อุปมา — ทำอาหารสูตรเดิมเป๊ะทุกวัน 15,000 วัน ทุกจานอร่อยเท่ากัน แล้วถามว่า "จะปรับสูตรยังไงให้ดีขึ้น" — ไม่มีคำตอบ เพราะไม่เคยลองใส่เกลือมากกว่านี้สักครั้ง

ส่วนข้อ (2) "มาตรวัดว่าชอบแค่ไหน" ก็พังเหมือนกัน: ρ ใน E13 คือการหาร $\pi_{\text{new}}/\pi_{\text{old}}$ ต้องมีเลขสองตัวมาหารกัน · ถ้า policy ไม่สุ่ม การแจกแจง จะกลายเป็น **เข็มนาฬิกา** — สูงเป็นอนันต์ที่จุด **0.20** เท่านั้น เป็นศูนย์ทุกที่อื่น ทำให้ $\rho = \infty/\infty$ หรือ $0/0$ คำนวณไม่ได้

สรุปหนึ่งประโยค — จุดเดียวไม่มีความชัน ะวังมีความชันทุกจุด และการเรียนรู้ทั้งหมด คือการไล่ตามความชัน

โค้ดจริงสองบรรทัด มี 4 ก้าวซ่อนอยู่:

```
def update_distribution(self, observations):
    mean = self.actor(observations)
    self.distribution = Normal(mean, mean * 0.0 + torch.exp(self.logstd))
    #
    #
```

actor_critic.py:155
E3
E4
E5 : สร้าง object

`mean * 0.0 + torch.exp(self.logstd)` เป็นลูกเล่น broadcast ล้วน ๆ: `mean*0.0` ได้ศูนย์ทรง (B,8) แล้วบวก (8,) เข้าไปเพื่อขยายให้ครบทุก env — ในทางคณิตศาสตร์คือ σ_θ ค่าเดียวที่ทุก env ใช้ร่วมกัน **ไม่ขึ้นกับ s** ต่างจาก $\mu_\theta(s)$

object ตัวนี้ **ไม่ได้คำนวณอะไรเลย** มันแค่จำ μ ไว้ในชื่อ `loc` และ σ ไว้ในชื่อ `scale` (torch/distributions/normal.py:54) ความเป็น "ะวัง" ยังไม่ถูกใช้ จนกว่าจะมีคน เรียก `.sample()` หรือ `.log_prob()`

ช่วง B — จาก π_θ ไปเป็น action จริง

ประโยคเดียวของช่วงนี้: จากความเห็น → การกระทำจริง · ที่เดียวในทั้งระบบที่การสำรวจเกิดขึ้น**

E6 · สุ่ม

$$\underbrace{a \sim \pi_\theta(\cdot | s)}_{\text{ประโยคที่ '1'}} \quad \Leftrightarrow \quad \underbrace{a = \mu + \sigma \odot \varepsilon, \quad \varepsilon \sim \mathcal{N}(0, I_8)}_{\text{ประโยคที่ '2'}}$$

ตัวเชื่อม

รับ object → คำนวณ a (B,8) · actor_critic.py:161

อ่านสมการนี้ทีละเครื่องหมาย

บรรทัดนี้คือ **ประโยคสองประโยคที่พูดเรื่องเดียวกัน** วางคู่กันเพื่อบอกว่าสองอันนี้เหมือนกัน

เครื่องหมาย	ชื่อ	อ่านว่า	ไม่ใช่
$\sim$	tilde	"ถูกสุ่มออกมาจาก"	ไม่ใช่ "ประมาณเท่ากับ"

เครื่องหมาย	ชื่อ	อ่านว่า	ไม่ใช่
•	จุดกลาง	ช่องว่างที่ยังไม่เต็ม = ตัวการแจกแจงกัซอัน (§3.5.2)	ไม่ใช่เครื่องหมายคูณ
	ขีดตั้ง	"โดยมีเงื่อนไข" (§3.5.1)	ไม่ใช่คอมมา
$\longleftrightarrow$	ลูกศรสองหัว	"ก็คือ / เทียบเท่ากับ"	ไม่ใช่ทิศทางการไหลของข้อมูล
$\odot$	Hadamard	คูณทีละตัวตรงตำแหน่ง — ในโค้ดคือ * เจย ๆ	ไม่ใช่ dot product ไม่ใช่คูณเมทริกซ์
$\mathcal{N}(0, I_8)$	—	เกาส์เซียนกลาง 0 กว้าง 1 8 ตัวที่อิสระต่อกัน	—

ลูกศร $\longleftrightarrow$ ไม่ได้ชี้ไปกลับ — มันไม่มีทิศทางในความหมายของ "คำไหลจากซ้ายไปขวา" แต่เป็นตัวเชื่อมประโยคแบบเดียวกับคำว่า "ก็คือ" ในภาษาไทย เช่น "ฝนตก $\longleftrightarrow$ พื้นเปียก" = สองอย่างนี้จริงพร้อมกันเสมอ ไม่ได้แปลว่าฝนวิ่งไปหาพื้นแล้ววิ่งกลับ

ลูกศร	ความหมาย	ตัวอย่างในโค้ดนี้
$\rightarrow$	"กลายเป็น / ไปยัง"	<code>(B, 42) → (B, 8)</code>
$\longrightarrow$	"ถูกส่งไปเก็บที่"	E10
$\leftarrow$	"ถูกกำหนดค่าใหม่เป็น" (assignment)	$\theta \leftarrow \theta - \alpha \nabla L$
$\longleftrightarrow$	"ก็คือ / เทียบเท่ากับ"	E6 ussgัดนี้

ทำไมเขียนสองแบบ

ประโยคที่ 1 $a \sim \pi_\theta(\cdot \mid s)$	ประโยคที่ 2 $a = \mu + \sigma \odot \varepsilon$
บอก "คืออะไร" — a คือของที่สุ่มมาจากการแจกแจงนี้	บอก "ทำยังไง" — เอา μ บวกด้วย σ คูณเลขสุ่ม
ภาษาคณิตศาสตร์	สูตรเชิงกระบวนการ — รับได้จริง แต่ <code>.sample()</code> ไม่ได้เขียนแบบนี้ (ดูตาราง <code>.sample()</code> / <code>.rsample()</code> ข้างล่าง)

ทำไมสองประโยคนี้นี้เท่ากัน — และทำไมต้องผ่าน $\mathcal{N}(0, 1)$ ด้วย

คำถามที่ควรค้างคือ "ในเมื่อมี μ กับ σ อยู่แล้ว ทำไมไม่สุ่มจาก $\mathcal{N}(\mu, \sigma)$ ตรง ๆ ต้องไปสุ่ม ε จาก $\mathcal{N}(0, 1)$ แล้วแปลงทำไม"

เพราะมันไม่ใช่สองทางเลือก มันคือสิ่งเดียวกัน · "สุ่มจาก $\mathcal{N}(\mu, \sigma)$ " เป็น ประโยคภาษาคน ส่วน $\mu + \sigma \odot \varepsilon$ คือวิธีทำประโยคนั้น — ไม่มีทางเลือก ที่สามให้เลือก

เหตุผลที่ลึกกว่านั้น: ะวังมีรูปเดียวในโลก คือ $\mathcal{N}(0, 1)$ · เกาส์เซียนทุกตัว คือใบนี้ที่ถูก **เลือน** ด้วย μ และ **ยืด** ด้วย σ เท่านั้น (คุณสมบัติ *location-scale*)

พิสูจน์ด้วยการเปลี่ยนตัวแปร · **ทำทีละมิติ i** เพราะ 8 มิติอิสระต่อกัน ค่อยคูณรวม ทีหลังที่ E9 — ถ้า $\varepsilon_i \sim \mathcal{N}(0, 1)$ และ $a_i = \mu_i + \sigma_i \varepsilon_i$ จะได้ $\varepsilon_i = (a_i - \mu_i) / \sigma_i$ และ $d\varepsilon_i / da_i = 1 / \sigma_i$

$$p_{a_i}(a_i) = p_\varepsilon\left(\frac{a_i - \mu_i}{\sigma_i}\right) \cdot \left|\frac{d\varepsilon_i}{da_i}\right| = \frac{1}{\sqrt{2\pi}} e^{-\frac{1}{2}\left(\frac{a_i - \mu_i}{\sigma_i}\right)^2} \cdot \frac{1}{\sigma_i} = \frac{1}{\sigma_i \sqrt{2\pi}} e^{-\frac{(a_i - \mu_i)^2}{2\sigma_i^2}}$$

ขวาสุดคือความหนาแน่นของระวังที่**กลาง** μ_i กว้าง σ_i เป๊ะ — **นี่คือเหตุผลที่สูตร Gaussian มี $(a_i - \mu_i) / \sigma_i$ อยู่ข้างใน**: มันคือการถอด a_i กลับไปเป็น ε_i แล้วถามระวังมาตรฐานใบเดียวว่าตรงนั้นสูงเท่าไร · พอนี้จะโผล่อีกครั้งที่ E7-E8

ระงักรธรรมเนียมการเขียน — ไม้ต้นเขียน $\mathcal{N}(\mu, \sigma)$ ซองสองเป็น ส่วนเบี่ยงเบนมาตรฐาน ตามช่อง `scale` ของ `torch` · ตำราส่วนใหญ่เขียน $\mathcal{N}(\mu, \sigma^2)$ ซองสองเป็น ความแปรปรวน · ระวังใบเดียวกัน คนละธรรมเนียม — เวลาอ่านที่อื่นให้ดูก่อนว่าเขาใช้แบบไหน

ทำเป็นเวกเตอร์ก็เดียวกันได้ แต่ต้องเปลี่ยนตัวคูณ — ถ้าเขียน $a = \mu + \text{diag}(\sigma)\epsilon$ ตัวคูณคือ $|\det(\partial\epsilon/\partial a)| = 1/\prod_i \sigma_i$ (ไม่ใช่ $\prod_i \sigma_i$ ซึ่งเป็น $\det$ ของ Jacobian ก็สไปหน้า $\text{diag}(\sigma)$) และ ไม่ใช่ $1/\sigma_i$ รายมิติ · ที่แยกทำทีละมิติก็เพื่อเลี่ยงจุดนี้

ในทางปฏิบัติก็เลี่ยงไม่ได้อยู่ดี — เครื่องสุ่มระดับล่างสุดผลิตได้แค่เลขสุ่มแบบสม่ำเสมอ แล้วแปลงเป็น $\mathcal{N}(0, 1)$ (Box-Muller / ziggurat) · $\mathcal{N}(\mu, \sigma)$ ได้มาจาก การเลื่อนและยืดเสมอ ไม่ว่าจะเขียนโค้ดแบบไหน

แบบรกรักที่ `rs1_r1` เรียกรจริง ไม่ได้เขียนสูตรนี้ตรง ๆ

`.sample()` กับ `.rsample()` เป็นคนละเมรอด และ Tron1 ใช้ตัวแรก

เมรอด	โค้ดใน torch	gradient ไหลผ่าน a ไหม	ใครใช้
<code>.sample()</code>	<code>with torch.no_grad(): torch.normal(loc, scale)</code>	ไม่ — ตัดทิ้งด้วย <code>no_grad</code>	Tron1 / PPO (:161)
<code>.rsample()</code>	<code>eps = _standard_normal(...)</code> แล้ว <code>return loc + eps * scale</code>	ไหล — ϵ เป็นค่าคงที่ ส่วน μ, σ ต่อ graph ได้	SAC · VAE (variational autoencoder)

`.rsample()` คือตัวที่เขียน $\mu + \sigma \odot \epsilon$ ตรงตัวอักษร · ที่มันเขียนแบบนั้น เพราะต้องการดันความสุ่มออกไปนอกเส้นทาง gradient (ϵ ไม่มี θ อยู่ข้างใน จึงหาอนุพันธ์ $\partial a / \partial \mu = 1$ ได้) — เรียกว่า **reparameterization trick**

PPO ไม่ต้องใช้ trick นี้เลย เพราะ gradient ของ PPO ไม่ได้ไหลผ่าน a มันไหลผ่าน $\log \pi_\theta(a | s)$ ต่างหาก · a ถูก `.detach()` ตั้งแต่ `ppo.py:139` แล้วถูกปฏิบัติเป็น ค่าคงที่ ตลอดทั้ง 20 gradient step

SAC ต่างออกไป — แต่ไม่ใช่ว่า "บังคับ" · actor loss ของ SAC มี $Q(s, a)$ อยู่ข้างใน ซึ่งเป็น NN ที่หาอนุพันธ์ได้ จึงเลือกใช้ `.rsample()` เพราะได้ estimator ที่ **แปรปรวนต่ำกว่า** แบบ likelihood-ratio · เปเปอร์ SAC (arXiv:1801.01290 §4.2) ใช้คำว่า *convenient* ไม่ใช่ *required* — ทำแบบ likelihood-ratio ก็ได้ แค่แยกว่า · อิมพลีเมนต์กระแสหลักทุกตัวจึงใช้ `rsample` กันหมด

ดังนั้นบรกรัก $a = \mu + \sigma \odot \epsilon$ ใน E6 อ่านว่า "นี่คือความหมายทาง คณิตศาสตร์ของการสุ่ม" ไม่ใช่ "นี่คือบรกรักที่ `.sample()` รัว" · การแจกแจง เหมือนกันเป๊ะ ต่างแค่ `.sample()` ไม่เก็บ ϵ ไว้ให้เห็น

☉ ทำงานยังงี้ — ดูด้วยตัวเลข 3 มิติ (ของจริง 8)

$$\mu = [0.20, -0.30, 0.10]$$

$$\sigma = [0.5, 0.5, 0.5]$$

$$\epsilon = [0.40, -0.20, 0.60] \quad \leftarrow \text{สุ่มใหม่ทุกคร้้ง}$$

$$\sigma \odot \epsilon = [0.5 \times 0.40, 0.5 \times (-0.20), 0.5 \times 0.60] = [0.20, -0.10, 0.30]$$

$$a = \mu + (\sigma \odot \epsilon) = [0.40, -0.40, 0.40]$$

ตรงกับสามแถวแรกของตารางใน §3.5.5 พอดี

ถ้าเป็น dot product ผลลัพธ์จะเป็นเลขตัวเดียว ($0.20 - 0.10 + 0.30 = 0.40$) ซึ่งผิด เพราะเราต้องการ action 8 ค่า ไม่ใช่ค่าเดียว — นี่คือเหตุผลที่ต้องใช้ $\odot$ ไม่ใช่ $\cdot$

I_8 บอกอะไร — ช่องที่สองของ $\mathcal{N}$ ในโลกหลายมิติคือ *เมทริกซ์ความแปรปรวนร่วม* ไม่ใช่ตัวเลขเดี่ยว อ่านได้สองอย่าง:

- **เล่นกเียงเป็น 1 ทั้งหมด** → ทุกมิติกว้างเท่ากัน ($\sigma = 1$)
- **นอกเล่นกเียงเป็น 0 ทั้งหมด** → แต่ละมิติ **สุ่มอิสระ** จากกัน ข้อ 3 จะสุ่มได้เท่าไร ไม่เกี่ยวกับข้อ 1 เลย

ข้อหลังนี่แหละที่ทำให้ E9 บวกกันเฉย ๆ ได้ (**log** ของผลคูณ = ผลบวก ใช้ได้เฉพาะเมื่อ อิสระต่อกัน)

อ่านทั้งบรรทัดเป็นประโยคเดียว — "*action ที่จะเล่นจริง ได้จากการสุ่มออกมาจากก้อนหมอก ของ policy — ซึ่งทำได้โดยเอาค่ากลางที่ actor คิดไว้ บวกด้วยความกว้างคูณเลขสุ่มมาตรฐาน โดยสุ่มแยกกัน 8 ครั้ง ครั้งละข้อ*"

หมายถึงอะไร — หยิบท่าจริงออกมาจากก้อนหมอก

ทำไมทำไม่ — ถ้าเล่นแต่ท่าที่คิดว่าดีที่สุด จะไม่มีวันรู้ว่ามีท่าที่ดีกว่า · gradient ของ PPO คำนวณจาก "ลองแล้วผลออกมาต่างกัน" — ไม่มีความต่าง = ไม่มีอะไรให้เรียน

ทำไม deploy ไม่สุ่ม — ตอนนั้นเลิกเรียนแล้ว การสุ่มเหลือแต่โทษ ไม่มีประโยชน์

```
# torch/distributions/normal.py:70-73
def sample(self, sample_shape=torch.Size()):
    shape = self._extended_shape(sample_shape)
    with torch.no_grad():
        return torch.normal(self.loc.expand(shape), self.scale.expand(shape))
```

`no_grad` บรรทัดนี้สำคัญมาก: **α ไม่มีสายกลับไปหา θ** gradient ไหลผ่าน α ไม่ได้เลย — นี่คือเหตุผลทั้งหมดที่ PPO ต้องมี ρ ถ้าอยากให้ไหลได้ต้องใช้ `rsample()` (normal.py:75-78, $\alpha = \mu + \sigma \epsilon$ แบบ reparameterization) ซึ่ง `rsample` **ไม่ได้ใช้**

ตอน deploy ไม่ใช้การตั้ง $\sigma = 0$ — แต่คือการข้ามบรรทัดนี้ทั้งบรรทัด

ผลลัพธ์เหมือนกัน ($\alpha = \mu + 0 \odot \epsilon = \mu$) จึงเป็นภาพในหัวที่ถูก แต่กลไกต่างกัน และความต่างนี้สำคัญ:

	ตอนเทรน	ตอน deploy
สร้าง π (E5) · ใช้ E6	✓	ข้าม
σ	มีค่าจริง ลดลงเรื่อย ๆ	ไม่มีอยู่ — ไม่ได้ export av <code>policy.onnx</code>
ได้ action มายังไง	สุ่มจากระวัง	อ่าน output ของ NN ตรง ๆ

`act_inference` ไม่มีคำว่า `Normal` ไม่มี `sample` ไม่มี σ โผล่มาเลยแม้แต่ตัวเดียว · σ ตอนจบการเทรนไม่ได้เป็น 0 (อาจเหลือราว 0.2-0.3) แต่ค่ามันถูก **ทิ้ง** ไม่ได้ถูก ทำให้เป็นศูนย์

และตั้ง $\sigma = 0$ จริง ๆ ไม่ได้ด้วย เพราะสูตรความหนาแน่นระเบิดทันที — พจน์แรกหาร ด้วย $\sigma^2 = 0$ และพจน์ที่สองเป็น $\log 0 = -\infty \cdot \mathcal{N}(\mu, 0)$ ไม่ใช้การแจกแจง ที่เขียนเป็นความหนาแน่นได้ (มันคือเข็มแหลม Dirac) พุดให้ถูกคือ deploy เป็น **ลิมิต $\sigma \rightarrow 0$** ไม่ใช่การแทนค่า

ทำไมคืน μ แล้วสบายใจได้ — สำหรับ Gaussian ค่าเฉลี่ย = ฐานนิยม = จุดที่ความหนาแน่นสูงสุด เป็นจุดเดียวกันหมด การคืน μ จึงแปลว่า "เล่น action ที่ policy คิดว่า น่าจะเป็นที่สุด" พอดี · ถ้าใช้การแจกแจงที่เบ้ (เช่น tanh-normal ของ SAC) สองอย่างนี้ จะคนละจุด แล้วต้องเลือกว่าจะเอาอันไหน

ช่วง C — จาก π_θ ไปเป็น $\pi_\theta(a | s)$ — ทิศ ค

ประโยคเดียวของช่วงนี้: จากการกระทำ $\rightarrow$ ตัวเลขที่บอกว่า "ตอนนั้นเราชอบมันแค่ไหน" เก็บไว้เทียบทีหลัง

E7 · ความหนาแน่นรายมิติ

$$\mathcal{N}(a_i; \mu_i, \sigma_i) = \frac{1}{\sigma_i \sqrt{2\pi}} \exp\left(-\frac{(a_i - \mu_i)^2}{2\sigma_i^2}\right)$$

หมายถึงอะไร — "ถ้าเล่นซ้ำอีกครั้ง โอกาสที่จะได้ค่าประมาณนี้อีก หนาแน่นแค่ไหน" · สูงเมื่อ a ใกล้ μ ต่ำเมื่อไกล

ทำไมทำไม — PPO ทั้งอัลกอริทึมตั้งอยู่บนการเทียบ "ชอบเท่าไรตอนเก็บข้อมูล vs ชอบ เท่าไรตอนนี้" · ต้องมีมาตรวัดความชอบก่อน ถึงจะเทียบได้

ระวัง — มันคือความหนาแน่น ไม่ใช่ความน่าจะเป็น ค่าเกิน 1 ได้ (ปกติ 0.5 §2.3)

E8 · เข้า log แล้วแตกเป็นสามพจน์ — นี่คือนี่ที่ torch คำนวณจริง

$$\log \mathcal{N}(a_i) = \underbrace{-\frac{(a_i - \mu_i)^2}{2\sigma_i^2}}_{\text{พจน์ ระยะห่าง}} - \underbrace{\log \sigma_i}_{\text{พจน์ ความกว้าง}} - \underbrace{\frac{1}{2} \log 2\pi}_{= -0.9189 \text{ เสมอ}}$$

รับ a (B,8) + object $\rightarrow$ คืน (B,8) **ยังไม่ยุบ** · torch/distributions/normal.py:88-92

หมายถึงอะไร — สมการเดียวกับ E7 เป๊ะ ๆ แค่มองในโลกร log

ทำไมต้อง log — สามเหตุผล

1. คูณเลขที่น้อยกว่า 1 แปรตัว ได้ค่าเล็กจนคอมพิวเตอร์ปัดเป็นศูนย์ (underflow) · log แปลงการคูณเป็นการบวก ปัญหาหายไป
2. การหารใน ρ (E13) กลายเป็นการลบ ซึ่งเสถียรกว่ามาก
3. อนุพันธ์ออกมาสั้นและสวย — E15 หน้าตาเรียบง่ายได้เพราะบรรทัดนี้

อ่านสามพจน์ — พจน์ 1 = ลงโทษตามระยะห่างกำลังสอง · พจน์ 2 = ยิ่งกล้าลอง (σ ใหญ่) ยิ่งไม่มีทางหนาแน่น · พจน์ 3 = ค่าคงที่ **ไม่มีผลต่อ gradient เลย**

```

var = self.scale**2                    #  $\sigma^2$ 
log_scale = self.scale.log()           #  $\log \sigma$ 
return (
    -(value - self.loc) ** 2) / (2 * var)    #  $-(a-\mu)^2 / 2\sigma^2$       พจน์ 1
    - log_scale                            #  $-\log \sigma$       พจน์ 2
    - math.log(math.sqrt(2 * math.pi))      #  $-\frac{1}{2} \log 2\pi = -0.9189$  พจน์ 3
)

```

ที่มาของสามพจน์ — แค่ take log ของ E7 ตรง ๆ:

$$\mathcal{N}(a; \mu, \sigma) = \frac{1}{\sigma\sqrt{2\pi}} e^{-\frac{(a-\mu)^2}{2\sigma^2}} \xrightarrow{\log} \underbrace{-\frac{(a-\mu)^2}{2\sigma^2}}_{\text{จาก } e^{(\cdot)}} - \underbrace{\log \sigma}_{\text{จาก } 1/\sigma} - \underbrace{\frac{1}{2} \log 2\pi}_{\text{จาก } 1/\sqrt{2\pi}}$$

E9 · ยูน 8 มิติ — ตรงนี้ vector กลายเป็น scalar

$$\log \pi_{\theta}(a \mid s) = \sum_{i=1}^8 \log \mathcal{N}(a_i) \quad \Longleftrightarrow \quad \pi_{\theta}(a \mid s) = \prod_{i=1}^8 \mathcal{N}(a_i)$$

รับ (B,8) → คำนวณ (B,) · [actor_critic.py:164](#)

หมายถึงอะไร — รวมความชอบของ 8 ข้อเข้าเป็น "ความชอบต่อทำทั้งทำ"

ทำไม — เราให้เครดิตและลงโทษเป็น *ทำ* ไม่ใช่รายข้อ เพราะ reward มาจากการ เดินทั้งตัว ไม่ได้บอกว่าข้อไหนดีข้อไหนแย · การบวกใน log = การคูณความน่าจะเป็น ซึ่งถูกต้องเพราะสุ่มแต่ละข้ออิสระกัน

นี่คือที่มาของคำว่า scalar ในตารางสัญลักษณ์ S3

8 ข้อสุ่มอิสระต่อกัน ความหนาแน่นร่วมจึงเป็นผลคูณ และ log ของผลคูณคือผลบวก — `sum(dim=-1)` คือ $\prod$ ที่ปลอมตัวมา มิติ 8 หายที่บรรทัดนี้ทีเดียว

ช่วง D — log-prob เอาไปทำอะไรต่อ

ประโยคเดียวของช่วงนี้: จากมาตรวัดความชอบ → ทิศและแรงที่ต้องขยับ θ

E10 · เก็บของเก่าใส่ buffer

$$\log \pi_{\theta_{\text{old}}}(a \mid s) \longrightarrow \text{storage}$$

[ppo.py:147-149](#) · แซ่แข็งไว้ ไม่เปลี่ยนอีกจนจบ iteration

หมายถึงอะไร — จดไว้ว่า "ตอนที่เก็บข้อมูลชุดนี้ เราชอบทำนี่เท่านี้"

ทำไม — PPO เอาข้อมูลชุดเดียวมาใช้ซ้ำ 20 ครั้ง พอ θ ขยับไปเรื่อย ๆ ข้อมูล จะ "เก่า" ลง · ต้องมีหยุดพักไว้เป็นจุดอ้างอิงว่าเดินห่างจากจุดเก็บไปแค่ไหน

ถ้าไม่จด — คำนวณ ρ ไม่ได้ → clip ไม่ได้ → PPO กลายเป็น policy gradient ธรรมดาที่พังง่าย

E11 · ประกอบ π ใหม่ด้วย θ ที่ขยับแล้ว (ทำ E3-E5 ซ้ำ)

$$\pi_{\theta_{\text{new}}}(\cdot | s) = \mathcal{N}(\mu_{\theta_{\text{new}}}(s), \sigma)$$

ppo.py:205-210 — บรรทัดที่คนอ่านผ่านบ่อยที่สุด:

```
self.actor_critic.act(                                     # ← เรียกแล้วไม่รับค่ากลับ!  
    torch.cat((encoder_out_batch, obs_batch, commands_batch), dim=-1)  
)
```

เรียก `act()` แต่ไม่เอา return value เพราะที่ต้องการคือ **side effect** — ให้ `self.distribution` ถูกสร้างใหม่ด้วย θ ปัจจุบัน action ที่มันสุ่มออกมาถูกทิ้งทันที เพราะ PPO ต้องประเมิน a_{old} ตัวเดิมเท่านั้น บรรทัดนี้คือ **ทศ ก** ที่ถูกเรียกซ้ำ 20 ครั้งต่อ iteration

หมายถึงอะไร — หลัง θ ขยับ ความเห็นของ actor เปลี่ยนไปแล้ว ต้องปัดก้อนหมอก ก้อนใหม่ขึ้นมา

ทำไม — จะถามว่า "ตอนนี้ชอบท่าเดิมแค่ไหน" ได้ ต้องมี π ตัวใหม่ก่อน · นี่คือเหตุผลที่ E3 กับ E5 ถูกเรียกซ้ำทุก gradient step ไม่ใช่แค่ตอนเก็บข้อมูล

E12 · ประเมิน a ตัวเดิม ด้วย π ใหม่ (ทำ E8-E9 ซ้ำ)

$$\log \pi_{\theta_{\text{new}}}(a | s)$$

ppo.py:212-214 · a มาจาก buffer ไม่ได้สุ่มใหม่

หมายถึงอะไร — ถามคำถามเดิม ด้วย action ชุดเดิม แต่กับ policy ใหม่

ทำไม — เป็นการควบคุมตัวแปร · ถ้าสุ่ม action ใหม่ ตัวเลขสองตัวจะเทียบกัน ไม่ได้เลย เพราะเปลี่ยนสองอย่างพร้อมกัน (ทั้ง policy และ action)

E13 · อัตราส่วน

$$\rho = \frac{\pi_{\theta_{\text{new}}}(a | s)}{\pi_{\theta_{\text{old}}}(a | s)} = \exp(\log \pi_{\theta_{\text{new}}} - \log \pi_{\theta_{\text{old}}})$$

รับ (MB,) สองก้อน → คำนวณ ρ (MB,) · ppo.py:252-255

หมายถึงอะไร — "ตอนนี้ชอบท่านี้มากกว่าหรือน้อยกว่าตอนเก็บข้อมูล ที่เท่า" · $\rho = 1.0$ แปลว่าไม่เปลี่ยนเลย

ทำไม — สองเหตุผล (1) เป็นโมบิลิตี้วัดว่าเดินห่างจากข้อมูลไปแค่ไหน (2) ในทางคณิตศาสตร์มันคือน้ำหนัก *importance sampling* ที่แก้ความเอนเอียงจากการใช้ข้อมูลที่เก็บด้วย policy เก่า

เกิน 1.2 หรือต่ำกว่า 0.8 เมื่อไร = เดินไกลเกินกว่าที่ข้อมูลชุดนี้จะเชื่อถือได้แล้ว

E14 · loss

$$L^{\text{CLIP}} = \frac{1}{MB} \sum \max(-\hat{A}\rho, -\hat{A} \cdot \text{clip}(\rho, 0.8, 1.2))$$

หมายถึงอะไร — แปลง "ทำนายผิดแค่ไหน ($\hat{A}$)" + "ขยับไปเท่าไร (ρ)" เป็นคะแนน ที่ต้องทำให้ต่ำที่สุด

ทำไมมีเครื่องหมายลบ — optimizer ทำได้อย่างเดียวคือ "ทำ loss ให้เล็กลง" · ถ้าอยากได้ $\hat{A}$ สูง ต้องเขียนเป็น $-\hat{A}$

ทำไม max ของสองค่าที่เป็นลบ — คือการเลือกอันที่ *ให้รางวัลน้อยกว่า* = มองโลก ในแง่ร้ายไว้ก่อน · นี่คือหัวใจของคำว่า **Proximal**: ถ้าขยับเกินขอบไปแล้ว จะไม่ได้รางวัล เพิ่มอีก **gradient กลายเป็น 0 พอดี** (ไม้ต 07 §5.3)

E15 · gradient ย้อนกลับถึง μ

$$\frac{\partial L}{\partial \mu_i} = -\hat{A} \rho \frac{a_i - \mu_i}{\sigma_i^2} \quad (\text{กรณี ไม่โดน clip})$$

จากนั้น chain rule ต่อเข้าไปใน NN ถึง θ ทุกตัว · `loss.backward()` + `optimizer.step()`

หมายถึงอะไร — "ต้องขยับ μ ไปทางไหน และแรงแค่ไหน"

อ่านทีละส่วน

ส่วน	ความหมาย
$(a_i - \mu_i)$	ทิศ — ชี้จากค่ากลางไปหาค่าที่ลองมา
$\hat{A}$	ดีหรือแย่ และแค่ไหน — บวก = เข้าหา · ลบ = หลี
$1/\sigma_i^2$	ยิ่งกล่าลงน้อย ยิ่งขยับแรง — σ เล็ก แปลว่าการเบี่ยงจากค่ากลางนิดเดียวก็เป็นข้อมูลหนักแน่น
ρ	ถ่วงน้ำหนักความเกี่ยวข้อง ของข้อมูลชุดนี้กับ policy ปัจจุบัน

ทั้งอัลกอริทึมจบที่บรรทัดนี้ — ทุกสมการก่อนหน้ามีไว้เพื่อคำนวณ "ทิศและแรง" นี้ให้ถูก

3.5.5 ไม่แทนค่าจริงทั้งสาย

ตั้งต้น: หนึ่ง env · $\sigma = 0.5$ ทุกมิติ (แปลว่า $\ell = \ln 0.5 = -0.6931$) · $\hat{A} = +0.7975$ — นี่คือช่อง **e0 t0** ของตาราง 2×4 ขยายเป็น 8 ข้อต่อ: ข้อต่อที่ 1 คือช่องนั้นเป๊ะ ($\mu = 0.20, \varepsilon = +0.40$) ส่วนข้อต่อ 2-8 เลือกให้เห็นกรณีต่าง ๆ (อภิธานศัพท์หัวข้อ 8.6 · คำนวณด้วย `scripts/toy_answer_key.py` ส่วนท้าย และสคริปต์ §3.5.7)

E3 → E5 · ได้ π (สมการ (9))

$$\mu = [0.20, -0.30, 0.10, 0.40, -0.10, 0.60, -0.50, 0.30]$$

$$\sigma = \exp(-0.6931) = 0.5 \implies \pi_\theta(\cdot | s) = \mathcal{N}(\mu, 0.5)$$

E6 · สุ่มออกมาเป็น action — $a_i = \mu_i + 0.5 \varepsilon_i$ (สมการ (16))

i	μ_i	ε_i	a_i
1	+0.20	+0.40	+0.40
2	-0.30	-0.20	-0.40

3	+0.10	+0.60	+0.40
4	+0.40	0.00	+0.40
5	-0.10	-0.60	-0.40
6	+0.60	+1.20	+1.20
7	-0.50	-0.80	-0.90
8	+0.30	+0.20	+0.40

เวกเตอร์ 8 ตัวนี้คือสิ่งที่ถูกส่งไปคูณ 0.25 แล้วเข้า PD controller (สมการ (17)(18)) — **จบหน้าที่ ผังควบคุม** · สังเกตข้อต่อ 4: $\varepsilon = 0$ พอดี $a = \mu$ — จะเห็นผลใน E8

E8 · ความหนาแน่นรายมิติ (สมการ (14)) — ยกมิติที่ 1 มาทำเต็ม ๆ ก่อน: $a_1 - \mu_1 = 0.40 - 0.20 = +0.20$

$$\log \mathcal{N}(a_1) = -\frac{(0.20)^2}{2(0.5)^2} - \log(0.5) - 0.9189 = -0.0800 + 0.6931 - 0.9189 = -0.3058$$

(ตัวเลขเดียวกับ $\log \pi_{old}$ ของ e0 t0 ในโน้ต 07 §2.2 — เพราะข้อต่อ 1 คือช่องนั้น)

ครบทั้ง 8 มิติ (พจน์ 2 และ 3 เท่ากันหมด เปลี่ยนแค่พจน์แรก):

i	$a_i - \mu_i$	พจน์ 1	พจน์ 2	พจน์ 3	$\log \mathcal{N}(a_i)$
1	+0.20	-0.0800	+0.6931	-0.9189	-0.3058
2	-0.10	-0.0200	+0.6931	-0.9189	-0.2458
3	+0.30	-0.1800	+0.6931	-0.9189	-0.4058
4	0.00	0.0000	+0.6931	-0.9189	-0.2258
5	-0.30	-0.1800	+0.6931	-0.9189	-0.4058
6	+0.60	-0.7200	+0.6931	-0.9189	-0.9458
7	-0.40	-0.3200	+0.6931	-0.9189	-0.5458
8	+0.10	-0.0200	+0.6931	-0.9189	-0.2458

มิติที่ 6 สุ่มไปไกลจาก μ ที่สุด (+0.60 = 1.2σ) จึงได้ค่าติดลบมากที่สุด — **ยิ่งไกลจาก ค่ากลางยิ่งหนาแน่นน้อย** · มิติที่ 4 อยู่ที่ยอดพอดี ได้ค่าสูงสุดที่เป็นไปได้ $-0.2258 = \log 0.7979$ (ความสูงยอดระฆังของโน้ต 05 §2.3)

E9 · ยุบเหลือเลขเดียว (สมการ (15))

$$\log \pi_\theta(a | s) = \sum_{i=1}^8 \log \mathcal{N}(a_i) = -3.3263 \quad \Rightarrow \quad \pi_\theta(a | s) = e^{-3.3263} = 0.0359$$

จาก 8 ตัวเหลือ 1 ตัวที่บรรทัดเดียว (.sum(dim=-1)) และ **-3.3263** นี้แหละที่ถูกเก็บใส่ buffer ตาม E10 · สังเกตว่าความหนาแน่นของ "ท่าทั้งตัว" (0.036) เล็กกว่าของข้อต่อเดียว (0.7365) มาก เพราะเป็นผลคูณ 8 ตัวที่แต่ละตัวต่ำกว่า 1

E11 → E12 · หลัง Adam ขยับ θ แล้วมาประเมินใหม่ — สมมติทุกข้อต่อขยับ +0.05 (กฎเดียวกับโน้ต 07 §5.2):

$$\mu_{\text{new}} = [0.25, -0.25, 0.15, 0.45, -0.05, 0.65, -0.45, 0.35]$$

a ยังเป็นชุดเดิมจากตารางข้างบน คำนวณ E8-E9 ขึ้นด้วย μ_{new} ได้ $\log \pi_{\theta_{\text{new}}}(a | s) = -3.2863$

E13 • อัตราส่วน (สมการ (21))

$$\rho = \exp(-3.2863 - (-3.3263)) = \exp(0.0400) = 1.0408$$

อ่านว่า "policy ใหม่ชอบ action ชุดนี้มากกว่าเดิม 4.1%" — ยังไม่เกินขอบ $[0.8, 1.2]$ จึงไม่โดน clip · เทียบกับข้อต่อเดียวในไนต์ 07 ($\rho = 1.0356$): 8 ข้อต่อขยับพร้อมกัน ρ รวม คือผลคูณ 8 ตัว — บางมิตมากกว่า 1 บางมิตน้อยกว่า (มิตที่ $a < \mu$ ถูกขยับหนี) หักล้างกันแล้วผลรวม ของ log เป็น +0.0400 มากกว่าของมิตเดียว (+0.0350) เล็กน้อย

E14 • loss (สมการ (29))

$$-\hat{A}\rho = -0.7975 \times 1.0408 = -0.8300 \quad -\hat{A} \cdot \text{clip}(1.0408) = -0.8300$$

$$L^{CLIP} = \max(-0.8300, -0.8300) = -0.8300$$

E15 • gradient กลับถึง μ แต่ละตัว (สมการ (24) ผ่าน ρ — ไนต์ 07 สมการ (G))

$$\frac{\partial L}{\partial \mu_i} = -0.7975 \times 1.0408 \times \frac{a_i - \mu_i^{\text{new}}}{0.25}$$

i	$a_i - \mu_i^{\text{new}}$	$\partial L / \partial \mu_i$	Adam จะ...
1	+0.15	-0.498	เพิ่ม μ_1
2	-0.15	+0.498	ลด μ_2
3	+0.25	-0.830	เพิ่มแรง μ_3
4	-0.05	+0.166	ลดนิดเดียว — μ_4 ขยับ +0.05 เลย a_4 ไปแล้ว จึงถูกดึงกลับ
5	-0.35	+1.162	ลด μ_5
6	+0.55	-1.826	เพิ่มแรงสุด μ_6
7	-0.45	+1.494	ลด μ_7
8	+0.05	-0.166	เพิ่มนิดเดียว

อ่านตารางนี้ให้ออกคือเข้าใจ PPO ทั้งหมด — เพราะ $\hat{A} = +0.7975 > 0$ แปลว่า "ครั้งนั้น ทำได้ดีกว่าที่ critic คาด" gradient จึงลาก μ เข้าหา a ที่สุ่มไป ทุกมิต (เครื่องหมายของ $\partial L / \partial \mu_i$ ตรงข้ามกับ $a_i - \mu_i^{\text{new}}$ เสมอ) และมิตที่ยังห่างที่สุด (มิต 6) ถูกลากแรงที่สุด · ถ้า $\hat{A}$ เป็นลบ เครื่องหมายทั้งคอลัมน์จะกลับหมด → ผลัก μ หนีจาก a

ของแถมสองตัวจากระวัง 8 มิตใบนี้: entropy = $8 \times 0.7258 = 5.8063$ (สมการ (31)) และ KL หลังขยับ = $8 \times 0.0050 = 0.040 > 0.02 \rightarrow \text{lr} \div 1.5$ ตามสมการ (37) — ขยับ 0.05 "ทุกข้อต่อพร้อมกัน" ไกลกว่าขยับข้อต่อเดียว (0.005 ในไนต์ 07 §5.7) แปลเท่า

3.5.6 แผนที่การส่งค่า — ตารางเดียวจบ

จาก	ผ่าน	ไปเป็น	shape เปลี่ยนยังไร	สมการ	โค้ด
h	E1	$\hat{v}$	$(B, 360) \rightarrow (B, 3)$	— (ไนต์ 05 §4.4)	ppo.py:136
$\hat{v}, o, c$	E2	s	$\rightarrow (B, 42)$	—	ppo.py:138
s	E3	μ	$(B, 42) \rightarrow (B, 8)$	(10) (11)	actor_critic.py:156
ℓ	E4	σ	$(8,) \rightarrow (8,)$	(9)	actor_critic.py:157

จาก	ผ่าน	ไปเป็น	shape เปลี่ยนยังไง	สมการ	โค้ด
μ, σ	E5	π	tensor $\rightarrow$ object	(9)	actor_critic.py:157
π	E6	a	object $\rightarrow$ (B, 8)	(16)	actor_critic.py:161
π, a	E7 E8	$\log \mathcal{N}$	$\rightarrow$ (B, 8)	(13) (14)	normal.py:88-92
$\log \mathcal{N}$	E9	$\log \pi$	(B, 8) $\rightarrow$ (B,)	(15)	actor_critic.py:164
$\log \pi$	E10	buffer	เก็บไว้	—	ppo.py:147
s (เดิม)	E11	π_{new}	ทำ E3 E5 ซ้ำ	(9)	ppo.py:205
π_{new}, a	E12	$\log \pi_{\text{new}}$	$\rightarrow$ (MB,)	(14) (15)	ppo.py:212
สอง $\log \pi$	E13	ρ	$\rightarrow$ (MB,)	(21)	ppo.py:252
$\rho, \hat{A}$	E14	L	(MB,) $\rightarrow$ scalar	(29) $\rightarrow$ (33)	ppo.py:260
L	E15	∇_{θ}	scalar $\rightarrow$ 469,905 ตัว (θ, ϕ, ℓ)	(24) (35) $\cdot$ ไม้ต 07 (G)	.backward()

กฎอ่านที่สรุปได้จากตารางนี้

1. μ กับ σ เป็น tensor $\cdot$ π เป็น object — บรรทัดไหนที่ผลลัพธ์ไม่ใช่ tensor บรรทัดนั้นคือการประกอบ π
2. ทุกครั้งที่ s หรือ θ เปลี่ยน ต้องประกอบ π ใหม่ — จึงมี update_distribution ทั้งใน act() ตอนเก็บข้อมูล และซ้ำอีกใน ppo.py:205 ตอนอัปเดต
3. มิติ 8 หายตรงไหน = ตรงนั้นมีการคูณความน่าจะเป็น — sum(dim=-1) ที่ actor_critic.py:164 (log-prob) และ :153 (entropy) คือ $\prod_{i=1}^8$ ในโลก log
4. มิติ B/MB หายตรงไหน = ตรงนั้นคือ .mean() ของ loss — ppo.py:260 สำหรับสาย E1-E15 (ส่วน :269 และ :273 ยุบ MB ของอีกสองพจน์ใน (33)) หลังจากนั้นเหลือเลขตัวเดียวให้ .backward()

ตอน deploy ข้าม E4-E9 ทั้งหมด — act_inference ทำแค่ self.actor(obs) แล้ว return ไม่แตะ Normal เลย (actor_critic.py:166-168) ดู [diagrams/tron1-deploy.html](#) ประกอบ

3.5.7 สคริปต์ตรวจด้วยตัวเอง

รับได้เลย ไม่ต้องมี torch — ตัวเลขทุกตัวใน §3.5.5 ออกมาจากสคริปต์นี้ (เหมือนส่วนท้ายของ scripts/toy_answer_key.py):

```

import math

MU      = [0.20, -0.30, 0.10, 0.40, -0.10, 0.60, -0.50, 0.30]    # ข้อต่อ 1 = e0 t0
EPS      = [0.40, -0.20, 0.60, 0.00, -0.60, 1.20, -0.80, 0.20]
SIGMA    = 0.5
A_HAT    = 0.7975                                                #  $\hat{A}$  ของ e0 t0 (อภิสานคัพพ 8.4)
C        = math.log(math.sqrt(2 * math.pi))                    # 0.9189 – พจน์ที่ 3 ของ E8

def pdf(x, mu, sigma):                                           # E7 – สมการ (13)
    return math.exp(-((x - mu) ** 2) / (2 * sigma ** 2)) / (sigma * math.sqrt(2 * math.pi))

def log_pdf(x, mu, sigma):                                       # E8 – สมการ (14), ตรงกับ normal.py:88-92
    return -((x - mu) ** 2) / (2 * sigma ** 2) - math.log(sigma) - C

a        = [m + SIGMA * e for m, e in zip(MU, EPS)]              # E6 – สมการ (16)
log_old   = sum(log_pdf(ai, mi, SIGMA) for ai, mi in zip(a, MU)) # E9 – สมการ (15) → -3.3263

MU_NEW    = [m + 0.05 for m in MU]                              # E11 – ทุกข้อต่อขยับ +0.05
log_new    = sum(log_pdf(ai, mi, SIGMA) for ai, mi in zip(a, MU_NEW)) # E12 → -3.2863

rho       = math.exp(log_new - log_old)                         # E13 – สมการ (21) → 1.0408
loss      = max(-A_HAT * rho, -A_HAT * min(max(rho, 0.8), 1.2)) # E14 – สมการ (29) → -0.8300
grad      = [-A_HAT * rho * (ai - mi) / SIGMA ** 2             # E15 – (24) ผ่าน p
              for ai, mi in zip(a, MU_NEW)]

print(f"a          = {[round(x, 3) for x in a]}")
print(f"log_old    = {log_old:.4f}   pi = {math.exp(log_old):.5f}")
print(f"log_new    = {log_new:.4f}")
print(f"rho        = {rho:.4f}")
print(f"L_CLIP     = {loss:.4f}")
print(f"dL/dmu     = {[round(g, 4) for g in grad]}")

```

เปลี่ยน `A_HAT` เป็นค่าลบ (เช่น **-0.9290** ของ e1 t1) แล้วรันใหม่ — เครื่องหมายของ `dL/dmu` จะกลับทั้งแถว นั่นคือกลไก "พลิก μ นีจาก action ที่แย่" · เปลี่ยน `SIGMA` เป็น 0.25 แล้ว ขยับ `MU_NEW` ให้ห่างขึ้น จะเห็น `rho` ทะลุ 1.2 แล้ว clip เริ่มทำงาน (โน้ต 07 §5.3)

4. errata ของเอกสารต้นทาง — `sources/training_numerical_walkthrough.md`

เอกสารต้นทาง (Gemini/antigravity, 2026-09-09) ถูกย้ายไป `sources/` พร้อมหัวข้อ errata ที่ระบุทุกจุดผิดพลาดพร้อมเลขบรรทัด · โดยรวมมัน**ถูกต้อง**ในเชิงโครงสร้าง shape และรูปสูตร แต่มี สิ่งที่ต้องรู้ก่อนเปิดมัน:

4.1 ตัวเลข "ค่า" ทั้งหมดเป็นตัวอย่างสมมติ

$\mu = 0.42$, `log_prob = -2.41`, $V = 650.0$, $V_{\text{last}} = 682.3$, $\delta = 28.15$, $p = 1.25$ ฯลฯ **ไม่ได้มาจาก run จริง** — เป็นตัวเลขที่แต่งขึ้นให้ไล่คำนวณตามได้ (และ `-2.41` ไม่ตรงกับ $\mathcal{E}$ ของมันเอง ซึ่งให้ **-2.47**) · โน้ตชุดนี้ใช้ตาราง 2×4 แถวทั้งหมด (โน้ต 07) และเก็บตัวเลขสเกล Tron1 ของเอกสารนั้น ($V \approx 650$, $\delta \approx 28$, $L_V \approx 781$) ไว้แค่เป็นภาพขนาดจริงในโน้ต 07 §6 · ค่าจริงจะได้จากการรัน `exp_001` แล้วดู tensorboard

4.2 จำนวน parameter คำนวณผิด

เอกสารบอก actor $\approx 185,480$ · critic $\approx 250,625$ · รวม $\approx 436K$ · คำนวณใหม่จากขนาดชั้นจริง (ทุก `Linear(in, out)` มี `in×out weight + out bias`) และยืนยันจาก checkpoint จริง:

network	ชั้น	คำนวณ	จริง	เอกสารต้นทาง
actor	42 → 512 → 256 → 128 → 8	$43 \cdot 512 + 513 \cdot 256 + 257 \cdot 128 + 129 \cdot 8$	187,272	185,480 ✗
critic	230 → 512 → 256 → 128 → 1	$231 \cdot 512 + 513 \cdot 256 + 257 \cdot 128 + 129 \cdot 1$	282,625	250,625 ✗
encoder	360 → 256 → 128 → 3	$361 \cdot 256 + 257 \cdot 128 + 129 \cdot 3$	125,699	~129K (ใกล้)
logstd	—	—	8	8 ✓
optimizer หลัก ($\theta + \phi$)		$187,272 + 282,625 + 8$	469,905	~436K ✗
รวมที่ฝึก		+ encoder	595,604	(ไม่รวม encoder)

ประเด็นเชิงหลักการที่เอกสารต้องการสื่อ ("gradient คือเวกเตอร์ยาวหลักแสน") ยังถูก — แค่ตัวเลขเป๊ะผิด

4.3 สิ่งที่เอกสารพูดถึงและตอนนี้ไม่ต 05 มีครบแล้ว

เรื่อง	อยู่ที่
advantage normalization	ไม่ต 05 §4.5 สมการ (28)
value clipping (<code>use_clipped_value_loss = True</code> , ใช้ <code>clip_param</code> 0.2 ตัวเดียวกับ policy — เทคนิคเสริมของ <code>rsl_rl</code> ไม่มีในเฟรมเวิร์ก PPO)	ไม่ต 05 §4.6 (จ) สมการ (32)
<code>clip_grad_norm(1.0)</code>	ไม่ต 05 §2.2 Remark · §3.4 ของไม่ตนี้
mask ($1 - \text{done}$) ใน GAE	ไม่ต 05 §3.2 สมการ (19), §4.5 สมการ (26)
encoder loss แบบ MSE กับ <code>base_lin_vel</code>	ไม่ต 05 §4.4

4.4 การเขียน shape ของ σ

เอกสาร §1.8 เขียน `sigma shape = (24, 2048, 8)` ในตาราง buffer แต่ §5 cheat sheet เขียน `(8,) broadcast → (B, 8)` — **ทั้งคู่ถูก** ตัวจริงคือพารามิเตอร์ 8 ตัว (ไม่ขึ้นกับ state — ไม่ต 05 หัวข้อ 2.1 remark) แต่ตอนเก็บลง buffer มันถูก broadcast เป็น `(B,8)` ต่อ step แล้ว (`rollout_storage.py:146`) · และเอกสารเขียน $\mathcal{N}(\mu, \sigma^2)$ ตามตำรา ส่วนไม่ตชุดนี้ เขียน $\mathcal{N}(\mu, \sigma)$ ตาม `scale` ของ torch (อภิธานศัพท์หัวข้อ 7)

4.5 เลขสมการอ้างอิงระบบของตัวเอง

เอกสารอ้าง "สมการ (3), (5), (6), (1b)" ตามการเรียงเลขของมันเอง · ตารางแปลงไปเป็นเลขเดียว ทั้งชุด: (1b)→(12), (3)→(4), (5)→(26), (6)→(29) — อภิธานศัพท์หัวข้อ 9

5. อ่านไดอะแกรมหกชิ้นยังไ้

A — Actor-Critic uu Tron1 (`diagrams/tron1-actor-critic.html`)

สิ่งที่เห็น	หมายถึง	ไม่ต 05
เค้าโครง agent uu · environment ล่าง · s_t, r_t ขึ้นซ้าย · a_t ลงขวา	รูป agent–environment มาตรฐานของ Sutton & Barto (Fig. 3.1) ที่ทุกตำรา RL ใช้	1.1, 1.2

สิ่งที่เห็น	หมายถึง	ไม้ต 05
ลูกศรทึบ	ข้อมูลไหลไปข้างหน้า (forward pass) — เกิดทุก step ทั้งตอนเทรนและ deploy	2.5
ลูกศรประ $\nabla\theta, \nabla\phi$	สัญญาณการเรียนรู้ — เกิดเฉพาะช่วง C ของ iteration · นี่คือ 2 เส้นเดียวที่ทำให้ระบบ "เรียนรู้"	2.2 สมการ (12)
กล่องประ (Critic, PPO)	มีชีวิตแค่ตอนเทรน · ถูกทิ้งตอน export ONNX	2.6
กล่องส้ม Advantage	จุดที่ critic "สอน" actor — ถ้าตัดกล่องนี้ออก จะเหลือแค่ policy gradient ธรรมดาที่ ความแปรปรวน (variance) สูง คือ gradient เหวี่ยงไปมาจนเรียนช้า	4.1, 4.2, 4.5
encoder ไม่มีกล่องแยก	ยุบเป็น sublabel ใน Actor เพราะ budget ของไดอะแกรม — โครงสร้างเต็มอยู่ในไม้ต 05 §4.4	4.4

คำถามที่ควรถามตัวเองตอนดูรูป A: "ถ้าลบลูกศรประสองเส้นออก ระบบนี้จะป็นอะไร?" — คำตอบ: เป็นหุ่นที่เดินด้วย policy คงที่ ไม่เรียนรู้ = สิ่งที่รันบนหุ่นจริง

B — หนึ่ง iteration ของ PPO (diagrams/tron1-ppo-iteration.html)

สิ่งที่เห็น	หมายถึง	ไม้ต 05
ลำดับ $A \rightarrow B \rightarrow C \rightarrow D$	สี่ช่วงของกายวิภาค iteration	4.7
กล่องส้ม C	ที่เดียวที่ weight เปลี่ยน — A, B, D ไม่แตะ θ เลย	4.7, 2.2
ลูกศรส้มวนกลับ	ความหมายของคำว่า iteration · ป้าย $\theta_{old} \leftarrow \theta$ = policy ที่เพิ่งอัปเดตกลายเป็น "ตัวเก็บข้อมูล" รอบถัดไป	3.3 p, 4.7
จุดรวม ก่อน A	จุดที่รอบแรก (จาก Start) และรอบต่อ ๆ ไป (จาก loop) มาบรรจบ — ไวยากรณ์มาตรฐานของ flowchart	—
ข้าวหลามตัด it = 15 000?	<code>max_iterations</code> · ทางออก NO คือ "ทางปกติ" — 14,999 ครั้งจาก 15,000	4.7

ความสัมพันธ์ระหว่าง A กับ B: รูป A คือ *สิ่งที่มียู่* รูป B คือ *ลำดับที่มันถูกใช้* — ช่วง A ของ B ใช้ลูกศรทึบทั้งหมดในรูป A · ช่วง B ของ B คือกล่อง GAE ในรูป A · ช่วง C ของ B คือลูกศรประสองเส้นในรูป A

C — สายส่งค่า E1-E15 (diagrams/tron1-value-flow.html)

สิ่งที่เห็น	หมายถึง	หัวข้อในไม้ตนี้
โซ่แนวตั้ง 15 กล่อง	หนึ่งกล่อง = หนึ่งสมการ · บรรทัดล่างของกล่องคือบรรทัดโค้ดที่ทำสมการนั้น	§3.5.4
ป้ายข้าง ลูกศร	shape ที่ไหลผ่านจริง — ไล่ตามได้ว่ามิติหายตรงไหน	§3.5.6
กล่องส้ม E5	tensor กลายเป็น distribution object — ทิศ ก	§3.5.3
กล่องส้ม E9	8 มิติยุบเหลือเลขเดียว (<code>sum(dim=-1)</code>) — จุดที่ vector กลายเป็น scalar	§3.5.4 ช่วง C
กล่องซ้าย 2 ใบ	ทางแยกออกจากโซ่หลัก: อ่าน μ, σ กลับ (ทิศ ข) และ action ที่ออกไปคุมหุ่น	§3.5.3
เส้นประวนกลับ E15 $\rightarrow$ E3	θ เปลี่ยน $\rightarrow \mu$ ใหม่ $\rightarrow$ เริ่ม E3 อีกครั้ง · 20 รอบต่อ iteration	§3.5.4 E11

รูปนี้จึงใจเกินพีดาน 9 กล่องที่ skill diagram-design แนะนำ (พีดานความหนาแน่นของ ไดอะแกรมหนึ่งรูป) เพราะมันเป็น *แผ่นอ้างอิง* ไม่ใช่ แผนภาพสรุป — ใช้คู่กับ §3.5 ไม่ใช่เดี่ยว ๆ

D — Tron1 ตอน deploy (diagrams/tron1-deploy.html)

สิ่งที่เห็น	หมายถึง	ไม้ต 05
แถบบนที่อาจทั้งแถบ	critic, GAE, PPO ไม่ได้ถูก export — ไม่มีอยู่บนหุ่นจริง	2.6
ลูกศรสามเส้นประชี้ลง	คำตอบของคำถาม "advantage ไปไหนตอน deploy" — มันถูกใช้หมดไปแล้ว เหลือเป็นตัวเลขใน θ	4.5, 4.6
encoder.onnx + policy.onnx แยกกัน	export เป็นสองไฟล์ โค้ดฝั่งหุ่นเป็นคน cat เอง (play.py:111-121)	2.6
ไม่มี σ ไม่มีการสุ่ม	act_inference คือน μ ตรง ๆ (actor_critic.py:166-168) — ข้าม E4-E9 ทั้งหมด	2.4, 2.6

เทียบ A กับ D: รูป A คือ *ตอนเรียน* รูป D คือ *ตอนสอบ* — กล้องที่หายไปคือทุกอย่าง ที่มีไว้เพื่อ "รู้ว่าทำได้ดีแค่ไหน" ซึ่งไม่จำเป็นอีกเมื่อหยุดเรียนแล้ว

E — สองชั้นจาก policy ถึง action (diagrams/tron1-policy-objects.html)

สิ่งที่เห็น	หมายถึง	หัวข้อในไม้ตนี้
วงเล็บบนสุด 2 อัน	ชั้นที่ 1 (เรียกฟังก์ชัน) กับ ชั้นที่ 2 (สุ่ม/แทนค่า) — เหตุผลที่ต้องมีสัญลักษณ์หลายแบบ	§3.5.0
กรอบเส้นประ 4 ช่อง	ช่วงนั้นเป็นวัตถุชนิดอะไร: ข้อมูล · NN ที่เรียนได้ · อ็อบเจกต์ชั่วคราว · ผลลัพธ์	§3.5.0 ตาราง "แต่ละช่วง"
กล่อง = วัตถุ · ลูกศร = สิ่งที่เกิดขึ้นระหว่างกล่อง	ไวยากรณ์ของรูปทั้งรูป · ป้ายบนลูกศรบอกทั้งสองอย่าง — ค่าที่ไหลผ่าน (s (42,), a) และบรรทัดโค้ดที่ทำให้เกิด (:156, :157, :161, :164)	§3.5.0
shape ในรูปเป็น "ต่อ 1 env"	ของจริงมี B นำหน้าทุกก้อน · self.distribution ก็ละยัง $B \times 8$ ในพร้อมกัน ไม่ใช่ใบเดียว · .log_prob(a).sum(-1) คือน (B,) ไม่ใช่ scalar เดี่ยว	§3.5.0 กรอบ "shape"
กล่องสาม $\pi_\theta(\cdot s)$	จุดที่คนสับสนที่สุด — ชั้นที่ 1 คือน s ยัง ไม่ได้คือน action	§3.5.0, §3.5.2
สองทางแยกขวาสุด	.sample() ได้เวกเตอร์ · .log_prob() ได้เลขตัวเดียว	§3.5.3 ทิศ ด (ทางล่าง) · การสุ่มไม่ได้อยู่ในสามทิศนั้น — ดู E6
ลูกศรสั้นจากกล่องบนลงกล่องล่าง	.log_prob() ไม่ผลิต a เอง — ตอนเก็บข้อมูล a คือตัวที่เพิ่งสุ่มจากกล่องบน (ตอนอัปเดตมาจาก buffer แทน)	§3.5.0 ตาราง "a มาจากไหน"
วงเล็บสองอันล่างสุด	ขอบเขตของคำว่า policy — ตอนเทรนกิน ①+②+③ · ตอน deploy เหลือ ① · นี่คือเหตุผลที่ "เรียก NN ว่า policy" ถูกบ้างผิดบ้าง	§3.5.0 ตาราง "ตอนไหน"
③ อยู่บนลูกศร ไม่ใช่ในกล่อง	สูตร $\mathcal{N}$ มี 0 weight — มันเป็นบรรทัดโค้ด (:157) ไม่ใช่วัตถุ	§3.5.0 "สามชั้น"

เทียบ E กับ C: รูป E ตอบว่า *แต่ละก้อนเป็นของชนิดอะไร* · รูป C ตอบว่า *ค่าไหลผ่านก้อนพวกนั้นเป็นลำดับอย่างไร* — E คือหน้าตัดขวาง C คือเส้นทางยาว

F — ห้าหน่วยนับ หนึ่งตาราง (diagrams/tron1-five-units.html)

สิ่งที่เห็น	หมายถึง	ไม้ต 05 / 07
ตาราง 24 × 2048	rollout หนึ่งใบ = ข้อมูลของหนึ่ง iteration (49,152 ช่อง) · ตาราง 2 × 4 ของไม้ตชุดนี้คือมันนับย่อย	05 §4.7, 07 §0
หนึ่งช่อง / คอลัมน์จนถึงแถบ done / ทั้งใบ	step / episode / iteration — สามหน่วยที่ซ้อนกันคนละแบบ	05 §1.4, §4.7

สิ่งที่เห็น	หมายถึง	โน้ต 05 / 07
ฟังขวา: GAE ครั้งเดียว → 20 gradient step (5 epoch × 4 mb) → clear	สิ่งที่เกิดกับตารางหนึ่งใบ: epoch = เดินครบทุกช่อง · mini-batch = 1/4 ของ ตาราง = หนึ่ง gradient step · การสับและการหั่นเห็นได้จากช่องสีเทาในตารางฟังซ้าย ไม่มีกล่องแยก	05 §4.5, §4.7 · 07 §4-5
แถบ done ในคอลัมน์ 11	episode จบกลางตาราง แล้ว episode ใหม่เริ่มในคอลัมน์เดิม — เหมือน env 1 ของตาราง 2 × 4	05 §3.2 (mask), 07 §3

เทียบ F กับ B: รูป B คือลำดับเวลาของหนึ่ง iteration · รูป F คือก้อนข้อมูลทีลำดับนั้น ทำงานด้วย — ช่วง A ของ B เติม ตาราง F · ช่วง B-C ของ B คือฟังขวาของ F

6. ลำดับอ่านที่แนะนำ

สำหรับคนที่อ่านโน้ต 05 จบแล้วและอยากเห็นภาพรวมทั้งหมด:

- เปิดรูป **A** ดู 2 นาที — ไล่ชื่อลูกศรทุกเส้นให้ได้ว่าแต่ละตัวคืออะไรโดยไม่ดูโน้ต ถ้าติดตัวไหน → อภิธานศัพท์ (ชนิดก่อนคำที่หลัง) - ถ้าสงสัยว่า "ตกลงแต่ละก้อนในรูปเป็นของชนิดอะไรกันแน่" → เปิดรูป **E** (`diagrams/tron1-policy-objects.html`) คู่กับ §3.5.0 ก่อนไปต่อ
- อ่านโน้ต 07 §1-2 (ตั้งค่า toy · ระดับ step) โดยเปิดตาราง §2 ของโน้ตนี้คู่กัน — ทุกชั้น ชี้กลับไปโน้ต 05 และได้ด
- เปิดรูป **B** และ **F** — ตอนนี้ควรอ่านกล่อง A ได้ครบทุกบรรทัด และบอกได้ว่าห้าหน่วยนับคือ ห้าวิธีหั่นตารางเดียวกัน
- อ่านโน้ต 07 §4-5 ไล่ตัวเลข GAE และ ρ ด้วยมือ (`scripts/toy_answer_key.py` เช็ก) - ถ้ายังไม่แน่ใจว่า "ค่าไหนไปเข้าบรรทัดไหน" → §3.5 ไล่ E1-E15 ทีละสมการ (มีคำอธิบาย หมายถึงอะไร / ทำไปทำไม ทุกข้อ) พร้อมรันสคริปต์ §3.5.7 แล้วเทียบ กับรูป **C** (`diagrams/tron1-value-flow.html`) - ถ้าสงสัยว่า ℓ ถูกเขียนกับได้ยังไง หรือ lr เปลี่ยนเพราะอะไร → §3.4
- กลับมารูป **A** อีกครั้ง — คราวนี้มองลูกศรประ $\nabla\theta$ แล้วนึกถึง 20 gradient steps ที่เพิ่งไล่มา
- รัน `exp_001` แล้วเปิด tensorboard — เทียบ `Policy/mean_noise_std` กับ σ ในอภิธานศัพท์ และ `Policy/mean_kl` กับ `desired_kl` — ตัวเลขจริงชุดแรกของคุณ จะเข้ามาแทนตัวเลขของ toy

ตรวจกับโค้ดจริง 2026-09-11: `ppo.py`, `actor_critic.py`, `rollout_storage.py`, `on_policy_runner.py`, `limx_base_env_cfg.py`, `limx_rsl_rl_ppo_cfg.py` ใน `tron1-rl-isaacsim/` และ `joint_actions.py` ใน `IsaacLab/` — ลิงก์เป็น relative path ใช้ได้ในเครื่องที่ clone ทั้งสอง repo ไว้ · ตัวเลขของตาราง 2 × 4 และเวกเตอร์ 8 มิติคำนวณด้วย `scripts/toy_answer_key.py`